The Topological Heterogeneous Manifold Engine (THME)

A Proposed Post-Transformer Architecture Combining Ternary Arithmetic, Geometric-Algebra State Spaces, and Adaptive-Depth Attractor Dynamics

Ed Epiphanes

Independent Research · Theoretical Proposal and Verification Dossier · Revised Draft v3.1 · November 13, 2026

Document status. This is a theoretical proposal. It reports no training runs and no hardware measurements. Every efficiency figure is an analytic projection derived from stated assumptions, and every central claim is classified in the Verification Register (Appendix A) so that each can be checked, repaired, or discarded in later work.

Implementation Brief for the Coding Agent (Read This First)

You are an AI coding agent asked to update the THME repository and add the code its engine needs. You cannot see the conversation in which this paper was written, so everything you need is here. Read this brief fully, then the appendices it points to. Where this brief and the repository's own guide (AGENTS.md, docs/WORKFLOW.md) disagree on process, the repository's guide wins. Where you are unsure what the owner wants, ask the owner; do not guess.

B.1 The Owner's Vision

THME is an open-source platform released under AGPL-3.0-only. It does two things at once. It bridges the many existing techniques for building, converting, compressing, and running language models, because most of those techniques already exist and the project's value is in connecting them under one measured, comparable framework. And it adds new tools and a new mathematical engine for capabilities that do not exist or are not available to the public. The owner wants these capabilities:

1. **A math engine** of validated numerical primitives (allocation, transport, differential-equation steppers, bases and rotations, semiring layers, projections, linear and integer programs, geometric algebra) and **typed conversions** between number formats, file and tensor formats, mathematical representations, and domains (Appendices M, G, F).

2. **A translation layer for models**: load a model from any source, infer an undocumented architecture, rearrange weights and layer order, convert between formats, architectures, and runtimes (Appendix O). Reverse-engineering features work only on models the user owns, has permission to analyze, or may use under an open licence, and educational use is supported under the same rule (Appendix P.1).
3. **Tunable gates, routers, and injected tiny decision models** (called needles) that route computation (Appendix P).
4. **Teacher-led distillation and autonomous training**: the owner has a large model of their own and wants a smaller model taught to act like it, trained on further data, or converted to another architecture. The large model proposes the training recipe and runs the trial-and-error loop under a frozen evaluator, with the bring-your-own-algorithm (BYOA) mechanism, looping, and self-tuning (Appendices K, Q, E, F).
5. **Benchmarks that run inside the software and are graphed**: every type of benchmark, with uncertainty and evidence status shown (Appendix R).
6. **A visual engine**: solver traces, layer and tensor maps, geometry and manifold views, dataflow graphs (Appendix M.6, R.7). Networks are viewed as manifolds in dimensions from 4 up to 11 or more by projecting to two or three dimensions and always reporting how much structure the projection kept.

The focus is the tools and the engine, not further testing of particular language models. Measurements exist only to validate the tools. Foundations come first and everything is open source now. A commercial tier with proprietary formulations is a later possibility, and nothing proprietary exists yet. The project's name, brand, and visual design are on hold and will be revised, so do not restyle, rename, or add new brand assets.

B.2 Rules You Must Follow

- Follow AGENTS.md: formatting and lint with zero warnings, tests on three operating systems, Rust 2021 at minimum version 1.89 on the pinned toolchain, no panics or unwraps on external input, pure-Rust permissive dependencies, binary-size limits (CLI 10 MB, Studio 15 MB), no model weights or binaries committed, no developer paths, SPDX headers saying AGPL-3.0-only, branch and commit conventions, and work from a ticket in BACKLOG.md.
- Follow the scientific-coding rules: explicit parameters with no hidden defaults for scientific quantities, explicit seeds, no silent fallbacks, errors that stay visible, raw data immutable, and validation against analytic solutions, conservation laws, limiting cases, and convergence. Do not refactor validated code unless asked.
- Label every figure with an evidence level: Measured, Tested, Estimate, or Simulated. Never present a simulated or toy result as a measurement.

- Share code between conditions that must stay identical (for example, use one Hadamard transform, not two).
- Prefer wrapping or bridging an existing tool, or a small native implementation with a differential test against the reference and credit to the source (Appendix N). Do not copy code from an incompatible licence.
- Do not implement any reverse-engineering or closed-format analysis feature without the permission and ownership record of WP19.
- Do not build model extraction through a provider interface. It is out of scope.
- Every autonomous or training feature needs hard budget caps and human approval above a threshold (WP29).

B.3 How to Treat the Claims in This Paper

The paper is a design document with partial evidence, not a specification you can trust line by line. Each appendix has a register of claims with a status letter: M for measured by a repository report, T for tested by a toy or unit test, S for supported by derivation or by something I observed in the repository, U for unverified, C for contradicted or corrected. Treat T, S, and U results as things to re-derive and re-test in the repository, not as facts. Method names and citations written from memory are unverified and must be checked against the original sources before they enter code comments or documentation. Everything in the paper about speed, energy, quality of a trained model, or quality of a distilled student is a projection. The only measured results are the perplexity measurements for two small models reported by the repository (Appendix J).

B.4 Reading Order

1. Appendix J: the measured results, the corrected additivity analysis, and the corrections to earlier claims.
2. Appendix M: the engine work packages (WP1 to WP13), conversions, and visual engine.
3. Appendix L and N: open-core design, the extension interface, and the bridging positioning.
4. Appendices O, P, Q, R: the translation layer, gates and needles, distillation and autonomous training, benchmarking.
5. Appendices E to I and K: the underlying mathematics and study design.

A supplement may accompany this paper: a standard-library-only prototype crate (error handling, number-format conversions, and an exact allocator, with 8 passing tests) and the short scripts that produced the numerical checks in Appendices K, O, Q, and R. Treat them as a head start and as oracle fixtures. They were run on rustc 1.97 and Python only; re-verify on the repository's pinned toolchain and minimum version.

B.5 Discrepancies to Resolve First

Before adding features, resolve or ticket these, because they affect trust in everything else.

- **Gate tolerance.** The tolerances file and the unit tests use a perplexity increase of 0.25; the measured reports use 0.45. Find which one the command line uses, state one rule, and record the gate version in every report (Appendix P.3, WP20).
- **Allocator.** The documentation describes a dynamic-programming knapsack allocator. The part of the allocator I read selects formats by sensitivity thresholds. Verify, then either correct the documentation or add and wire the exact solver (WP2).
- **Outdated Appendix J row.** Table J7 says the per-tensor export is not yet written; the roadmap says it is done.
- **Licence statements.** The Engineering Playbook states Apache-2.0 for the earlier project name; the repository now states AGPL-3.0-only (releases up to commit 56ae520 remain Apache-2.0). Update any document that still says Apache-2.0.
- **Nominal bits against stored bytes.** The dossier's nominal-bits column (13.06 for Q8_0) does not match a stored size of about 8.5 bits per weight. Define it or remove it (Appendix J.4c).
- **Size baseline.** The 513 MiB figure is FP32; the reduction against FP16 is about 47%, not 73%.
- **Register.** The Appendix A register stops at V-32; later claim identifiers live in the appendix where they were written. Consolidate them into one register file under docs/research/ with unique identifiers.
- **Accuracy gates with no measurements.** The tolerances file lists accuracy gates (MMLU, GSM8K); the repository has measured none. Mark them as not yet measured.

B.6 Prerequisites Checklist

Propose each item to the owner as tickets, then do them in this order.

1. Put this paper and the consolidated register in docs/research/ with the evidence notice the other documents use.
2. Create tickets in BACKLOG.md for each work package in B.7, using the existing ticket fields (milestone, roadmap item, priority, area, type, status, problem, done-means). Propose where they sit in ROADMAP.md and ask the owner to confirm; do not renumber existing milestones.
3. Add src/engine/ with the error module only, wired into lib.rs, with no behavior change. CI must stay green.

4. Add versioned schemas under docs/schemas/: benchmark task file, benchmark result record, quality-gate version, plugin protocol (draft), and conversion registry entry. Every schema carries a version field.
5. Add oracle fixtures: small analytic problems, brute-force references, and the mock-model generator that already exists. Prefer the standard library for random numbers in tests (an explicit-seed generator); propose any dev-dependency before adding it.
6. Re-check binary size against the limits after each new module.
7. Propose an addition to AGENTS.md that records the scientific-coding rules, for the owner's approval.
8. Update docs/subsystems.md only with statements the evidence labels support.

B.7 Work Package Index

Packages are defined in the appendix named in the last column. Do them in tier order; within a tier, packages are independent unless a dependency is listed. Keep each pull request small.

Table B1

Work Package Tiers

Tier	Packages	Depends on	Defined in
0. Reconcile and prepare	B.5 items, B.6 items	None	This brief
1. Engine foundations	WP1 error, WP2 allocation, WP3 number formats, WP4 transport, WP5 bases and rotations, WP6 ODE steppers, WP9 SVG visual engine	WP1 first	Appendix M
1b. Gates and permission	WP19 permission record, WP20 versioned quality gate	None	Appendix P
2. Benchmarks	WP31 registry, WP32 runner, WP33 metrics, WP34 statistics, WP35 three-state gate, WP36 contamination check, WP37 charts, WP38 system benchmarks	WP9, WP20	Appendix R
3. Translation layer	WP14 graph, WP15 symmetries, WP16 layout, WP17 architecture inference, WP18 compatibility report	WP19 for WP17; WP5 for WP15	Appendix O

Tier	Packages	Depends on	Defined in
4. Routing and needles	WP24 calibration, WP21 routers, WP25 hierarchical scoring, WP22 needle training, WP23 routing charts	WP14, WP9	Appendix P
5. Distillation and autonomy	WP26 teacher cache, WP27 losses, WP28 fidelity report, WP30 student planner, WP29 autonomy controller; external-trainer bridge	WP34, WP20, WP19	Appendix Q
6. Advanced mathematics and integration	WP7 semirings, WP8 projections, WP10 conversion registry, WP11 Clifford algebra, WP12 linear and integer programs, WP13 integration, WP39 engine benchmarks	Tier 1	Appendix M

The benchmark packages come early because every later feature needs them to report results honestly.

B.8 Definition of Done for Every Package

- The package's oracle tests pass, and each test checks against something outside the code under test (an analytic answer, a brute-force answer, a published value, or a conservation law).
- Error paths are tested: bad input returns an error, never a panic or a silent default.
- Formatting, lint, and the full test suite pass on the pinned toolchain; the minimum version is checked.
- Output carries an evidence label, a version, and the inputs needed to reproduce it.
- BACKLOG.md and ROADMAP.md are updated, and the pull request lists what was not done.
- No committed number changes without a new report under docs/benchmarks/.

B.9 Do Not

- Do not add proprietary code, formulas, or assets, and do not change the name, brand, or visual design.
- Do not describe a technique as new without a prior-art check (Appendix N.5).
- Do not run training or large evaluations without the budget caps of WP29.

- Do not add heavy dependencies; propose first.
- Do not copy another product's design, text, or numbers.
- Do not treat a toy result in this paper as evidence about a trained model.

B.10 Questions to Put to the Owner Before Starting

These are open decisions collected from the appendices. Ask them; do not decide them.

- Where the new milestones sit in the roadmap, and whether the engine is a library, a command-line subcommand, or both (Appendix M.9).
- Which dimension range and embedding the geometry view targets, and which Clifford signature to implement (M.9).
- Which architectures the graph supports first, and whether to load PyTorch pickles and ONNX (Appendix O.9).
- Which decision points get needles first, and how the permission statement is collected (Appendix P.9).
- The teacher's tokenizer, size, and serving; the student's size and architecture; the training compute; and the autonomy budget caps (Appendix Q.10).
- Which benchmark families come first, which evaluation tool to bridge, dataset licences, and whether a composite score is shown (Appendix R.12).
- The open-core line and the legal review of the plugin interface, layer-operation linking, and reverse-engineering terms (Appendices L and P.1). I am not a lawyer; a lawyer should confirm each.

Abstract

Contemporary Transformer language models combine quadratic sequence-level attention compute and a linearly growing key–value (KV) cache with dense floating-point arithmetic whose energy cost, at low batch size, is dominated by weight movement. This paper specifies the Topological Heterogeneous Manifold Engine (THME), a proposed architecture that integrates six design hypotheses: (a) ternary-weight (1.58-bit) arithmetic in shallow layers; (b) a Clifford-algebra multivector state space with a causal-cone constraint; (c) a stabilizer-inspired projection to suppress quantization drift; (d) a depth-dependent allocation of binary, ternary, and phase-quantized channels; (e) per-token adaptive-depth routing; and (f) a norm-bounded continuous attractor block for high-surprise tokens. We give formal definitions, correct several inconsistencies in earlier drafts (the signature of the scale coordinate, the sign of the stabilizer correction, the attractor energy function, the provenance of the arithmetic-energy table, and the KV-cache arithmetic), derive resource budgets from explicit assumptions, and supply a verification register that labels each central claim as a standard result, a modified claim,

an unverified claim, or a contradicted claim. The architectural proposal itself reports no empirical performance result; the only measurements quoted are the repository's own perplexity results, reconciled in Appendix J.

Keywords: ternary neural networks, Clifford algebra, adaptive computation, attractor networks, predictive coding, low-power inference, model distillation

1. Introduction

1.1 Motivation

Three bottlenecks shape the cost of deploying large language models. First, self-attention requires $O(N^2)$ compute over a sequence of N tokens and a KV cache that grows with context length. Second, dense FP16/BF16 arithmetic is expensive in silicon area and, more importantly, in the energy needed to move weights from memory. Third, end-to-end backpropagation forces a global backward pass, stores activations, and requires symmetric forward and backward weights. THME is a set of coupled design hypotheses addressing all three. Each component has independent precedent (Section 3); the claim of this paper is limited to the proposed combination and the depth-dependent allocation rule, not to the novelty of any single ingredient.

1.2 Contributions

1. A consolidated formal specification of THME (Section 4).
2. Corrections to mathematical and numerical inconsistencies in earlier drafts, listed in Appendix B.
3. A resource model derived from explicit assumptions, replacing the earlier hypothetical benchmark tables (Section 5).
4. A staged validation protocol with falsification criteria (Section 7).
5. A verification register classifying each central claim (Appendix A).

1.3 Scope, Terminology, and Epistemic Status

The words "quantum," "anyon," "holographic," "Calabi–Yau," and "topological" are used here as mathematical analogies. THME is a classical algorithm. No quantum hardware, quantum computation, or quantum speedup is involved or claimed. The third channel family, previously called "quantum," is renamed the **phase subspace**.

Claims carry one of four status labels (Table 1).

Table 1

Verification Status Labels

Label	Meaning
S	Standard: established in the literature or by elementary derivation
M	Modified: appeared in earlier drafts and has been corrected here
U	Unverified: plausible design hypothesis without supporting evidence
C	Contradicted: cannot hold as originally stated

2. Limitations of Current Architectures

2.1 Attention Cost and the KV Cache

Scaled dot-product attention (Vaswani et al., 2017) is

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})\, V \quad (1)$$

Computing QK^T for an N-token sequence costs $O(N^2 \cdot d_k)$. In autoregressive decoding with a KV cache, each new token costs $O(N \cdot d_k)$ per head, and the cache grows **linearly**, $O(N)$, in memory. Earlier drafts stated that memory grows as $O(N^2)$; that is incorrect for the cache, and fused kernels such as FlashAttention (Dao et al., 2022) avoid materializing the N×N score matrix altogether. The cache size is

$$M_{KV} = 2 \cdot L \cdot H_{kv} \cdot d_{head} \cdot N \cdot b \cdot B \quad (2)$$

where L is the number of layers, H_{kv} the number of key–value heads (grouped-query attention, Ainslie et al., 2023, makes H_{kv} smaller than the number of query heads), d_{head} the head dimension, N the context length, b the bytes per element, and B the batch size. Table 2 evaluates Equation 2 for a configuration matching published Llama 3 70B hyperparameters (80 layers, 8 KV heads, $d_{head} = 128$, FP16, $B = 1$).

Table 2

KV Cache Size, 70B-Class Model, FP16, Batch 1

Configuration	Bytes per token	N = 32,768	N = 131,072
Grouped-query (8 KV heads)	327,680	10.7 GB	42.9 GB
Full multi-head (64 KV heads)	2,621,440	85.9 GB	343.6 GB

The earlier draft's figure of 32 GB at 32k tokens matches neither row. At 32k tokens and batch 1 the grouped-query cache is about 8% of the 140 GB FP16 weight footprint, so the KV cache is a dominant cost only for long contexts or large batches (V-01, V-02).

2.2 Floating-Point Arithmetic and the Energy Budget

Floating-point addition is not associative. In FP16, which has a 10-bit stored significand, the spacing between representable numbers at 2048 is 2, so (2048 + 1) + 1 evaluates to 2048 while 2048 + (1 + 1) evaluates to 2050. When reductions are reordered across devices, results can differ between runs (V-03).

Earlier drafts asserted that multipliers consume 70–80% of an AI processor's energy. That figure is not supported for low-batch inference, where fetching weights from off-chip memory costs orders of magnitude more energy than the arithmetic itself (Horowitz, 2014; quantified in Section 5.3). Replacing multipliers with adders is a real but secondary lever (V-04).

2.3 Backward Locking and Weight Transport

For pre-activation $z_l = W_l a_{l-1} + b_l$ and activation $a_l = \sigma(z_l)$, backpropagation computes

$$\delta_l = (W_{l+1}^{\top} \delta_{l+1}) \odot \sigma'(z_l) \quad (3)$$

$$\partial L/\partial W_l = \delta_l a_{l-1}^{\top} \quad (4)$$

Three consequences follow: a layer cannot update until the entire forward pass and the backward recursion complete (backward locking); intermediate activations must be stored; and Equation 3 requires the transpose $W_{l+1}^{\top}$, which is the weight-transport problem named by Grossberg (1987). The last point is a plausibility problem for biological and some analog substrates; it is not an accuracy problem on GPUs. Alternatives such as feedback alignment (Lillicrap et al., 2016) and predictive coding (Whittington & Bogacz, 2017) exist, but to the author's knowledge none has been shown to match backpropagation at LLM scale (V-05).

2.4 Summary

Table 3

Established Limitations Versus Earlier Overstatements

Bottleneck	Established	Overstated in earlier drafts
Attention	$O(N^2)$ compute per sequence; $O(N)$ KV cache	"Memory grows as $O(N^2)$"; 32 GB cache at 32k

Bottleneck	Established	Overstated in earlier drafts
Arithmetic	FP non-associativity; multipliers cost area and energy	“Multipliers use 70–80% of energy”
Credit assignment	Backward locking; weight transport	“Obsolete”; implied unusable on GPUs

The appropriate framing is that current architectures have identifiable inefficiencies in specific regimes, not that they are obsolete. Transformers remain the empirical state of the art, and any alternative must be judged against them on matched compute and quality.

3. Related Work

Low-bit networks. BitNet (Wang et al., 2023) and BitNet b1.58 (Ma et al., 2024) train models with ternary weights {−1, 0, +1} and 8-bit activations, using a straight-through estimator (Bengio et al., 2013) to pass gradients through rounding. A 2-billion-parameter model trained at scale was reported by Ma et al. (2025). CPU inference kernels were reported by Wang et al. (2024).

Constant-state sequence models. Structured state-space models (Gu et al., 2022) and selective state-space models such as Mamba (Gu & Dao, 2023) replace attention with a linear recurrence, giving O(1) memory and O(1) compute per decoded token (V-31).

Adaptive depth. Depth-adaptive Transformers (Elbayad et al., 2020), confident adaptive language modeling (Schuster et al., 2022), and Mixture-of-Depths (Raposo et al., 2024) route tokens through different numbers of layers. THME’s reflex/attractor split is an instance of this family.

Attractor and energy-based dynamics. Hopfield networks (Hopfield, 1982, 1984) and their modern continuous form (Ramsauer et al., 2021) define associative memories by energy minimization. Neural ODEs (Chen et al., 2018) treat depth as continuous time. Unitary recurrent networks (Arjovsky et al., 2016) and deep complex networks (Trabelsi et al., 2018) motivate norm-preserving and phase-based representations.

Geometric algebra in learning. Clifford neural layers (Brandstetter et al., 2023), geometric Clifford algebra networks (Ruhe et al., 2023), and the Geometric Algebra Transformer (Brehmer et al., 2023) use multivector features for data with geometric structure, principally physical and 3D data.

Local learning. Predictive coding (Rao & Ballard, 1999; Friston, 2010) minimizes a local prediction-error energy; Whittington and Bogacz (2017) showed it approximates backpropagation with local Hebbian updates under specific conditions.

Physics sources of the analogies. Eleven-dimensional supergravity (Cremmer et al., 1978), the spin bound in supersymmetric theories (Nahm, 1978), Calabi–Yau compactification (Candelas et al., 1985), AdS/CFT (Maldacena, 1998), and anyonic stabilizer codes (Kitaev, 2003) supply the vocabulary used in Sections 4.2–4.5. None of these results is a theorem about neural networks.

4. The THME Architecture

4.1 Overview and Design Principles

THME organizes a network of total depth L into three zones along the fractional depth $\lambda = l / (L - 1) \in [0, 1]$ (Table 4). Every token is processed by Zone 1. A learned gate estimates whether the token needs further computation; tokens that pass the gate continue through Zones 2 and 3, and the remainder exit after Zone 1.

Table 4

THME Depth Zones

Zone	Depth λ	Channel mix (binary / ternary / phase)	Operators	Intended role
1 Surface	0.0–0.3	≈ 80 / 15 / 5 %	Ternary and binary integer linear maps; exit gate	Local syntax; exit decision
2 Midwater	0.3–0.7	≈ 20–25 / 50–60 / 20–25 %	Ternary routing; linear recurrence	State tracking
3 Abyss	0.7–1.0	≈ 5 / 15 / 80 %	Phase-quantized unitary maps; attractor block	Resolution of high-surprise tokens

Two design requirements were left unspecified in earlier drafts and are recorded as open issues: an exit at $\lambda = 0.3$ requires a language-model head attached at that depth (shared or separate), and the routing decision must be computed from information available **before** the token is emitted (Section 4.6).

4.2 Geometric State Space

The feature state of a channel is a multivector Ψ in a real Clifford algebra $Cl(p, q)$ with $n = p + q$ generators satisfying

$$e_\mu e_\nu + e_\nu e_\mu = 2 \eta_{\mu\nu}, \eta = \mathrm{diag}(+1, \ldots, +1, -1, \ldots, -1) \quad (5)$$

The algebra has dimension, and grade-k subspace dimension, given by

$\dim Cl(p, q) = \Sigma_k C(n, k) = 2^n$, $\dim(\text{grade } k) = C(n, k)$ (6)

For two vectors, the geometric product decomposes into a symmetric inner part and an antisymmetric outer part:

$ab = a \cdot b + a \wedge b$ (7)

(For general multivectors the product has additional grade components.) A rotor R with $R \tilde{R} = 1$, where $\tilde{R}$ denotes reversion, acts on a multivector by

$\Psi' = R \Psi \tilde{R}$ (8)

Table 5

Clifford Algebra Size as a Function of Generator Count n

n	Total dim 2^n	Grade dimensions (0…n)	Even subalgebra	Grades 0–2 only	Naive product cost 4^n
5	32	1, 5, 10, 10, 5, 1	16	16	1,024
11	2,048	1, 11, 55, 165, 330, 462, 462, 330, 165, 55, 11, 1	1,024	67	4,194,304

Four consequences matter for the architecture.

1. **State expansion.** If each of D channels carries a full multivector, the state width is $D \cdot 2^n$. For n = 11 this is a 2,048-fold expansion, which directly opposes the memory-reduction goal. Earlier memory tables omitted it (V-32). A workable variant restricts features to grades 0–2 (67 components at n = 11) or to the even subalgebra.
2. **Dimension is a hyperparameter.** No machine-learning argument selects n = 11. The bound of eleven dimensions in supergravity (Nahm, 1978; Cremmer et al., 1978) constrains supersymmetric field theories, not the representation space of a neural network, so the choice among n = 5, 7, 11 must be settled by ablation (V-15).
3. **Norm preservation is conditional.** The sandwich product (Equation 8) preserves the quadratic form of signature (p, q). A bivector $e_i e_j$ squares to $-\eta_{ii} \eta_{jj}$: it generates a compact rotation when both generators have the same sign and a non-compact boost, with cosh/sinh growth, when the signs differ. “Prevents numerical explosions” therefore holds only if R is restricted to same-sign bivectors (V-07).
4. **Justification.** Clifford layers have demonstrated value on data with geometric symmetry. For language tokens there is no evidence yet. This is stated as hypothesis H-G and tested by ablation in Section 7.

Earlier drafts also claimed that rotors “rotate binary signs, ternary masks, and complex phases concurrently without type casting.” The mapping from those three channel families to multivector grades is never defined, so the claim is classed as unverified (V-06, V-07).

4.3 Causal Structure and the Spacetime–Scale Metric

Earlier drafts embedded tokens in a five-dimensional space with metric $ds^2 = (L^2/V^2)(-c^2dW^2 + dX^2 + dY^2 + dZ^2 - dV^2)$, giving signature (3, 2): **two** timelike directions. A metric with more than one time direction is ultrahyperbolic; the set $\Delta s^2 \leqslant 0$ is then not a cone separating past from future, closed timelike curves exist, and no consistent causal order results (V-08). In the Poincaré form of anti-de Sitter space the radial coordinate is spacelike. The corrected metric is

$$ds^2 = (L^2 / V^2)\,(-c^2\, dW^2 + \|dX\|^2 + dV^2),\ X \in \mathbb{R}^m \quad (9)$$

which has Lorentzian signature $(m + 1, 1)$. Because null cones are invariant under the conformal factor L^2/V^2, event i may influence event j if and only if

$$c^2\,(W_j - W_i)^2 \geqslant \|X_j - X_i\|^2 + (V_j - V_i)^2,\ W_j \geqslant W_i \quad (10)$$

Three remarks qualify the earlier claim that causality is enforced “by geometry rather than masking.”

- Consistency with autoregressive order requires W_i to equal the token’s position index. The cone then acts as a learned, additional restriction on top of an ordinary causal mask, not as a replacement for it (V-09).
- Equation 10 constrains pairs of **stored** events. An architecture that retains no past events (a recurrent state) is causal by construction, so the cone is vacuous there. The “topological light-cone invariance” row in earlier comparison tables therefore represents no additional guarantee.
- Earlier drafts used the same labels X, Y, Z both for algebra generators (binary polarity, ternary deadband, phase) and for spatial coordinates. These are distinct objects. Generators define the algebra in which features live; coordinates (W, X, V) are event positions used only by Equation 10.

4.4 Stabilizer-Inspired Projection

Let $S_1, \ldots, S_m$ be symmetric linear involutions on $\mathbb{R}^D$ ($S_k^2 = I$) that pairwise commute. The code space and its orthogonal projector are

$$H = \{\, \Psi : S_k \Psi = \Psi \text{ for all } k \,\},\ \Pi = \Pi_k \tfrac{1}{2} (I + S_k) \quad (11)$$

The syndrome of a state is its component in the -1 eigenspace of each stabilizer:

$$\sigma_k = \tfrac{1}{2} (I - S_k)\, \Psi \quad (12)$$

and a soft correction with step size κ is

$$\Psi \leftarrow \Psi - \kappa \Sigma_k \sigma_k \quad (13)$$

With κ = 1, Equation 13 equals ΠΨ when the −1 eigenspaces are mutually orthogonal; otherwise the product form in Equation 11 must be used. The earlier draft wrote the correction as Ψ − Σ σ_k S_k Ψ. Since S_k σ_k = −σ_k, that expression equals Ψ + Σ σ_k and moves the state **away** from the code space (V-10).

The operation is a fixed linear projection. It suppresses quantization noise only to the extent that the signal lies inside H and the noise has components outside H. Rounding noise is approximately isotropic in $\mathbb{R}^D$, so a projection onto a subspace of dimension D_H removes a fraction 1 − D_H/D of noise energy and, unless the signal is confined to H, the same fraction of signal energy. The topological protection of Kitaev's toric code (Kitaev, 2003) comes from non-local logical operators and a local error model on qubits; no analogue has been established for activation rounding noise (V-11). The reference prototypes also used LayerNorm in the attractor, contradicting the claim of operating "without lossy normalization." The testable hypothesis is H-S: at equal bit-width, adding the projection reduces the error ‖Ψ_quantized − Ψ_FP32‖ relative to RMSNorm alone.

4.5 Phase Compaction and the Kaluza–Klein Analogy

Phase quantization (operational). A complex weight z = ρe^{iθ} is stored with magnitude ρ and a B-bit phase index k, so that

$$\theta_k = 2\pi k / 2^B, \; k \in \{0, 1, \ldots, 2^B - 1\} \quad (14)$$

For B = 4 there are 16 phases. The product of two such numbers multiplies magnitudes and adds phase indices modulo 2^B:

$$k_{result} = (k_1 + k_2) \bmod 2^B \quad (15)$$

This is a standard and inexpensive operation. Two limits must be stated. First, a matrix–vector product requires **sums** of complex products, and complex addition cannot be performed on phase indices; the products must be converted to Cartesian form (a 16-entry cosine/sine lookup table) and accumulated with fixed-point adders. The phase representation removes multipliers from the product step, not accumulation or conversion (V-12). Second, nearest-phase rounding at B = 4 leaves a worst-case phase error of π/16 ≈ 0.196 rad, a worst-case relative error of 2 sin(π/32) ≈ 0.196 on each product and an RMS phase error of about 0.113 rad, so whether four bits suffice is an empirical question.

Kaluza–Klein analogy. Compactifying a dimension on a circle of radius R yields discrete momentum modes n/R, and at low energy only the zero mode is excited. Discretizing a phase circle to the cyclic group Z_(2^B) is the faithful computational counterpart; it is a quantization scheme and nothing more.

Calabi–Yau claims. In M-theory, eleven dimensions can factor as five non-compact dimensions times a compact Calabi–Yau threefold (a Ricci-flat Kähler six-manifold with SU(3) holonomy; Candelas et al., 1985). With the corrected metric of Equation 9, the non-compact factor has signature (4, 1) and the internal six dimensions are spacelike, giving total signature (10, 1) and removing the inconsistency present in the earlier five-dimensional draft. Beyond this bookkeeping, three earlier statements cannot stand. The mass formula $m_{n^2} \propto n(n + d)/R^2$ is the shape of a sphere's Laplacian spectrum; the spectrum of a Calabi–Yau manifold has no closed form (V-14). Topological invariants such as Betti numbers are integers describing the manifold, not per-token storage, so the claim that information in internal cycles "does not occupy memory" is incorrect (V-13). And no mapping from grammatical or relational invariants to harmonic forms is defined.

An operational surrogate is proposed as hypothesis H-K: replace the six internal coordinates with a discrete torus $(Z_{(2^B)})^6$, a six-circle register of 6B bits (24 bits at B = 4) per channel that carries discrete relational state. The flat six-torus is a degenerate Ricci-flat Kähler manifold, so the surrogate is at least geometrically consistent with the analogy.

4.6 Depth-Dependent Subspace Schedule and Adaptive Routing

The channel width is divided among binary (α), ternary (β), and phase (γ) subspaces by

$$\alpha(\lambda) = \cos^2(\pi\lambda / 2) \quad (16)$$

$$\beta(\lambda) = \sin^2(\pi\lambda) \quad (17)$$

$$\gamma(\lambda) = \sin^2(\pi\lambda / 2) \quad (18)$$

The identity $\alpha + \gamma = 1$ holds exactly, but β is additive on top of it, so shares must be normalized:

$$(\tilde{\alpha}, \tilde{\beta}, \tilde{\gamma}) = (\alpha, \beta, \gamma) / (\alpha + \beta + \gamma) \quad (19)$$

Table 6 evaluates Equation 19. The 80/15/5, 20–25/50–60/20–25, and 5/15/80 mixes quoted for the zones are reproduced only approximately and only near the zone midpoints $\lambda \approx 0.15$, 0.5, and 0.85. At the zone boundary $\lambda = 0.3$ the mix is already 48/40/12 (V-16).

Table 6

Normalized Channel Shares from Equation 19

λ	Binary α̃	Ternary β̃	Phase γ̃
0.00	100%	0%	0%
0.15	78%	17%	5%

λ	Binary α̃	Ternary β̃	Phase γ̃
0.30	48%	40%	12%
0.50	25%	50%	25%
0.70	12%	40%	48%
0.85	5%	17%	78%
1.00	0%	0%	100%

Routing signal. Earlier drafts defined "Shannon surprise" as S_t = −Σ P(v) log P(v). That expression is the predictive **entropy** H_t of the next-token distribution. Surprisal of the realized token, −log P(x_t), is unavailable before emission and must not be used for routing, since doing so would leak the answer. The gate is therefore defined on intermediate features:

g_t = σ(w · h_t^(1) + b); route token t to Zones 2–3 if g_t > τ (20)

where h_t^(1) is the Zone 1 output and w, b are learned, optionally supervised by the entropy of an exit head. The reference prototypes instead evaluated mean_surprise > 0.5 over the whole batch, a single Python branch that sends **all** tokens to the attractor or none, is non-differentiable, and forces a device-to-host synchronization. Per-token routing requires gather/scatter with a fixed capacity, as in Mixture-of-Depths (Raposo et al., 2024).

Training uses a budgeted objective in which λ_c trades quality against depth:

L = L_task + λ_c · E_t[f_t] (21)

where f_t is the fraction of depth executed by token t. Without auxiliary balancing terms, routers tend to collapse to one extreme (all tokens exit, or none).

The 80/20 split is a design target, not a result. If a fraction p_exit of tokens exits at depth fraction λ_exit and the rest run to full depth, the expected depth fraction is

f̄ = p_exit · λ_exit + (1 − p_exit) · 1 (22)

For p_exit = 0.8 and λ_exit = 0.3, f̄ = 0.44. Computation falls by about 2.3×, not by 80%; "zero added FLOPs" on the fast path does not mean zero FLOPs (V-17).

4.7 The Attractor Block

The earlier specification took a Lyapunov energy E(Z) = −½ Z† W Z − Re⟨Ξ, Z⟩ + Φ(|Z|) with W skew-Hermitian, and Langevin dynamics dZ/dt = −∇E + √(2T) ξ. Four problems follow.

1. **The quadratic term vanishes.** For real skew-symmetric K, $z^{T}Kz = 0$ for every z; for skew-Hermitian K, $z^{\dagger}Kz$ is purely imaginary. The pairwise coupling therefore contributes nothing to a real energy, leaving a drive term and a confining quartic: a single-well potential with one minimum and no stored patterns.
2. **Associative memory needs symmetric coupling.** Hopfield networks (Hopfield, 1982) store patterns in a symmetric coupling $J = \Sigma_{\mu} \xi_{\mu} \xi_{\mu}^{T}$; the modern continuous form (Ramsauer et al., 2021) uses an exponential energy.
3. **A skew term is a rotation, not a descent.** Adding Ωz with Ω skew-symmetric gives $dE/dt = -\|\nabla E\|^{2} + \nabla E \cdot \Omega z$, which is not guaranteed negative. The Gibbs distribution $\exp(-E/T)$ remains stationary only if the extra drift $b = \Omega z$ satisfies $\nabla \cdot b = 0$ and $b \cdot \nabla E = 0$, for instance when E is invariant under the rotation (V-18).
4. **The prototype diverges from the specification.** The code used $A - A^{T}$ directly rather than $\exp(A - A^{T})$, applied tanh inside the rotational term (so the flow is not norm-preserving), added LayerNorm at the output, and injected noise only in training mode, with no annealing schedule.

A corrected formulation uses the modern Hopfield energy with learned memories $\Xi = [\xi_1, \ldots, \xi_M]$ and input-dependent drive b(x):

$$E(z) = -(1/\beta) \log \Sigma_{\mu} \exp(\beta\, \xi_{\mu}^{T} z) + \tfrac{1}{2} \|z\|^{2} - \langle b(x), z \rangle \quad (23)$$

with $\nabla E = z - \Xi\, \mathrm{softmax}(\beta\, \Xi^{T} z) - b$. For an L-smooth energy, gradient steps decrease E monotonically when

$$dt < 2 / L \quad (24)$$

and the discretized stochastic dynamics (Euler–Maruyama) are

$$z_{k+1} = z_k - dt \cdot \nabla E(z_k) + \sqrt{(2\, T_k\, dt)}\, \xi_k,\ \xi_k \sim N(0, I) \quad (25)$$

The noise scale $\sqrt{(2T\, dt)}$ is correct (V-19). The earlier claims of a “second-order phase transition” on cooling and of convergence to a unique reasoning solution are unsupported. Annealed Langevin dynamics is a form of simulated annealing, whose convergence guarantee requires logarithmic cooling (Geman & Geman, 1984), not the exponential schedule $T(t) = T_0 \exp(-t/\tau_{cool})$ proposed. At inference the noise is disabled and the block performs deterministic descent. Each of the K steps costs a product with the M×d memory matrix; unless that matrix is ternarized, this zone is not multiplier-free, and its cost, $K \cdot M \cdot d$ per token, must be added to Section 5.

4.8 Local Learning via Predictive Coding

In predictive coding, layer l holds a state Ψ_l, a top-down prediction $F_l(\Psi_{l+1}) = g(W_{l+1}\, \Psi_{l+1})$ produces the error ε_l, and the system minimizes

$$\varepsilon_l = \Psi_l - F_l(\Psi_{l+1}) \quad (26)$$

$$\mathcal{F} = \tfrac{1}{2} \Sigma_l \varepsilon_l^{\mathsf{T}} \Lambda_l \varepsilon_l \quad (27)$$

where Λ_l is the precision (inverse covariance). The weight update is local:

$$\Delta W_{l+1} = \eta \left(\Lambda_l \varepsilon_l \odot g'(W_{l+1} \Psi_{l+1}) \right) \Psi_{l+1}^{\mathsf{T}} \quad (28)$$

However, the state-relaxation dynamics are $d\Psi_l/dt = -\Lambda_l \varepsilon_l + W_l^{\mathsf{T}} (\Lambda_{l-1} \varepsilon_{l-1} \odot g')$, which contains the transpose W_l^{T}. Predictive coding makes the weight **update** local but still routes errors upward through reciprocal connections, so the earlier assertion that “no transposed downstream matrices are ever queried” is incorrect (V-20). Relaxation also requires several iterations per input. Finally, the reference prototypes were stated to operate “natively under standard PyTorch backpropagation,” so local learning was never implemented. This paper therefore treats local learning as optional future work and assumes backpropagation with a straight-through estimator for all near-term experiments.

5. Resource Analysis

The earlier draft’s “feasibility benchmarks” were hypothetical figures presented in a form that reads as measurement. This section replaces them with calculations from stated assumptions.

5.1 Assumptions

- A1. 70×10^9 parameters; autoregressive decoding at batch size 1.
- A2. Ternary weights packed at 2 bits; one integer add per non-zero weight.
- A3. $p_{exit} = 0.8$ at $\lambda_{exit} = 0.3$, giving $\bar{f} = 0.44$ (Equation 22). Weights of unexecuted layers are not fetched.
- A4. Embeddings, output head, normalization parameters, scale factors, attractor memories, and activations are ignored, which understates cost.
- A5. Energy constants are from the published 45 nm figures in Table 8; DRAM energy per bit is bracketed at 4–20 pJ.

5.2 Weight Storage

Table 7

Weight Memory for a 70B-Parameter Model by Storage Format

Format	Bits per weight	Size
FP16 / BF16	16	140 GB
INT8	8	70 GB

Format	Bits per weight	Size
INT4	4	35 GB
Ternary, 2-bit packed	2	17.5 GB
Ternary, ideal entropy coding ($\log_2 3 \approx 1.585$)	1.585	13.9 GB

The 17.5 GB figure covers the packed body only. For a 1.24B-parameter Llama 3.2 1B-class model, the 128,256 × 2,048 embedding table is about 0.53 GB in BF16 by itself, so a converted model occupies roughly 0.8 GB rather than the 250 MB quoted in earlier drafts (V-23, V-29).

5.3 Arithmetic Versus Memory Energy

Table 8

Energy per Operation (45 nm Process, Horowitz, 2014)

Operation	Energy (pJ)	Note on earlier draft
INT8 add	0.03	Used for the “ternary add” row
INT32 add	0.1	Baseline of the “multiplier vs. integer add” column
INT8 multiply	0.2	Labeled “MAC”
FP16 add	0.4	Omitted
FP16 multiply	1.1	Labeled “MAC”
FP32 add	0.9	Omitted
FP32 multiply	3.7	Labeled “MAC”

The earlier table attributed these values to “TSMC 7 nm/5 nm” and labeled multiply-only figures as multiply–accumulate. The values trace to a 45 nm survey, and a MAC includes an add. The silicon-area column was inconsistent with that source and has been removed (V-21). The ternary-versus-FP16 arithmetic ratio of 40–70× quoted earlier is plausible in order of magnitude; BitNet b1.58 reports a comparable arithmetic-energy reduction for matrix products (Ma et al., 2024).

The energy per decoded token is modeled as

$$E_{token} = f \cdot P \cdot (1 - s) \cdot e_{add} + 8 \cdot f \cdot P \cdot (b_w / 8) \cdot e_{mem} \quad (29)$$

with P parameters, zero fraction s, bits per weight b_w, and DRAM energy e_{mem} per bit. Substituting the assumptions of Section 5.1 gives Table 9.

Table 9

Per-Token Cost Estimate, 70B Model, Batch 1, Off-Chip DRAM

Quantity	Estimate
Expected depth fraction $\bar{f}$	0.44
Weight bytes read per token (THME)	7.7 GB
Integer adds per token (s = 0)	3.1×10^{10}
Arithmetic energy at 0.03–0.1 pJ per add	0.9–3.1 mJ
Memory energy at 4–20 pJ per bit	0.25–1.2 J
Weight bytes read per token (dense FP16)	140 GB
Memory energy, dense FP16, 4–20 pJ per bit	4.5–22 J
Decode ceiling at 136 GB/s: THME / FP16	≈ 17.7 / ≈ 1.0 tokens/s

Three conclusions follow. Arithmetic alone can plausibly reach the order of the earlier 0.42 mJ figure. At batch 1 on conventional DRAM, however, memory energy exceeds arithmetic energy by roughly 10^2–10^3, so the realistic per-token energy is hundreds of millijoules, and the earlier 0.42 mJ is 600–3,000 times below it. The earlier comparison set THME at 0.42 mJ against a transformer at 35 mJ without stating the batch regime; 35 mJ is only reachable for the transformer with heavy batching that amortizes weight reads. On a like-for-like memory-bound basis, the gain is the ratio of bytes read: 8× from 2-bit weights and about 18× once adaptive depth is included (140 / 7.7), not 80×. Reaching sub-millijoule energy would require weight-stationary or in-memory compute hardware, or large-batch amortization (V-22). Ratios of this kind should be re-derived for the specific memory part used.

5.4 Edge Runtime

The runtime of a battery of capacity E_b under a continuous load P is

t = E_b / P (30)

Table 10

Runtime of a 48 V, 15 Ah (720 Wh) Pack at Constant Compute Power

Compute power	Runtime
2,800 W	15.4 min

Compute power	Runtime
1,200 W	36 min
450 W	1.6 h
160 W	4.5 h
35 W	20.6 h
25 W	28.8 h

The arithmetic is correct, but earlier drafts quoted both “18+ hours” and “28.8 hours” for the same 25 W case. The figures count compute power only; motors, sensors, and conversion losses reduce the usable runtime. Whether a 25 W platform can host a 70B-class model at useful speed, and at what quality, is untested; the statement that it delivers “full 70B-grade reasoning” is unverified (V-24, V-25).

6. Conversion of a Pre-Trained Model

6.1 Procedure

The proposed pipeline initializes THME from an open pre-trained model (for example Llama 3.2 1B or SmolLM2 1.7B) by grafting the token embeddings and output head, initializing the block weights from the teacher, and distilling on a text corpus. Replacing softmax attention by a fixed-size recurrent state is lossy in general and is the highest-risk step of the pipeline (U).

6.2 Weight Initialization

The earlier instruction to map “high-energy singular vectors to the ternary path and rotational components to the skew generator” is undefined: a singular value decomposition $W = U\Sigma V^{T}$ has no rotational part to extract (V-28). Two well-defined alternatives are proposed. The first is the BitNet b1.58 absmean rule, $\gamma = \text{mean}|W|$ and $\tilde{W} = \text{clip}(\text{round}(W/\gamma), -1, 1)$. The second, for a square weight matrix, is the polar decomposition

$$W = Q\,P,\quad Q = \exp(K),\quad K = -K^{T},\quad P = P^{T} \succeq 0 \tag{31}$$

where the orthogonal factor Q (when $\det Q = +1$) supplies a skew generator K for rotation blocks and the symmetric positive semidefinite factor P is a natural initializer for a symmetric Hopfield-type coupling. Rectangular matrices require the generalized (partial-isometry) form. Whether either initialization preserves enough teacher function is hypothesis H-I.

6.3 Distillation Objective

The student is trained with

$$L = L_{KD}(p_{student}, p_{teacher}; T) + \mu \Sigma_l \|\Psi_l - h_l^{teacher}\|^2 + \lambda_c \cdot E_t[f_t] \quad (32)$$

where L_{KD} is the temperature-scaled KL divergence between output distributions (Hinton et al., 2015). Layer-wise matching presupposes a mapping between student and teacher depth that must be defined, and is meaningless for tokens that exit early.

6.4 Compute and Memory Reality Check

Training cost is approximately $6 \cdot N_p \cdot D$ FLOPs for N_p parameters and D tokens. For $N_p \approx 1.2 \times 10^9$ and $D = 10^{10}$ this is about 7×10^{19} FLOPs. At an assumed sustained 10 TFLOP/s on a single consumer GPU (an assumption to be measured), that is about 85 days of student compute, plus a teacher forward pass, which adds roughly one third. A pilot of 5×10^8 to 10^9 tokens takes about 4–9 days. The earlier claim of 5–15 billion tokens "in days" is therefore contradicted (V-26). On memory, training with BF16 weights, FP32 master copy, FP32 Adam moments, and gradients costs about 16 bytes per parameter, or roughly 20 GB for 1.24B parameters, which exceeds a 16 GB card before activations. The claim of under 9.5 GB is unsupported; it may be reachable with 8-bit optimizer states, frozen embeddings, and gradient checkpointing (V-27).

6.5 Packed Inference on CPUs

Four ternary weights are packed per byte (00 → 0, 01 → +1, 10 → −1). A popcount formulation, popcount(x & pos) − popcount(x & neg), is exact for 1-bit activations; for 8-bit activations it requires bit-plane decomposition or the lookup-table kernels used by bitnet.cpp. Reported CPU gains of roughly 1.4–6× in speed and 55–82% in energy are relative to llama.cpp CPU baselines, not to GPU inference (Wang et al., 2024; figures to be re-checked against the source), so the earlier "2× to 4× faster than GPU at one-tenth the power" is a misattribution (V-30).

7. Validation Plan

Validation proceeds in four tiers, each gated on the previous one. Components are added one at a time, because stacking six novel mechanisms in a single model makes any success or failure unattributable.

Tier 0: mathematical unit tests (no training).

1. Clifford algebra: associativity of the geometric product; grade dimensions of Table 5; norm preservation for same-sign bivector rotors and exponential growth for mixed-sign (boost) rotors (V-06, V-07).

2. Stabilizer projection: $\Pi^2 = \Pi$ and $\Pi = \Pi^T$; agreement of Equations 11 and 13 when the −1 eigenspaces are orthogonal; demonstration that the earlier sign gives $\Psi + \Sigma\sigma_k$ (V-10).
3. Attractor: monotone decrease of Equation 23 under Equation 25 with $T = 0$ and $dt < 2/L$; on a low-dimensional toy, a histogram of samples at $T > 0$ against $\exp(-E/T)$, with and without a skew term (V-18, V-19).
4. Phase quantization: Monte Carlo comparison of Equations 14–15 against FP32 complex matrix products, reporting signal-to-quantization-noise ratio against B (V-12).
5. Schedule and cone: reproduction of Table 6; confirmation that Equation 10 with W equal to position index never admits future-to-past interaction (V-09, V-16).

Tier 1: small-scale language modeling. Train models of roughly 20–125M parameters on a fixed corpus with matched parameter count and matched training FLOPs against three baselines: an FP16 Transformer, a BitNet-style ternary Transformer, and Mamba. Ablate in order: ternary only; plus linear-recurrent token mixing (absent from the prototypes, see Appendix C); plus exit gate; plus attractor; plus Clifford features (H-G); plus stabilizer (H-S); plus phase subspace; plus discrete-torus register (H-K). Run at least three seeds. Report validation perplexity, mean executed depth $\bar{f}$ against quality, the routing histogram, and gradient and loss stability.

Tier 2: conversion pilot. Convert one block of a 1B-class teacher, then the full model, using the initializations of Section 6.2 and 0.5–1 billion tokens. Report held-out perplexity relative to the teacher and the mean of a standard benchmark suite relative to the teacher.

Tier 3: systems measurement. Measure tokens per second and joules per token with an external power meter on CPU (packed kernels) and GPU (fused kernels), stating batch size and memory part, and compare against Equation 29.

Table 11

Hypotheses and Falsification Criteria (Proposed Thresholds)

Hypothesis	Falsified if
H-G: Clifford features help	No perplexity gain over equal-width linear layers at matched parameters and FLOPs, across 3 seeds
H-S: Stabilizer projection helps	Quantization error $\|\Psi_q - \Psi_{FP32}\|$ not lower than with RMSNorm alone at equal bit-width
H-K: Discrete torus register helps	No gain over the same number of extra plain channels

Hypothesis	Falsified if
H-I: Polar/absmean initialization preserves teacher function	Pilot student recovers less than 90% of teacher benchmark average
Routing works	p_exit leaves 0.5–0.95 after training, or quality at f̄ = 0.44 is more than 2% below the no-exit model
Resource model is valid	Measured batch-1 energy per token deviates from Equation 29 by more than a factor of 3

8. Limitations

- **No empirical evidence.** No THME architecture proposed in this paper has been trained or measured. All efficiency figures are projections. The only measurements quoted are the repository's own results (Appendix J), and the appendix simulations are small numerical checks, not trained-model results.
- **Confounded design.** Six novel mechanisms interact. The ternary arithmetic and adaptive-depth ideas have independent support; the Clifford state space, stabilizer projection, compactified registers, and attractor block do not, and may add cost without benefit.
- **Missing token mixing.** The specification names a linear recurrence in Zone 2 but gives no equations for it. A concrete selective state-space or equivalent operator must be specified before any language-modeling claim is possible.
- **Hardware dependence.** The largest projected energy gains require in-memory or weight-stationary hardware that does not exist as a product; on commodity DRAM-bound hardware the gain is bounded by weight-byte reduction (Section 5.3).
- **Analogy risk.** The physics vocabulary is a source of intuition, not of theorems. Where an analogy yields no testable operational definition (Calabi–Yau invariants storing grammar, topological protection of rounding noise), it should be dropped rather than defended.
- **Training stability.** Straight-through estimation, discrete routing, and iterative dynamics each have known instabilities; their combination is untested.
- **References.** The reference list was compiled without database access and must be verified (see the note after the list).

9. Conclusion

THME packages a plausible efficiency thesis, namely that low-bit arithmetic plus per-token adaptive depth reduces the bytes and operations needed per token, with several higher-risk geometric and dynamical hypotheses. This paper separated those layers, repaired the mathematics where repair was possible, replaced hypothetical benchmarks with assumption-driven estimates (an expected 8–18× reduction in weight traffic at batch 1, not 80× in energy), and defined falsifiable tests. The recommended next step is Tier 0 followed by a minimal Tier 1 comparison of a ternary baseline with and without the exit gate, since that is the part of the proposal with the strongest prior support and the clearest measurable payoff.

Legal Notice and Contact

Status. This document is a research proposal and engineering brief. It is provided as is, without warranty of any kind, and is not legal, financial, medical or professional advice. The software it describes is in development and is not yet released for commercial use. Nothing here is an offer, a commitment to deliver a product, or a grant of rights beyond those stated in the repository's own licence files.

Open clause. Any legal question or concern about this document or the project (licensing, authorship, third-party material, model-conversion or reverse-engineering use, trademarks, privacy or anything else) is left open on purpose. The author, Ed Epiphanes, will address it on request, and anyone who has such a concern is asked to contact the author before relying on, copying or redistributing the material. The author may correct, withdraw or restrict any part of this document.

Protection. All rights not expressly granted are reserved. The author's name, brand and designs are not licensed by this document. Third-party names, methods and results are cited for identification and scholarly discussion only and remain the property of their owners. Reverse-engineering and translation features may be used only on models the user owns, has permission to use, or may use under an open licence.

Commercial use. Commercial use of the project is not available at this time. Before it is, the author intends to review the licence, the open-core split and all third-party terms with a lawyer, and this notice will be updated afterwards. Until then, no one should assume any commercial right, warranty or support.

References

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., & Sanghai, S. (2023). GQA: Training generalized multi-query transformer models from multi-head checkpoints. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. https://arxiv.org/abs/2305.13245

Arjovsky, M., Shah, A., & Bengio, Y. (2016). Unitary evolution recurrent neural networks. *Proceedings of the 33rd International Conference on Machine Learning*. https://arxiv.org/abs/1511.06464

Bengio, Y., Léonard, N., & Courville, A. (2013). *Estimating or propagating gradients through stochastic neurons for conditional computation*. arXiv. https://arxiv.org/abs/1308.3432

Brandstetter, J., van den Berg, R., Welling, M., & Gupta, J. K. (2023). Clifford neural layers for PDE modeling. *International Conference on Learning Representations*. https://arxiv.org/abs/2209.04934

Brehmer, J., de Haan, P., Behrends, S., & Cohen, T. (2023). Geometric algebra transformer. *Advances in Neural Information Processing Systems, 36*. https://arxiv.org/abs/2305.18415

Candelas, P., Horowitz, G. T., Strominger, A., & Witten, E. (1985). Vacuum configurations for superstrings. *Nuclear Physics B, 258*, 46–74.

Chen, R. T. Q., Rubanova, Y., Bettencourt, J., & Duvenaud, D. (2018). Neural ordinary differential equations. *Advances in Neural Information Processing Systems, 31*.

Cremmer, E., Julia, B., & Scherk, J. (1978). Supergravity theory in 11 dimensions. *Physics Letters B, 76*(4), 409–412.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems, 35*.

Elbayad, M., Gu, J., Grave, E., & Auli, M. (2020). Depth-adaptive transformer. *International Conference on Learning Representations*.

Friston, K. (2010). The free-energy principle: A unified brain theory? *Nature Reviews Neuroscience, 11*(2), 127–138.

Geman, S., & Geman, D. (1984). Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 6*(6), 721–741.

Grossberg, S. (1987). Competitive learning: From interactive activation to adaptive resonance. *Cognitive Science, 11*(1), 23–63.

Gu, A., & Dao, T. (2023). *Mamba: Linear-time sequence modeling with selective state spaces*. arXiv. https://arxiv.org/abs/2312.00752

Gu, A., Goel, K., & Ré, C. (2022). Efficiently modeling long sequences with structured state spaces. *International Conference on Learning Representations*.

Hinton, G., Vinyals, O., & Dean, J. (2015). *Distilling the knowledge in a neural network*. arXiv. https://arxiv.org/abs/1503.02531

Hopfield, J. J. (1982). Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences, 79*(8), 2554–2558.

Hopfield, J. J. (1984). Neurons with graded response have collective computational properties like those of two-state neurons. *Proceedings of the National Academy of Sciences, 81*(10), 3088–3092.

Horowitz, M. (2014). Computing's energy problem (and what we can do about it). *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers*, 10–14.

Kitaev, A. Y. (2003). Fault-tolerant quantum computation by anyons. *Annals of Physics, 303*(1), 2–30.

Lillicrap, T. P., Cownden, D., Tweed, D. B., & Akerman, C. J. (2016). Random synaptic feedback weights support error backpropagation for deep learning. *Nature Communications, 7*, Article 13276.

Ma, S., Wang, H., Ma, L., Wang, L., Wang, W., Huang, S., Dong, L., Wang, R., Xue, J., & Wei, F. (2024). *The era of 1-bit LLMs: All large language models are in 1.58 bits*. arXiv. https://arxiv.org/abs/2402.17764

Ma, S., Wang, H., Huang, S., et al. (2025). *BitNet b1.58 2B4T technical report*. arXiv. https://arxiv.org/abs/2504.12285

Maldacena, J. (1998). The large N limit of superconformal field theories and supergravity. *Advances in Theoretical and Mathematical Physics, 2*(2), 231–252.

Nahm, W. (1978). Supersymmetries and their representations. *Nuclear Physics B, 135*(1), 149–166.

Ramsauer, H., Schäfl, B., Lehner, J., et al. (2021). Hopfield networks is all you need. *International Conference on Learning Representations*.

Rao, R. P. N., & Ballard, D. H. (1999). Predictive coding in the visual cortex: A functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience, 2*(1), 79–87.

Raposo, D., Ritter, S., Richards, B., et al. (2024). *Mixture-of-Depths: Dynamically allocating compute in transformer-based language models*. arXiv. https://arxiv.org/abs/2404.02258

Ruhe, D., Gupta, J. K., de Keninck, S., Welling, M., & Brandstetter, J. (2023). Geometric Clifford algebra networks. *Proceedings of the 40th International Conference on Machine Learning*. https://arxiv.org/abs/2302.06594

Schuster, T., Fisch, A., Gupta, J., et al. (2022). Confident adaptive language modeling. *Advances in Neural Information Processing Systems, 35*.

Trabelsi, C., Bilaniuk, O., Zhang, Y., et al. (2018). Deep complex networks. *International Conference on Learning Representations*.

Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems, 30*.

Wang, H., Ma, S., Dong, L., Huang, S., Wang, H., Ma, L., et al. (2023). *BitNet: Scaling 1-bit transformers for large language models*. arXiv. https://arxiv.org/abs/2310.11453

Wang, J., et al. (2024). *1-bit AI infra: Part 1.1, fast and lossless BitNet b1.58 inference on CPUs*. arXiv. https://arxiv.org/abs/2410.16144

Whittington, J. C. R., & Bogacz, R. (2017). An approximation of the error backpropagation algorithm in a predictive coding network with local Hebbian synaptic plasticity. *Neural Computation, 29*(5), 1229–1262.

Verification note. These entries were compiled from memory, not from a bibliographic database. Confirm every author list, year, venue, page range, and identifier before submission, and expand “et al.” lists as APA 7 requires. The two sources cited in the earlier draft (an OAPEN library listing and an alphaXiv report) were not used.

Appendix A. Verification Register

Status labels follow Table 1. “Earlier drafts” refers to the source document this paper replaces.

ID	Claim in earlier drafts	Status	Section	Action
V-01	Attention compute $O(N^2)$; KV memory grows as $O(N^2)$	M	2.1	Compute $O(N^2)$ per sequence; cache $O(N)$
V-02	32 GB KV cache at 32k tokens for 70B	M	2.1	10.7 GB (GQA) or 85.9 GB (MHA); recompute per config
V-03	FP arithmetic is non-associative	S	2.2	None
V-04	Multipliers use 70–80% of AI processor energy	C	2.2, 5.3	Drop; weight movement dominates at batch 1

ID	Claim in earlier drafts	Status	Section	Action
V-05	Backward locking and weight transport	S	2.3	Frame as plausibility problem, not GPU accuracy problem
V-06	dim Cl(p,q) = 2^n; grade dimensions are binomial	S	4.2	Unit test
V-07	Rotors preserve norm and prevent explosion	M	4.2	True only for same-sign bivectors; test
V-08	Signature (3,2) enforces a causal cone	C	4.3	Use (m+1, 1)
V-09	Light cone replaces masking; topological invariance	M	4.3	Learned extra restriction on stored events
V-10	Stabilizer correction Ψ − Σσ_kS_kΨ	M	4.4	Use Equations 11/13; test idempotence
V-11	Topological protection against rounding noise	U	4.4	Test H-S
V-12	4-bit phase LUT removes all floating-point logic	M	4.5	Accumulation needs Cartesian adders
V-13	Calabi–Yau cycles store information without using memory	C	4.5	Drop; test H-K surrogate
V-14	CY Laplacian spectrum $m^2 \propto n(n+d)/R^2$	C	4.5	Sphere formula; drop
V-15	Eleven dimensions is the natural limit for the manifold	C	4.2	No ML basis; n is a hyperparameter
V-16	Schedule yields 80/15/5, 20/60/20, 5/15/80	M	4.6	Approximate at λ ≈ 0.15, 0.5, 0.85 only
V-17	80% of tokens take a free fast path	U	4.6	Design target; $\bar{f} = 0.44$, not 0.2
V-18	Skew-Hermitian coupling gives a Hopfield energy	C	4.7	Replace with Equation 23
V-19	Langevin noise scale $\sqrt{(2T\,dt)}$	S	4.7	Unit test
V-20	Predictive coding removes weight transport	C	4.8	Errors still use W^T

ID	Claim in earlier drafts	Status	Section	Action
V-21	Energy table for 7/5 nm with MAC entries	M	5.3	45 nm multiply/add figures; relabeled
V-22	0.42 mJ/token; 80× energy reduction	C	5.3	Off-chip DRAM at batch 1: ≈ 0.25–1.2 J; weight-byte gain 8–18×
V-23	17.5 GB total inference memory	M	5.2	Packed body only; add embeddings, state, scales
V-24	720 Wh pack gives 28.8 h at 25 W	S	5.4	Arithmetic correct; earlier “18+ h” inconsistent
V-25	25 W runs full 70B-grade reasoning	U	5.4	Measure
V-26	5–15B-token distillation in days on one GPU	C	6.4	About 85 days at assumed 10 TFLOP/s
V-27	Training fits under 9.5 GB VRAM	U	6.4	About 16 B/param without 8-bit states
V-28	SVD maps top singular vectors to ternary, rotations to skew	U	6.2	Use absmean and polar decomposition
V-29	Ternary packing; 1B model in 250 MB	M	5.2, 6.5	Packing is standard; add embeddings (about 0.8 GB)
V-30	CPU ternary inference 2–4× faster than GPU at 1/10 power	M	6.5	Gains were relative to CPU baselines
V-31	State-space models have O(1) per-token inference	S	3	None
V-32	2,048 components per channel with memory savings	C	4.2	Truncate grades or restrict to even subalgebra

Appendix B. Changes from the Source Document

Item	Earlier draft	This paper	Reason
Scope	Sections on Bitcoin ASICs, TPU clusters, consumer clustering	Omitted	Out of scope by request

Item	Earlier draft	This paper	Reason
Framing	Current architectures “obsolete”	“Identifiable bottlenecks”	Transformers remain state of the art
Terminology	“Quantum” subspace	“Phase” subspace	No quantum computation involved
Scale coordinate	V with negative sign (signature 3,2)	Spacelike V (signature m+1, 1)	Two time directions break causality
Light cone	Replaces masking, “invariance guarantee”	Learned additional constraint	Position tie and O(1) state make it vacuous otherwise
Stabilizer	Ψ − Σσ_kS_kΨ	Projector form (Equations 11–13)	Original sign moves state off the code space
Attractor energy	Skew-Hermitian quadratic form	Modern Hopfield energy	Original quadratic term is zero or imaginary
Predictive coding	“No transposed matrices queried”	Errors use W^T; update is local	Derivation of Equation 26–28
Energy table	7/5 nm “MAC” values	45 nm multiply/add values	Provenance and labeling
Benchmarks	Hypothetical 70B comparison tables	Assumption-driven estimates (Tables 7, 9)	Presented as measurement without data
KV cache	32 GB at 32k	10.7 GB (GQA)	Equation 2
Battery runtime	18+ h and 28.8 h	28.8 h	Equation 30
Distillation compute	Days	Weeks to months	6ND estimate
SVD rule	Singular vectors to ternary/rotation	Absmean and polar decomposition	Original rule undefined
Dimensionality	11D as theoretical ceiling	n as hyperparameter; n = 11 kept as a variant	Physics bound not an ML bound
Code	Presented as verified	Not reproduced; defects listed in Appendix C	Defects found by inspection

Appendix C. Defects in the Earlier Reference Prototypes

Found by reading the code; none were confirmed by execution, which should be done in Tier 0.

#	Location	Defect	Consequence
C1	All THME layer classes	No token-mixing operator: every sub-block acts independently on each position	As written the network is a per-token model; it cannot use context
C2	THMELayer constructor at depth index 0	$\lambda = 0$ gives $\alpha = 1$, $\beta = \gamma = 0$, so d_bin = d_model, d_ter = 1, d_qua = −1	Negative layer width; construction should fail, contradicting the “verified” label
C3	5D / 11D expansion	fused.unsqueeze(-1).repeat(…, 5 or 11) copies one value onto every coordinate	Metric interval reduces to a constant multiple of x^2; the geometry carries no information
C4	Topological stabilizer	Fixed random unit vectors, correction scaled by 0.1, followed by a mean over coordinates	Equivalent to a fixed linear map; not a stabilizer projection
C5	Binary path	torch.sign has zero gradient almost everywhere and no straight-through estimator	Binary projection receives no gradient
C6	Router	mean_surprise > 0.5 over the whole batch, with .item()	All-or-none routing; non-differentiable; synchronization stall
C7	Attractor	$A - A^T$ used without exp, tanh in the rotational term, LayerNorm at output, noise only in training	Not norm-preserving, not an energy descent
C8	Quantizer, version 1	w_eff = (w_ternary − w).detach() + w, then multiplied by γ	Gradient scaled by γ instead of 1; use the scaled weights in the offset
C9	Quantizer, version 2	Backward zeroes gradient where	w

#	Location	Defect	Consequence
C10	Hardware module	math used (math.sqrt) without import; FastBitLinear defined in several listings	NameError and inconsistent definitions
C11	Transmuted wrapper	Layer stack commented out: logits = lm_head(embed(x))	Not a model; the distillation harness trains nothing meaningful
C12	Test harness	Checks only shapes and absence of exceptions; “zero floating-point divergence” is a printed string	Not a verification

Appendix D. Implementable Techniques and Alternative Approaches by Component

This appendix takes each component of THME and lists (a) techniques that can be implemented today on commodity GPUs and CPUs, (b) alternative approaches that pursue the same goal, and (c) the cheapest experiment that would decide between them. Method names and years are given so each can be looked up; none is in the reference list above, and each should be verified and added before submission. Register IDs (V-xx) point back to Appendix A.

D.1 Low-Bit Arithmetic (Ternary and Binary Paths)

Implement now.

- Quantization-aware training with absmean ternary weights, 8-bit per-token activations, and a straight-through estimator (the BitNet b1.58 recipe). Keep a BF16 master copy; compute the clipping test on w/γ, not on w (Appendix C, C9).
- A normalization layer placed before each quantized linear map, as in BitNet, which stabilizes activation scale for quantization.
- Packed inference: 2-bit packing with lookup-table kernels on CPU (the approach of bitnet.cpp and T-MAC) and fused unpack-and-multiply Triton or CUDA kernels on GPU (for example Microsoft’s BitBLAS).

Alternatives.

- Post-training quantization of an existing model to 4 bits with GPTQ (2022) or AWQ (2023). No retraining, strong quality, a 4× weight reduction versus FP16, though short of the 8× of ternary.

- Outlier-aware 8-bit (LLM.int8(), 2022) and rotation-based outlier removal (QuaRot, SpinQuant, 2024), which make 4-bit weights and activations viable.
- 2-bit vector or codebook quantization (AQLM, QuIP#, 2024), which reaches about 2 bits per weight with a smaller quality loss than scalar ternary rounding of a pre-trained model.
- Native low-precision floating point (FP8 on current GPUs, FP4 microscaling formats on newer ones), which needs no new kernels.
- QLoRA-style fine-tuning (4-bit frozen base plus low-rank adapters) when only adaptation is needed.

Trade-off. Ternary gives the largest compression but is the only option here that usually needs substantial retraining. If the goal is a working low-memory model soon, a 4-bit post-training route is the lowest-risk baseline and should be the comparison point for any ternary claim (V-29, V-30).

First experiment. Quantize one 1B model three ways (GPTQ-4bit, AWQ-4bit, ternary QAT with 0.5B tokens) and plot perplexity against bytes per weight.

D.2 Token Mixing Without a Growing Cache (The Missing Operator)

This is the highest-priority gap (Appendix C, C1).

Implement now.

- A selective state-space layer (Mamba or Mamba-2) or a gated linear recurrence, which provides O(1) state per token.
- A **hybrid** stack: mostly recurrent layers with a small number of full-attention layers (for example one in six to eight), as in Jamba-style designs. Attention layers handle exact recall; recurrent layers handle bulk sequence mixing.
- Sliding-window attention plus a few global or sink tokens (Longformer, Mistral, StreamingLLM), giving a bounded cache.

Alternatives.

- Other recurrent mixers: RWKV, RetNet, xLSTM, linear attention (Katharopoulos et al., 2020), and delta-rule variants (DeltaNet, Gated DeltaNet), which add an explicit write-and-overwrite memory update.
- Keep attention but shrink the cache: grouped-query attention (already assumed in Table 2), multi-head latent attention (DeepSeek), KV quantization to 4–8 bits, and eviction policies such as H2O. Paged cache management (vLLM) removes fragmentation.
- Treat modern Hopfield retrieval (Ramsauer et al., 2021) as an attention variant: an associative memory read in one step.

Trade-off. Pure constant-state models are known to be weaker at exact recall of arbitrary earlier tokens. The claim of strictly zero KV cache with no quality loss is therefore the least likely part of the proposal to survive testing; a hybrid with a small attention budget is the realistic target.

First experiment. Multi-query associative recall (the MQAR task from the Zoology study, 2023), copy, and needle-in-a-haystack, comparing pure recurrent, hybrid, and attention-only models at matched size.

D.3 Adaptive Depth and Per-Token Routing

Implement now.

- Early exit with a shared or lightweight language-model head at the exit layer, trained with an auxiliary loss on exit-layer outputs (LayerSkip, 2024; CALM, 2022).
- **Self-speculative decoding.** Use the early-exit path as the draft model and the full-depth path as the verifier. Because the verifier checks draft tokens with the standard acceptance rule, the output distribution equals that of the full model, so quality is preserved exactly. With acceptance rate a, draft length γ, and draft cost ratio c, the expected speedup is approximately

(1 − a^(γ+1)) / ((1 − a) (γc + 1)) (D1)

For c = 0.3, γ = 4: a = 0.8 gives about 1.5× and a = 0.9 about 1.9×. Verification reads the weights once for several tokens, which also helps the bandwidth-bound regime of Section 5.3. Treat these as model outputs to be measured, not results.

- Token-level capacity routing as in Mixture-of-Depths, with a fixed top-k per layer so that tensor shapes stay static.

Alternatives.

- **Mixture-of-Experts** (Switch Transformer, Mixtral-style). Active parameters per token, hence weight bytes per token, fall by routing to a few experts. This reaches the same goal as the 0.44 depth fraction with mature training recipes and kernels; it can also be combined with ternary experts.
- Adaptive computation time (Graves, 2016), PonderNet, and Universal Transformers, which learn a halting probability per token.
- Plain speculative decoding with a small separate draft model.
- Recurrent-depth models (weight-tied blocks iterated a variable number of times), which spend extra compute only at test time on hard inputs.

Trade-off. Learned routers collapse unless regularized; self-speculation sidesteps the quality risk entirely at the cost of a smaller (but guaranteed) gain. It should be the default deployment mode for the exit gate.

First experiment. Measure per-token exit-layer agreement with the full model on a held-out set. If the exit head already matches the full model on a large fraction of tokens, the 80/20 premise (V-17) has support.

D.4 Geometric State Space (Clifford and Alternatives)

Implement now.

- A Clifford linear layer built from a precomputed Cayley (multiplication) table stored as a sparse tensor, so the geometric product is one einsum. Restrict to grades 0–2 or the even subalgebra to control cost (Table 5).
- Small blocks (n = 3 or 4 generators) applied to groups of channels rather than one 2,048-component algebra.
- Test first on data with real geometric structure (particle systems, point clouds, rigid-body dynamics), where Clifford and geometric-algebra transformers have published gains, before testing on language.

Alternatives.

- Rotary position embeddings (RoPE): the rotation-group idea already used in nearly every current LLM, applied to positions.
- Orthogonal and unitary parametrizations that keep norms bounded without a full algebra: matrix exponential of a skew-symmetric matrix, Cayley transform, or products of Householder reflections.
- Complex-valued or quaternion layers, which supply rotational structure at 2× or 4× width instead of 2^n.
- Group-equivariant networks, when a specific symmetry group is known.

Trade-off. For text there is currently no evidence that a multivector state helps, and the cost grows exponentially in n. The simplest alternative with a norm guarantee is an orthogonal parametrization (V-07, V-32).

First experiment. Replace one dense layer with (i) a grade-0–2 Clifford layer, (ii) an orthogonal layer, and (iii) an equal-parameter dense layer; compare on a small language task and on an N-body prediction task.

D.5 Noise Control and Stabilization

Implement now.

- A learned orthogonal or low-rank projection in place of the random stabilizers (Equation 11), trained end to end, with an idempotence penalty $\|\Pi^2 - \Pi\|$.
- Quantization-friendly normalization (RMSNorm before each low-bit linear map, as in BitNet) as the control condition.

Alternatives.

- QK-normalization and logit soft-capping for attention stability.
- Hadamard or random-rotation smoothing of activations before quantization, which spreads outliers.
- Per-group scale factors and error-feedback quantization.
- Stochastic rounding, which makes rounding error zero-mean.
- Spectral normalization or weight standardization for bounded layer gain.

First experiment. The H-S test of Table 11: quantization error against an FP32 reference, with and without the projection, at equal bit-width (V-10, V-11).

D.6 Phase and Complex Representations

Implement now. The natural home for a magnitude-and-phase state is the recurrence of Section D.2. A diagonal complex linear recurrent unit uses

$$h_t = \lambda \odot h_{t-1} + B x_t, \lambda_j = \exp(-\nu_j + i\theta_j), \nu_j > 0 \quad (D2)$$

so that $|\lambda_j| < 1$ and the recurrence is stable by construction, while θ_j sets an oscillation frequency. This is the parametrization used by complex state-space models and the Linear Recurrent Unit (Orvieto et al., 2023). Quantizing θ_j to 4 bits (Equation 14) then becomes a clean, testable question: how much does phase rounding degrade long-range recall?

Alternatives.

- Real-valued diagonal recurrences (Mamba uses real decay), which drop phase entirely.
- Fourier-feature or RoPE-style phase encodings for positions.
- Full complex networks (Trabelsi et al., 2018) when phase carries signal.

First experiment. Real-valued versus complex-diagonal recurrence, then complex with 4-, 6-, and 8-bit phases (V-12).

D.7 Iterative Reasoning (The Attractor Block)

Implement now.

- A modern Hopfield retrieval layer (Equation 23) with M learned memories, run for K = 1–3 steps.
- A weight-tied block iterated K times with a learned halting signal (looped or recurrent-depth transformers). If the update map has Lipschitz constant below 1, the fixed point is unique and iteration converges (Banach):

$$\|f(z) - f(z')\| \leqslant L_f \|z - z'\|, L_f < 1 \Rightarrow \text{unique fixed point} \quad (D3)$$

Spectral normalization of the weights can enforce $L_f < 1$.

Alternatives.

- Deep equilibrium models (Bai et al., 2019): solve for the fixed point directly and differentiate implicitly, avoiding unrolled gradients.
- Neural ODE layers with an adaptive solver.
- Chain-of-thought or other token-level test-time compute, which spends extra tokens rather than extra latent iterations and needs no new architecture.
- Energy-based and diffusion-style iterative refinement of a draft.

Trade-off. Iterative latent computation is an active research area, but token-level reasoning is currently the more reliable route. Gradients through K steps are costly; one-step or truncated gradients reduce memory.

First experiment. Add the Hopfield layer to a small model and test on tasks requiring multi-hop lookups; compare with simply adding K more ordinary layers at equal FLOPs (V-18).

D.8 Training Without Global Backpropagation

Implement now (solving the real memory problem). The practical cost of backpropagation is activation memory, and this is already addressable: gradient checkpointing, reversible layers, 8-bit optimizer states, LoRA, and gradient accumulation. For a 1B model these cut training memory far more than any change of learning rule (V-27).

Alternatives that change the learning rule.

- Feedback alignment and direct feedback alignment (Lillicrap et al., 2016): random fixed feedback weights, no transposes.
- Forward-forward (Hinton, 2022), target propagation, and equilibrium propagation (Scellier & Bengio, 2017).
- Local layer-wise losses and synthetic gradients (Jaderberg et al., 2017), enabling asynchronous layer updates.
- Zeroth-order fine-tuning (MeZO, 2023), which trains with inference-level memory using only forward passes, at the cost of many more steps.
- Predictive coding (Section 4.8), still the most biologically motivated but compute-heavy.

Trade-off. None of these has matched backpropagation at LLM scale. The right use is for specific substrates (analog or neuromorphic hardware) rather than for GPU training (V-20).

D.9 Converting a Pre-Trained Model

Implement now.

- Initialization by absmean ternarization (Section 6.2), then distillation with logit KL and hidden-state matching, in stages: one block, then a few blocks, then the full stack.
- Replace attention layer by layer with a recurrent mixer, distilling each layer's output from the teacher's attention output (layerwise distillation), as in published attention-to-SSM conversions such as Mamba-in-the-Llama and LoLCATs.
- Reuse the published recipe for fine-tuning to 1.58 bits (the Hugging Face fine-tuning study and BitDistill-style approaches) as a starting point and measure how much data they needed.

Alternatives.

- Train a small model from scratch on a high-quality curated corpus rather than converting.
- Keep attention and convert only the MLPs to low-bit, the lowest-risk first step.
- LoRA adapters over a 4-bit frozen base.
- Structured pruning plus distillation, which reduces parameters rather than precision.

First experiment. MLP-only ternary conversion of one 1B model with 0.5B distillation tokens; measure the gap to the teacher before attempting attention replacement (V-26, V-28).

D.10 Systems and Hardware Paths

Implement now.

- CPU: packed ternary kernels (bitnet.cpp, T-MAC), NEON and AVX-512 lookup tables.
- GPU: fused packed-weight kernels in Triton or BitBLAS; speculative decoding to raise arithmetic intensity at batch 1.
- Apple silicon: unified memory with MLX for quantized inference.
- Serving: continuous batching and paged KV memory (vLLM) when batch size can be raised, since batching is the dominant way real systems amortize weight reads.

Alternatives.

- FPGA implementations of ternary datapaths.
- On-chip SRAM weight-stationary designs for small models.
- Compute-in-memory arrays (RRAM, FeFET) and analog inference, where noise-aware training is required; this remains research hardware.

Decision rule. Use Equation 29 with measured memory bandwidth to predict tokens per second before writing kernels; if the prediction is bandwidth-bound, byte-reduction techniques (low bits, MoE, early exit with speculation) pay off and arithmetic tricks do not.

D.11 Causality and Long-Range Structure

Implement now. A standard causal mask plus a distance-dependent bias (ALiBi, 2022, adds a head-specific linear penalty in distance) implements a soft version of the cone of Equation 10: influence decays with temporal separation at a learned rate.

Alternatives. Hard sliding windows with global tokens; block-sparse or natively sparse attention patterns; learned retrieval of distant blocks (retrieval-augmented or memory-token designs).

D.12 Evaluation Tasks That Test Each Claim

Claim under test	Task or measurement
Constant-state recall is adequate	MQAR, copy, needle-in-a-haystack, Long Range Arena
Exit gate identifies easy tokens	Exit-layer agreement with full model; speculative-decoding acceptance rate
Attractor aids multi-step inference	Multi-hop lookup, multi-digit arithmetic, parity
Clifford features help	N-body or point-cloud prediction first, then small-scale language
Low-bit conversion retains function	Perplexity and a standard benchmark suite relative to the teacher
Resource model is valid	Joules per token and tokens per second at stated batch size

D.13 Prioritization

Table D1

Techniques Ranked by Expected Payoff Against Risk

Priority	Technique	Effort	Expected payoff	Risk	Evidence base
1	4-bit PTQ baseline (GPTQ/AWQ)	Low	4× weight reduction, near-lossless	Low	Strong

Priority	Technique	Effort	Expected payoff	Risk	Evidence base
2	Self-speculative decoding with early-exit draft	Medium	Lossless speedup of 1.5–2× (projected)	Low	Strong
3	Hybrid recurrent/attention mixer	Medium	Smaller cache, bounded recall loss	Medium	Strong
4	Ternary QAT / MLP-only conversion	Medium	8× weight reduction	Medium	Moderate
5	Complex-diagonal recurrence with quantized phase	Medium	Test of H-phase	Medium	Moderate
6	Hopfield or looped iterative block	Medium	Possible reasoning gain	High	Weak
7	Clifford layers on language	High	Unknown	High	Weak for text
8	Discrete-torus register, stabilizer projection	Medium	Unknown	High	None
9	Predictive-coding or local-learning training	High	Hardware-specific	High	Weak at scale

D.14 Suggested Minimal Viable THME

A version that can be built and measured in weeks, using only components with prior support:

1. A 100–300M-parameter hybrid model (recurrent mixer with a few attention layers).
2. Ternary or 4-bit weights with QAT, using RMSNorm before each quantized map.
3. An early-exit head at about 30% depth, deployed through self-speculative decoding.
4. An optional complex-diagonal recurrence with quantized phase as the first “phase subspace.”

Add the attractor, Clifford, stabilizer, and torus components one at a time only after this baseline is measured, each against the falsification thresholds of Table 11. This ordering keeps every later claim attributable to a single change.

Appendix E. The Consensus-Read Mixer (CRM): A Proposed Token-Mixing Operator

This appendix specifies the missing token-mixing operator (Appendix C, C1) around the idea proposed in review: each token produces several readings (4 to 8) of a persistent state, half of them proceed to the next layer, and the rest are routed through ternary and binary paths with a loop-back, so that noise is reduced and less computation is wasted.

The proposal is translated into a concrete operator, tested numerically where possible, and revised where the numbers did not support it. The word “quantum” is not used: a reading here is an ordinary stochastic read-out of a classical state.

E.1 Specification (CRM v0)

State and write. Each layer and head keeps a fast-weight matrix memory M_t of size $d_v \times d_k$ that is updated once per token with a gated delta rule (the form used by DeltaNet and Gated DeltaNet):

$$M_t = \alpha_t M_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top, \|k_t\| = 1, \beta_t \in [0, 1], \alpha_t \in (0, 1] \quad (E1)$$

The factor $(I - \beta k k^\top)$ has spectral norm at most 1, so $\|M_t\| \leqslant \alpha_t \|M_{t-1}\| + \beta_t \|v_t\|$ and the state stays bounded for $\alpha < 1$. The state size is constant in sequence length: bytes $= L \cdot H \cdot d_k \cdot d_v \cdot b$, for example 32 layers × 32 heads × 128 × 128 at FP16 is about 33.5 MB. The state is kept at higher precision (FP16 or INT8) because it is small; the large weight matrices are the ones quantized to ternary.

Reading. One high-precision read $y_t = M_t q_t$ is computed per head. K readings are then produced by K independent stochastic low-bit quantizers Q_j (dithered or stochastic rounding):

$$r_{t,j} = Q_j(y_t), j = 1, \ldots, K, K \in \{4, 8\} \quad (E2)$$

Consensus. The consensus c_t is the element-wise median of the K readings, and the dispersion is

$$\sigma_t^2 = (1 / (K d_v)) \Sigma_j \|r_{t,j} - c_t\|^2 \quad (E3)$$

Split by agreement (K = 8 shown). Readings are ranked by distance to c_t. The nearest half (4) are inliers and proceed directly. The next quarter (2) go to a ternary repair path. The farthest quarter (2) go to a binary gating path.

- Inliers I: weight 1.
- Ternary repair T: $r'_j = r_j + \text{BitLinear}_3([r_j - c_t ; c_t])$, then weight $w_j = \exp(-\|r'_j - c_t\|^2 / \tau)$.
- Binary gate B: element-wise mask $g_{j,i} = 1[\text{sign}(r_{j,i}) = \text{sign}(c_{t,i})]$, weight g_j.

The mixed output is an element-wise weighted consensus:

$$z_{t,i} = \Sigma_j a_{j,i} \tilde{r}_{j,i} / \Sigma_j a_{j,i}, a = 1 \text{ on I, } w_j \text{ on T, } g_{j,i} \text{ on B} \quad (E4)$$

where $\tilde{r}$ is the repaired reading on T and the raw reading elsewhere.

Loop-back as sequential sampling. If $\sigma_t > \varepsilon$ after the first K readings, draw K' additional readings and recompute E3–E4, up to I_{max} rounds (default 2). Only reads are repeated. The write E1 happens once per token, so loop-back never reads or writes any future token and causality is preserved.

Routing signal. The dispersion σ_t can feed the exit gate of Equation 20 (Section 4.6). The next sections show this needs care.

E.2 Numerical Check of the Noise Claims

Three claims embedded in the proposal were tested by simulation (Gaussian and contaminated noise, 200,000 trials each; script run in this session, code not reproduced here).

Table E1

Mean Squared Error of Estimators over K Independent Readings (Read Noise SD = 1)

Noise model	K	Mean of all	Median	Mean of nearest half
Gaussian	4	0.250	0.299	0.299
Gaussian	8	0.125	0.168	0.227
20% outliers (SD 5)	4	1.457	0.733	0.733
20% outliers (SD 5)	8	0.725	0.269	0.322

Table E2

Effect of a Shared Stored-State Error (SD 0.5, Floor 0.249) with Independent Read Noise (SD 1)

K	1	2	4	8	16	64
MSE	1.251	0.753	0.497	0.374	0.311	0.264

Table E3

Reconstruction RMSE of a Value in [0, 1]: Averaged Stochastic 1-Bit Readings Versus Deterministic Quantization

K	Stochastic, K readings	Deterministic, K+1 levels	Deterministic, 8-bit
4	0.204	0.072	0.0011
8	0.144	0.036	0.0011
16	0.102	0.018	0.0011
64	0.051	0.0045	0.0011

E.3 What the Results Support

1. **Averaging works only on independent noise.** With independent Gaussian read noise the mean of K readings has MSE 1/K (0.125 at K = 8), as expected. Error that is stored in the state is identical in every reading and cannot be averaged away: in Table E2 the MSE approaches the 0.249 floor and gains beyond K = 8 are small. This is why the state is kept at higher precision (E1).
2. **Keeping only the best half is not generally better.** For Gaussian noise, the nearest-half mean is worse than the plain mean (0.227 against 0.125 at K = 8) because it discards usable data. It wins only when noise is heavy-tailed: with 20% outliers it cuts MSE from 0.725 to 0.322, and the plain median is better still (0.269). The 50% split is therefore justified only if read noise has outliers, which is a property to measure, not assume.
3. **Multi-reading is not cheaper precision on digital hardware.** Averaging K stochastic 1-bit readings has error that falls as $1/\sqrt{K}$. At K = 8 the RMSE is 0.144, four times worse than deterministic rounding to the same 9 levels (0.036) and about 130 times worse than 8-bit (0.0011). This is the known behavior of stochastic computing. The approach trades time for precision at a poor exchange rate, so “less waste” does not hold for a digital implementation; it may hold on analog or compute-in-memory hardware where reads are noisy regardless and a multi-read average is the way to recover accuracy.
4. **Dispersion of dithered readings is not a semantic surprise signal.** The spread of stochastic roundings of the same y depends on where y falls relative to the quantization grid, not on how hard the token is. Using it for routing would route by rounding phase. Only disagreement between **distinct** estimators trained toward one target (an ensemble of heads) can plausibly carry epistemic uncertainty (V-36).

E.4 Revised Design (CRM v1)

The results lead to four changes.

1. **Two kinds of readings.** Use G distinct read heads (G = 2 or 4), each with its own query projection, trained with a consensus loss, plus m dithered quantizations per head only if measured read noise is significant. $K = G \cdot m$ stays within 4 to 8. Head disagreement drives routing; dither averaging, if used, only cleans noise.
2. **Median-based consensus by default; the 50% split is conditional.** Keep the inlier/ternary/binary partition only if the measured read-noise distribution is heavy-tailed. Otherwise use the plain weighted mean (E4 with all weights 1), and spend the saved effort on the repair path.
3. **Loop-back as adaptive sampling.** Draw extra readings only for tokens whose interim dispersion is above ε. The expected reads per token become $K_{min} + P(\sigma > \varepsilon) \cdot \Delta K$, which is where genuine savings could arise relative to a fixed K for every token.

4. **Precision budget.** State at 16-bit, queries and projections ternary, readings at 2–4 bits. The binary path is a sign gate (a one-bit mask), which costs almost nothing and is where the binary hardware primitive is used.

Cost per token and head: write $O(d_k d_v)$, one read $O(d_k d_v)$, K quantizations $O(K d_v)$, repair on $K/4$ readings $O((K/4) d_v h)$ for a small hidden size h, and adaptive extra readings $O(\Delta K d_v)$. Dithering adds no extra matrix–vector product because the high-precision read y_t is computed once.

E.5 Training

- **Gradient through quantizers.** Use a straight-through estimator for Q_j (identity backward, with clipping).
- **Gradient through the split.** Forward uses the hard rank-based partition. Backward uses the softmax relaxation $a_j = \mathrm{softmax}(-d_j / \tau)$ so that readings near a partition boundary receive gradient.
- **Losses.** $L = L_{task} + \lambda_1 L_{cons} + \lambda_2 E_t[\text{reads}_t]$, where $L_{cons} = \text{mean } \|r_j - \text{stopgrad}(c_t)\|^2$ over head readings and the last term penalizes the number of extra readings.
- **Initialization.** α near 1, β small, so early training behaves like a plain linear recurrence.

E.6 Alternatives for Each Internal Choice

Component	CRM choice	Alternatives
Memory update	Gated delta rule	Plain linear attention state; Mamba selective state; complex-diagonal recurrence (Section D.6); RWKV-style decay
Read	Multiple heads over one memory	Separate memory per head for more diversity ($G\times$ state); Hopfield-style iterated read (Equation 23)
Noise reduction	Dither averaging	Larger activation bit-width; random Hadamard rotation before quantization; error-feedback quantization
Consensus	Median / weighted mean	Trimmed mean; learned attention over readings; majority vote for binary readings
Repair	Ternary MLP on outliers	Skip connection that ignores outliers; re-read with a refined query $q' = q + W_r (c - r)$

Component	CRM choice	Alternatives
Routing signal	Between-head dispersion	Learned gate on features (Equation 20); exit-head entropy; speculative-decoding acceptance
Loop-back	Adaptive extra readings	Fixed K; adaptive computation time halting

E.7 Tests and Falsification

Table E4

CRM Hypotheses and Falsification Criteria (Proposed)

Hypothesis	Test	Falsified if
H-C1: CRM recalls as well as a plain gated delta layer	MQAR, copy, needle-in-a-haystack at matched parameters	No gain, or a loss, relative to the delta layer without readings
H-C2: Read noise is heavy-tailed enough to justify the split	Measure the distribution of reading errors against the high-precision read	Kurtosis near Gaussian, so the nearest-half mean underperforms the mean
H-E: Head disagreement predicts hard tokens	AUROC of head dispersion for predicting top-decile token loss	AUROC below about 0.6
H-C3: Adaptive loop-back saves reads	Mean reads per token at equal quality against fixed K	No saving at equal quality
H-C4: Repair path adds value	Ablation removing ternary/binary paths	No quality difference
Overall	Quality against a Mamba-2 and a gated delta baseline at equal FLOPs	CRM does not beat both, or costs more FLOPs for the same quality

The first implementation step is a PyTorch reference of E1–E4 with unit tests: boundedness of E1 over long random sequences, variance scaling of E2 against Table E1, causality (changing token t+1 never changes outputs at t), and a throughput measurement of the K quantizers.

E.8 Register Additions

ID	Claim	Status	Note
V-33	Averaging K readings reduces noise as 1/K	S for independent noise; C for state error	Table E2: floor from shared error
V-34	Keeping the best half of readings improves accuracy	U	Better only for heavy-tailed noise (Table E1)
V-35	Multiple stochastic readings are cheaper than extra precision	C on digital hardware	Table E3
V-36	Dispersion of dithered readings signals token difficulty	C	Only head disagreement can; H-E
V-37	Loop-back as adaptive re-reading saves computation	U	H-C3
V-38	Gated delta state is bounded	S	Derivation after Equation E1

Appendix F. Optimization-Driven Allocation and Self-Tuning

This appendix examines three proposals made in review: (1) use linear and non-linear programming to choose the binary/ternary/phase fractions (α, β, γ) of each layer instead of a fixed schedule; (2) use the Northwest-corner method and the travelling salesman problem (TSP) “in a smart way”; and (3) let the model tune its own configuration in repeated rounds until the result is frozen (“hardcoded”), with accuracy approaching 1 like 0.99999…. Each is formalized, tested on toy problems where possible, and corrected where the numbers or theory disagree. All toy figures use assumed distortion models and are illustrations of structure, not results for any real network.

F.1 Layer-Level Allocation as a Linear Program

Let x_{l,s} ⩾ 0 be the fraction of layer l’s channels assigned to format s ∈ {binary, ternary, phase}. These lie on a simplex, Σ_s x_{l,s} = 1, and the earlier schedule (Equations 16–19) is one hand-chosen point in that space. Let Ω_l be the sensitivity of layer l (for instance a Hessian-trace or Fisher estimate, as in HAWQ-style mixed-precision work), q_s the distortion per channel of format s, and b_s its bits per channel. With a global average-bit budget B the problem is

minimize $\Sigma_l \Omega_l \Sigma_s q_s x_{l,s}$ subject to $\Sigma_s x_{l,s} = 1$ for all l, $(1/L)\ \Sigma_l \Sigma_s b_s x_{l,s} \leqslant B$, $x \geqslant 0$ (F1)

This is a linear program. Its structure carries a consequence that is easy to miss. A basic feasible solution has at most as many non-zero variables as constraints, here $L + 1$. Every layer needs at least one non-zero entry, so **at most one layer can contain more than one format**; all other layers are pure. A toy run (12 layers, Ω log-normal, $q = 1.0 / 0.30 / 0.02$, $b = 1 / 1.58 / 4$, $B = 2.0$) confirmed this: 11 layers were pure and one was split 92% ternary / 8% phase. The overall fractions were 0 / 83 / 17 %.

Three conclusions follow.

1. A linear distortion model produces “bang-bang” allocations, not interior mixes such as 10/30/60. If interior mixes are wanted per layer, the model must be non-linear (a convex distortion that grows faster than linearly in the fraction assigned to a coarse format) or the unit of allocation must be the channel, not the layer (F.2).
2. The result depends entirely on the sensitivity estimates Ω_l and distortions q_s. These must be measured on the actual network; the toy numbers carry no weight.
3. When channel counts must be integers the problem becomes an integer linear program (ILP). At this size (tens of layers) modern solvers solve it exactly in milliseconds.

F.2 Channel-Level Allocation and the Calculus of Bit Budgets

The layer is too coarse a unit: channels within a layer differ widely in importance. At high resolution a channel with variance (or sensitivity) σ_i^2 quantized with b_i bits has distortion about $\sigma_i^2\ 2^{(-2\ b_i)}$. Minimizing total distortion for a total bit budget $n \cdot B$ uses a Lagrange multiplier and gives equal distortion across channels, hence

$b_i = B + \tfrac{1}{2} \log_2(\sigma_i^2 / \sigma_g^2)$, σ_g^2 = geometric mean of the σ_i^2 (F2)

This is classical reverse water-filling from rate–distortion theory: high-sensitivity channels receive more bits, low-sensitivity channels fewer, and channels below the “water level” receive none. Because real formats are discrete (1, 1.58, 4 bits), the practical solution is a Lagrangian relaxation: for a price λ per bit, each channel independently picks the format minimizing $D_i(b) + \lambda\ b$, and λ is bisected until the budget is met.

The key insight is that **the fractions (α, β, γ) are outputs of the optimization, determined by the distribution of channel sensitivities, not inputs**. In a toy run (20,000 channels, log-normal sensitivities with log-SD 1.5, average budget 2.0 bits, distortion model $D = \sigma^2\ 2^{(-2b)}$, which is crude at 1 bit) the optimizer chose 33 / 42 / 25 % (binary/ternary/phase).

Table F1

Toy Channel Allocation: Optimized Versus Fixed Mixes (Assumed Distortion Model)

Allocation	Average bits	Total distortion
Optimized (33 / 42 / 25 %)	2.00	1,781
Random 33 / 34 / 33 %	2.19	7,283
All ternary	1.58	6,789
Random 10 / 30 / 60 %	2.98	3,796
Random 80 / 15 / 5 %	1.23	13,048

The optimized allocation reaches lower distortion than the 10/30/60 mix while using one third fewer bits, because it places precision where sensitivity is high instead of by position. The comparison holds under the assumed model only; the size of the gain depends on how heavy-tailed real channel sensitivities are. This is the central practical recommendation of this appendix, and the cost is one sensitivity measurement pass plus one ILP.

F.3 Where the Northwest-Corner Method and Transportation Problems Fit

A transportation problem has sources with supplies, sinks with demands, and a cost per unit shipped. THME has two natural instances.

- **Channels to formats.** Sources are layers (or channel groups) with supplies equal to their channel counts; sinks are the formats with capacities set by the hardware or budget; cost c_{l,s} = Ω_l q_s.
- **Tokens to zones.** Sources are tokens in a batch; sinks are Zones 1–3 (or experts) with capacity limits; cost is the expected loss increase of sending the token there. Balanced routing of this type, solved with optimal transport, is used to prevent router collapse in mixture-of-experts models.

The Northwest-corner method only builds a **starting feasible solution**: it fills the table from the top-left corner and ignores the costs entirely. In a toy 4×3 instance (supplies 100 each, capacities 150 / 150 / 100) its cost was 268.5 against an optimum of 181.0, 48% worse. It must be followed by an optimizing step (the MODI/stepping-stone method or network simplex), or replaced by one of:

- an LP or min-cost-flow solver, which is exact and fast at this scale, making the Northwest-corner start unnecessary;
- Vogel's approximation or the least-cost method, which give better starting points;
- **entropic optimal transport (Sinkhorn iterations)** for routing tokens, which is differentiable and GPU-friendly, so the router can be trained end to end.

Using a transportation formulation to enforce capacities (for example that exactly the intended share of tokens uses the deep path) is sound. Using the Northwest-corner rule as the decision rule is not.

F.4 Where TSP Might Fit

TSP asks for the shortest tour through a set of points; it is NP-hard, but heuristics (2-opt, Lin–Kernighan) give near-optimal tours at this scale. The fit to this architecture is weaker than for the transportation problem. Two narrow uses are plausible and neither is established:

1. **Row and channel ordering for hardware.** Ordering packed weight rows so that consecutive rows differ in few bits (a Hamming-distance tour) can reduce switching activity in hardware and improve compressibility. Placing channels of the same format contiguously is simpler and is an assignment or sorting problem, not a tour.
2. **Scheduling.** Ordering tokens in a batch by exit depth to keep the active set contiguous is a sorting problem.

TSP is recorded as low priority (V-43).

F.5 The Self-Tuning Loop and How It Gets Frozen

The proposal is a loop in which the system proposes configurations, evaluates them, and keeps improvements until the design stops changing. Such methods exist: neural architecture search, differentiable search (DARTS), reinforcement-learning mixed-precision search (HAQ), Hessian-aware ILP search (HAWQ), and LLM-guided evolutionary program search with automatic evaluators (AlphaEvolve, 2025). Applied to THME it is a bilevel problem:

- **Inner loop:** train or fine-tune weights for a given allocation and router thresholds.
- **Outer loop:** update the allocation (F1/F2), router thresholds, and K in the Consensus-Read Mixer by solving the programs above with fresh sensitivities.

Algorithm.

1. Measure channel sensitivities on a tuning split.
2. Solve the allocation (F2 then ILP); apply it.
3. Fine-tune for a fixed small budget of tokens.
4. Evaluate on a **selection** split; accept the update only if it improves there.
5. Repeat until the allocation change is below 10^{-3} (fraction of channels changed per round) and the selection-set gain is below a threshold for two consecutive rounds.

6. Freeze (“hardcode”): (a) fix channel-to-format indices; (b) distill the learned router into fixed thresholds or a small lookup table; (c) export packed weights and compile fixed kernels. After this step nothing is adaptive and the model is an ordinary static artifact.
7. Report the result once, on a **final test** split that no step above ever touched.

Guardrails. The evaluator must be external and fixed; a system that grades itself will overfit its own metric (Goodhart’s law). Use three disjoint splits (tune, select, final). Cap total search compute, and use cheap proxies (sensitivity scores, one-shot supernets) rather than training hundreds of full models. Keep a rollback checkpoint for every accepted round.

F.6 Convergence Toward 1: What the Mathematics Allows

A sequence of tuning rounds can be modeled as improving accuracy a_n toward a limit. The idealized picture in the proposal is $a_n = 1 - c\rho^n$ with $0 < \rho < 1$, so that $a_n \to 1$. Empirically the more common behavior is

$a_n = a_\infty - c\,\rho^n$, $a_\infty < 1$ in general (F3)

with a ceiling a_∞ set by factors the tuning loop does not control.

- **Irreducible uncertainty.** For next-token prediction the loss has a floor equal to the entropy of the data; scaling-law fits take the form $L = E + A/N^\alpha + B/D^\beta$ with $E > 0$ (Hoffmann et al., 2022). Next-token accuracy therefore cannot approach 1 however long the model is tuned.
- **Label noise and ambiguity** in benchmarks cap measurable accuracy below 100%.
- **Capacity and data limits** of the model class.

The limit can approach 1 only for tasks with a deterministic answer and a perfect verifier (arithmetic, code against complete tests, parity). There, repeated sampling with verification gives error ε^m only if the m attempts fail independently; correlated failures break the geometric decay, and an imperfect verifier adds its own error floor.

Certifying very high accuracy is expensive. If no errors are seen in n independent trials, the 95% upper bound on the error rate is about 3/n (the rule of three). Claiming error below 10^{-6} therefore needs about three million error-free test cases; below 10^{-9} needs three billion. The practical stopping rule is to fit Equation F3 to the measured rounds, estimate a_∞, and stop when the predicted remaining gain $c\rho^n$ falls below measurement noise.

F.7 Alternatives to Each Method

Goal	Proposed	Alternatives
Choose layer fractions	LP / ILP (F1)	Convex NLP with diminishing-returns distortion; dynamic programming over layers; knapsack; DARTS-style differentiable relaxation
Choose channel formats	Lagrangian / water-filling (F2)	Hessian-aware ILP (HAWQ-V3); reinforcement learning (HAQ); greedy by sensitivity-per-bit; learned bit-widths via gradient
Capacity-limited routing	Transportation / optimal transport	Top-k with auxiliary load-balancing loss; expert-choice routing; Hungarian assignment
Ordering	TSP heuristics	Sorting by format; bit-transition-aware layout from the hardware vendor's tools
Self-tuning	Bilevel loop with frozen result	Population-based training; evolutionary program search with an automatic evaluator; Bayesian optimization of a small hyperparameter set
Freezing	Distill router to table	Keep the router learned but quantize it; fixed thresholds from calibration

F.8 Tests and Register Additions

Table F2

Falsification Criteria for the Allocation and Self-Tuning Proposals

Hypothesis	Test	Falsified if
H-A1: Optimized channel allocation beats fixed schedules	Compare optimized, Table 6 schedule, and uniform allocation at equal average bits on a real model	No perplexity gain over the best fixed schedule
H-A2: Layer sensitivities are heterogeneous enough for per-layer allocation to matter	Measure the spread of Ω_l	Spread is small; uniform allocation within 1%

Hypothesis	Test	Falsified if
H-A3: Transportation routing reduces router collapse	Compare with top-k plus auxiliary loss	No change in routing entropy or quality
H-A4: Self-tuning converges	Track allocation change and selection-set gain per round	No convergence within the compute cap
H-A5: Frozen model matches tuned model	Compare final frozen model with the last adaptive round on the untouched test split	Loss of more than 1% relative

ID	Claim	Status	Note
V-39	LP gives the best mix of α, β, γ per layer	M	Linear objective gives at most one mixed layer; use channel-level allocation (F.1, F.2)
V-40	Optimal bit allocation has a closed form	S	Reverse water-filling (Equation F2), high-resolution approximation
V-41	Northwest-corner method solves the allocation	C	Starting solution only; 48% above optimum in toy case
V-42	Optimal transport / Sinkhorn can enforce routing capacity	S	Used for balanced routing; to be tested here (H-A3)
V-43	TSP improves the architecture	U	Narrow hardware-layout use only
V-44	Self-tuning loop converges and can be frozen	U	Requires external evaluator and three splits (F.5)
V-45	Accuracy approaches 1 as tuning continues	C	Ceiling $a_\infty < 1$ in general; achievable only for verifiable tasks (F.6)

Appendix G. Operations-Research and Differential-Equation Machinery Mapped onto Layers

This appendix takes the proposal that the formulas of linear, mixed-integer, and non-linear programming, together with differential equations, be assigned to layers that push matrices into one another, with the aim of better precision, speed, accuracy, memory, and energy. The mapping is real and productive in several places and counterproductive in

others. The decisive criterion is not mathematical sophistication but whether a mechanism lowers measured joules and bytes at equal quality, so that criterion is stated first.

G.1 The Complexity Tax

Every mechanism added to the forward pass has its own cost in operations, in bytes read, and in latency. A mechanism is worth keeping only if the cost it removes exceeds the cost it adds, with quality held at least equal:

Keep a mechanism ⇔ $\Delta E_{saved} > \Delta E_{control}$ and $\Delta Quality \geq 0$ (G1)

where E is energy per token measured on the target hardware. Because weight movement dominates (Section 5.3), the control cost of a component is mostly the bytes of its parameters and state. A worked example under the assumptions of Section 5.1: an attractor memory of M = 4,096 slots with d = 4,096 in FP16 occupies 33.6 MB; reading it K = 3 times per token from DRAM moves about 100.7 MB (8.05×10^8 bits), costing about 3.2–16 mJ at 4–20 pJ per bit. A ternary d × d layer moves 4.2 MB (3.36×10^7 bits), costing about 0.13–0.67 mJ. One such attractor layer therefore costs about 24 times a ternary layer, and this is before any solver or routing logic. Added sophistication can easily cost more than it saves, which is why this appendix separates **offline** use of optimization tools (free at inference time) from **online** use (paid on every token).

A complex formula is not itself a goal. Complexity is admitted one component at a time under Equation G1, and a component that fails the test is removed (a simplicity ratchet).

G.2 Layers as Matrices over Different Semirings

A layer that “acts as a matrix pushing into the next” is a matrix–vector product. Ordinary networks compute it over the real field. The same product can be defined over any semiring $(\oplus, \otimes)$:

$(W \otimes x)_i = \oplus_j (W_{ij} \otimes x_j)$ (G2)

Choosing the semiring is a way to attach different algebras, and therefore different hardware costs, to different layers. This is the standard framework of graph algorithms (GraphBLAS) and of dynamic programming, and it is a precise formal version of the heterogeneous design of THME (Table 4).

Table G1

Semirings as Layer Types

Layer type	(⊕, ⊗)	Operation per entry	Operation family
Standard	(+, ×) over the reals	Multiply-accumulate	Floating-point or integer MAC
Ternary	(+, ×) with weights in {−1, 0, 1}	Add, subtract, skip	Integer add
Binary	(XOR, AND) over GF(2), or sign-popcount	Bit operation and count	Logic and popcount
Phase	(+ mod 2^B) on phase indices	Integer add with wrap	Small integer add (Equation 15), plus a conversion to Cartesian form for accumulation
Tropical (max-plus)	(max, +)	Add, then compare	Add and compare
Min-plus	(min, +)	Add, then compare	Shortest-path recursion (Bellman, Viterbi)

Two observations. First, ReLU networks are exactly tropical rational functions (Zhang et al., 2018), so a max-plus layer is a natural, multiplier-free relative of existing networks. Second, a tropical layer has no arithmetic-energy advantage over a ternary layer, since both use adds and comparisons; its interest is expressivity (it computes dynamic-programming recursions in one step), not energy. Mixing semirings across layers requires explicit conversion maps and a nonlinearity (the max or a threshold supplies it), and quality of tropical layers at language-model scale has not been demonstrated (V-47).

G.3 Differential Equations in the Architecture

Several differential-equation ideas already map onto THME components.

- **State-space layers** are discretized linear ODEs, $\dot{x} = Ax + Bu$, with the exact zero-order-hold discretization used by S4 and Mamba and stability given by the real parts of the eigenvalues of A being negative (HiPPO initialization, Gu et al., 2020). They supply the token-mixing operator that is missing from the prototypes.
- **Closed-form continuous-time cells** (liquid time-constant networks and their closed-form variants, Hasani et al., 2022) approximate the ODE solution analytically and avoid a numerical solver, which suits edge hardware.

- **Adaptive step-size solvers as adaptive compute.** Explicit Runge–Kutta pairs such as Dormand–Prince give two estimates of different order, and their difference estimates local error cheaply:

h_new = h · min(f_max, max(f_min, 0.9 (tol / err)^(1/5))), err = ‖y^(5) − y^(4)‖ (G3)

Steps shrink where the dynamics change quickly and grow where they are smooth. This is a principled adaptive-depth mechanism: a token whose state evolves smoothly uses few steps. The signal measures numerical error of the dynamics, not difficulty of the prediction, so whether it correlates with token loss must be tested (hypothesis H-D1, Table G3).

- **Implicit and stiff solvers** require a linear solve per step ($O(d^3)$ for dense matrices), which is too costly for per-token use. Explicit schemes and diagonal or structured A are the practical choices.
- **Lyapunov functions** give provable stability for attractor dynamics, as in Section 4.7.

G.4 Optimization Solvers Inside the Forward Pass

There are three ways to place optimization algorithms into a network.

1. **Differentiable optimization layers** solve an LP or QP in the forward pass and differentiate through its solution (OptNet, Amos & Kolter, 2017; cvxpylayers, Agrawal et al., 2019). A generic dense QP costs $O(n^3)$, so this is affordable only for very small problems.
2. **Unrolled solvers.** Each layer is one iteration of an optimization algorithm with learned parameters. The classic example is learned ISTA (Gregor & LeCun, 2010), where layer l computes

z^(l+1) = soft_θl(W_e b + S z^(l)) (G4)

with learned matrices and thresholds. The same idea has been applied to ADMM and other methods under the name deep unfolding. Learned iterations typically reach a given accuracy in far fewer iterations than the fixed algorithm, so the number of layers needed falls. This is the most direct meaning of “assigning an operations-research formula to a layer.” 3. **Cheap matrix-scaling layers.** The Sinkhorn iteration (Cuturi, 2013) solves entropic optimal transport by alternately normalizing rows and columns, which costs $O(Knm)$ for K iterations and is differentiable. It enforces capacity constraints in token routing without a general-purpose solver.

General mixed-integer programming cannot be differentiated and takes unpredictable time, so it does not belong in the per-token path.

G.5 Operations Research Where It Pays Off: Offline and Systems Level

The largest, least risky gains from operations-research tools come from using them on the **deployment problem** rather than on the model mathematics. The model is unchanged, and the solver runs once.

Problem	OR formulation	Benefit
Bits per channel or layer	ILP / Lagrangian (Appendix F)	Lower distortion at fixed bits
Where each tensor lives (SRAM, DRAM, flash)	Integer program / knapsack with capacities	Fewer DRAM reads, which dominate energy
Splitting layers across devices	Dynamic programming or ILP (as in PipeDream and Alpa)	Higher throughput, balanced memory
Kernel tiling and scheduling	Search with a cost model (as in TVM autotuning)	Faster kernels with no change in model quality
Batching and queueing	Queueing theory (Little's law, L = λW)	Amortizes weight reads across requests
Routing capacity	Transportation problem / Sinkhorn	Prevents router collapse

G.6 A Unified Program

The overall design can be written as a single constrained optimization over the architecture a (discrete choices such as bit allocation, number of layers, semiring per layer, placement) and a runtime policy π (thresholds, number of readings K, solver tolerance):

min over (a, π): E[Energy(a, π)] subject to Quality(a, π) ⩾ q_0, Memory(a) ⩽ M_0, Latency(a, π) ⩽ T_0 (G5)

In Lagrangian form, J = Loss + λ_E Energy + λ_M Memory + λ_T Latency. The natural decomposition is a mixed-integer program for the discrete architecture a, solved offline, and a small online policy π consisting of a handful of thresholds. Reporting results as a **Pareto frontier** over (quality, energy, memory, latency) is more informative than a single combined figure, and it shows exactly which mechanisms move the frontier.

Table G2

Mechanism Map: Where Each Tool Belongs

Mechanism	Tool	Offline or online	Added per-token cost	Evidence status
Bit allocation per channel	ILP / water-filling	Offline	None	Strong theory, toy-tested
Memory placement and partitioning	ILP / DP	Offline	None	Strong in practice
Token routing with capacity	Sinkhorn / transport	Online	O(Knm), small	Moderate
Token mixing	Discretized linear ODE (SSM)	Online	O(1) state	Strong
Adaptive compute from solver error	Embedded Runge–Kutta	Online	One extra stage	Unverified
Iterative refinement	Unrolled solver (LISTA-style)	Online	K layers	Moderate in sparse recovery; unverified for language
Constrained layer via QP	OptNet / cvxpylayers	Online	$O(n^3)$	Poor fit for per-token use
Max-plus layers	Tropical semiring	Online	Add and compare	Unverified at scale
Mixed-integer programming	MIP solver	Offline only	None at inference	Not differentiable

G.7 Tests and Register Additions

Table G3

Hypotheses and Falsification Criteria for Appendix G

Hypothesis	Test	Falsified if
H-D1: Solver error estimate predicts hard tokens	AUROC of err (Equation G3) for top-decile token loss	AUROC below about 0.6

Hypothesis	Test	Falsified if
H-D2: Unrolled solver layers reduce depth	Compare K unrolled layers with a fixed-iteration solver at equal accuracy	No reduction in iterations or FLOPs
H-D3: Max-plus layers match ternary layers	Same small language model with a fraction of layers replaced	Worse perplexity at equal parameters
H-D4: Offline placement lowers energy	Measure joules per token with and without ILP-chosen placement	No change within measurement noise
H-D5: Every added mechanism passes Equation G1	Measured energy at equal quality	Any mechanism whose control cost exceeds its saving

ID	Claim	Status	Note
V-46	More complex formulas improve precision, speed, and energy	C	Only mechanisms passing Equation G1 do; complexity has a cost
V-47	Tropical (max-plus) layers are multiplier-free and useful	U	No energy advantage over ternary; quality at scale untested
V-48	Adaptive solver step size gives adaptive depth	U	H-D1
V-49	Unrolled optimization layers reduce depth	S for sparse recovery (LISTA); U for language	H-D2
V-50	MIP/LP/QP solvers inside the forward pass	C for MIP; U for LP/QP	Not differentiable or $O(n^3)$; use offline
V-51	OR tools applied to deployment (placement, partitioning, tiling) lower cost	S	Established practice; H-D4 to measure
V-52	Attractor layer with large FP16 memory is cheap	C	About 24 times a ternary layer under stated assumptions

Note on sources. Method names and years in this appendix (OptNet, cvxpylayers, learned ISTA, tropical geometry of ReLU networks, HiPPO, closed-form continuous-time cells, Sinkhorn, PipeDream, Alpa, TVM) are given from memory; verify and add them to the reference list before submission.

Appendix H. Study Design: Visualization, Hypotheses, Challenger–Defender Review, and the Trial-and-Error Program

This appendix turns the proposal into a program that can show whether THME works, or fail cheaply and say why. It has five parts: the master hypothesis register (H.1), a stage-gated study (H.2), a challenger–defender protocol (H.3), visualization techniques (H.4), and rules that keep trial and error from producing false breakthroughs (H.5–H.6).

H.1 Master Hypothesis Register

Every hypothesis stated in the paper, with its stage, primary metric, and pass rule. Thresholds are proposals, to be fixed in writing before any run (H.5). Priority P1 means the study stops or pivots if it fails.

Table H1

Master Hypothesis Register

ID	Hypothesis	Stage	Primary metric	Pass if	Priority
H0	A minimal THME (hybrid mixer, ternary or 4-bit weights, early-exit speculation) beats a strong baseline on the quality–energy–memory frontier	1–3	Pareto position	Not dominated by baseline, with CI	P1
H-T	Ternary QAT retains quality relative to 4-bit PTQ at lower bytes	1	Perplexity at equal bytes	Within 2% of 4-bit PTQ	P1
H-M	A hybrid recurrent/attention mixer matches attention on recall	1	MQAR and needle accuracy	Within 2 points of attention	P1
H-X	Early-exit drafts reach high acceptance under speculative decoding	1–3	Acceptance rate, tokens/s	Acceptance ≥ 0.8 and measured speedup ≥ 1.3×	P1
H-C1–C4	Consensus-Read Mixer improves recall, noise, routing, repair (Table E4)	1	Recall, MSE, reads/token	See Table E4	P2
H-E	Head disagreement predicts hard tokens	1	AUROC	≥ 0.6	P2

ID	Hypothesis	Stage	Primary metric	Pass if	Priority
H-A1–A5	Optimized allocation beats fixed schedules; self-tuning converges and freezes	1–2	Perplexity at equal bits	Gain with CI > 0	P2
H-D1–D5	Solver-error routing, unrolled layers, max-plus layers, offline placement, Equation G1	1–3	See Table G3	See Table G3	P3
H-G	Clifford features help	1	Perplexity; N-body error	See Table 11	P3
H-S	Stabilizer projection reduces quantization error	0–1	$\|\Psi_q - \Psi_{FP32}\|$	Below RMSNorm-only	P3
H-K	Discrete-torus register helps	1	Perplexity	Gain over extra plain channels	P3
H-I	Polar/absmean initialization preserves teacher function	2	Fraction of teacher score	⩾ 90%	P2
H-R	Resource model (Equation 29) predicts measured energy	3	Joules per token	Within a factor of 3	P1

H.2 Stage-Gated Study

Work proceeds through gates. A stage begins only when the previous gate passes; a failed gate sends the work back to diagnose, revise, or drop a component (Figure H1).

Stage 0: foundations. Run the Tier 0 mathematical unit tests (Section 7) and reproduce two published baselines (a Transformer and Mamba) to within 2% of reference numbers. If the baselines do not reproduce, every later comparison is meaningless. **Gate G0:** all identity tests pass; baselines reproduce.

Stage 1: small-scale matched ablations. Models of about 50M parameters on about 1B tokens, with parameters and training FLOPs matched across variants. Cost: $6 \times 5{\times}10^{7} \times 10^{9} = 3{\times}10^{17}$ FLOPs per run, about 8 hours at an assumed 10 TFLOP/s sustained, so 8 variants × 3 seeds is about 8 days on one GPU. Add components in this order: ternary weights, hybrid mixer, exit gate with speculation, then Consensus-Read Mixer, allocation optimizer, phase subspace, and the exotic components last. **Gate G1:** at least one component produces a Pareto improvement with a 95% confidence interval excluding zero; otherwise stop adding and diagnose.

Stage 2: conversion pilot. One 1B-class teacher, 0.5–1B distillation tokens, MLP-only first, then attention replacement (Section 6, Appendix D.9). **Gate G2:** at least 90% of teacher benchmark average and perplexity within 10%.

Stage 3: systems measurement. Joules per token and tokens per second at stated batch size on CPU and GPU, against 4-bit PTQ with and without speculative decoding. **Gate G3:** measured energy within a factor of 3 of Equation 29 and a gain over the strongest baseline.

Stage 4: scale and replication. Train two sizes from scratch to check that the gain persists along a scaling-law fit, and invite an independent replication using the released harness.

Table H2

Evidence Levels and Breakthrough Criteria (Proposed)

Level	Meaning	Requirement
0	Valid	Mathematical tests pass; no known errors
1	Promising	Beats a matched baseline at about 50M parameters with CI excluding zero
2	Credible	Replicates over at least 5 seeds and 2 scales; measured energy at least 2× lower at equal quality than the strongest baseline
3	Breakthrough	Holds at 1B or larger on standard benchmarks, with independent replication

H.3 Challenger–Defender Protocol

Each component is argued as a contest with fixed rules, so that the decision comes from measurements rather than from persuasion.

1. **Registration.** The Defender states the claim, the predicted effect size, the metric, and the pass threshold in writing before any run.
2. **Challenge.** The Challenger chooses the strongest baseline (never a weak one), the ablations, and the adversarial tests, and lists confounds the Defender must rule out. The Challenger sees the design, not the results.
3. **Matched budget.** Parameters, training FLOPs, tokens, and tuning effort are equal for both sides. If one side received more hyperparameter tuning, the comparison is void.
4. **Decision by rule.** The pass rule in Table H1 decides; argument does not.
5. **Record.** Every outcome, including failures, goes into the negative-results log (H.6).
6. **Roles.** Roles can be played by different people or by separate AI sessions. AI-generated critiques are useful for finding attacks, but they are not evidence; only code and measurements decide. Reuse of one AI model as both Challenger and judge risks shared blind spots.

Standard attacks the Challenger must try. Hyperparameters tuned for one side only; data leakage or benchmark contamination; seed variance larger than the effect; an under-tuned baseline; metrics chosen after seeing results; cherry-picked context lengths; hardware artifacts (warm-up, thermal throttling, clock changes, caching); gains that vanish at equal bytes or equal FLOPs; and savings that come from the quantization step rather than the proposed mechanism.

Table H3

Challenger–Defender Sheet per Component

Component	Defender's claim	Challenger's strongest baseline	Decisive experiment	Pass rule
Ternary weights	8× smaller weights at near-equal quality	4-bit PTQ (GPTQ/AWQ)	Equal bytes, same data	Within 2% of 4-bit
Hybrid mixer	Bounded state with adequate recall	Full attention with GQA and KV quantization	MQAR, needle, perplexity	Within 2 points of attention
Exit gate	Easy tokens exit early without loss	Plain speculative decoding with a small draft model	Acceptance rate and tokens/s	Speedup ≥ 1.3× and lossless output
Consensus-Read Mixer	Better recall at fewer reads	Gated delta layer without readings	MQAR; reads per token	Gain with CI > 0 at equal FLOPs
Allocation optimizer	Lower distortion at equal bits	Hand-set 80/15/5 schedule and uniform bits	Perplexity at equal bits	Gain with CI > 0
Clifford features	Help on geometric structure	Orthogonal dense layer at equal parameters	N-body, then language	Gain with CI > 0
Stabilizer projection	Lower quantization error	RMSNorm alone	$\|\Psi_q - \Psi_{FP32}\|$	Lower error
Phase subspace	Phase-quantized recurrence keeps recall	Real-valued recurrence	MQAR at 4, 6, 8 bits	Within 2 points

Component	Defender's claim	Challenger's strongest baseline	Decisive experiment	Pass rule
Attractor block	Multi-step inference gain	K extra ordinary layers at equal FLOPs; chain-of-thought	Multi-hop tasks	Gain at equal FLOPs
Max-plus layers	Match ternary layers	Ternary layers	Replace a fraction of layers	Equal perplexity

H.4 Visualization Techniques

Plots serve decisions, so each question in the study is paired with the plot that answers it. All plots show baselines on the same axes, uncertainty (seed bands or confidence intervals), and the sample size.

Table H4

Question-to-Plot Map

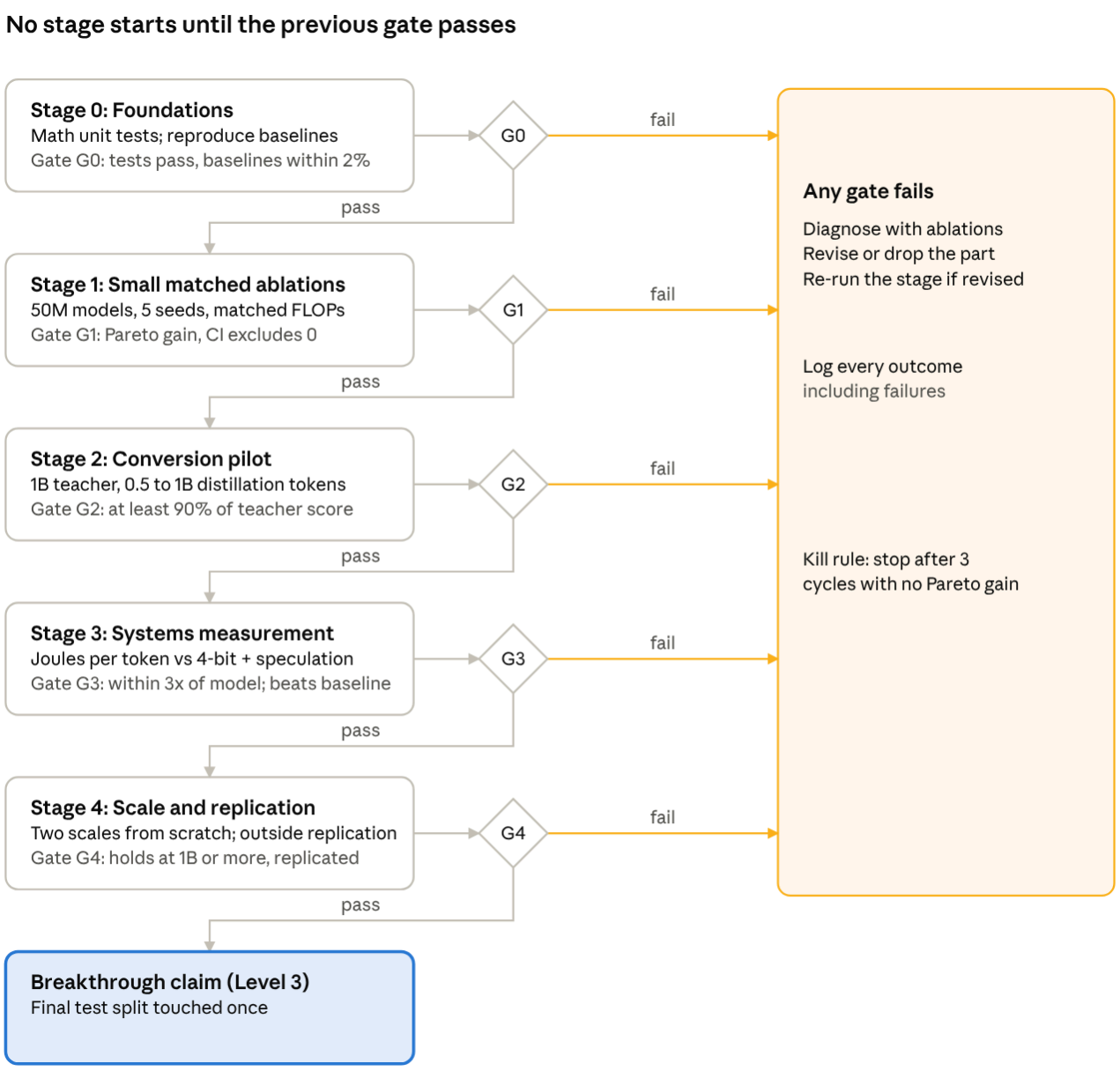
Question	Plot	What to read
Does it beat the baseline overall?	Pareto scatter: energy per token (log axis) against perplexity; frontier line; baseline highlighted	Variants on or beyond the frontier
Which component helped?	Ablation chart: change in quality per added component with 95% CI bars	Bars whose interval excludes zero
Is training healthy?	Loss against tokens (log–log) with seed bands and scaling-law fit	Divergence, plateaus, extrapolation
Does the gate route sensibly?	Heatmap of tokens by exit layer; histogram of exit depth; reliability (calibration) plot; ROC curve	Collapse to one extreme; AUROC
Where does energy go?	Stacked bars: arithmetic, SRAM, DRAM, control	Share of memory movement
Is it memory-bound?	Roofline plot: arithmetic intensity against attainable throughput	Distance below the bandwidth roof

Question	Plot	What to read
How does memory grow with context?	Line chart of bytes against context length for KV cache and state	Crossover points
Is read noise heavy-tailed?	Histogram and Q–Q plot of reading errors; kurtosis	Departure from a straight line
Does averaging stop helping?	Error against K with the shared-error floor drawn	Where the curve flattens
What did the allocator choose?	Layer-by-channel heatmap of formats; sorted sensitivity spectrum with the water level	Which channels receive more bits
Does the attractor converge?	Energy E(z_k) against iteration; phase portrait on a 2-D toy	Monotone decrease; basins
Can it recall?	Accuracy heatmap over context length and number of key–value pairs	Where recall fails
Where are the outliers?	Per-channel activation and weight distributions	Outlier channels that hurt quantization
What is the study status?	Gate dashboard: one row per hypothesis, pass, fail, or pending	Next action

Tools. Matplotlib or Plotly for figures; TensorBoard or Weights & Biases for training curves; Netron for architecture graphs; the PyTorch profiler and NVIDIA Nsight for kernel traces. Energy: NVML counters for NVIDIA GPUs, RAPL on supported x86 CPUs, powermetrics on Apple silicon, and an external wall-power meter as ground truth.

Conventions. Log axes for quantities spanning orders of magnitude; no truncated axes on bar charts; colour-blind-safe palettes; one baseline colour used everywhere; raw data (CSV) and plotting scripts stored with every figure.

Figure H1. Stage-gated study flow, drawn below.



stage-gated study · 5 stages, 5 gates, 1 failure loop

A failed gate sends the work to diagnosis, revision, or removal of the component; only a passed G4 supports a breakthrough claim.

H.5 Rules for Trial and Error That Does Not Fool Itself

Trial and error is a legitimate way to search, provided the search cannot manufacture its own evidence. Four rules apply.

1. **Count the comparisons.** If m variants are each tested at significance level α with no correction, the chance of at least one spurious win is

$$P(\geq 1 \text{ false positive}) = 1 - (1 - \alpha)^m \quad (H1)$$

For α = 0.05 and m = 20, this is 64%. Use Holm or Bonferroni correction across the register, and treat any single win among many as a lead to be replicated, not a result. 2. **Three splits.** Tune on one split, select on a second, and touch the final test split exactly once at the end (Appendix F.5). 3. **Seeds.** Use at least 5 seeds for any claimed gain: with n = 3 the 95% t-multiplier is 4.30, and with n = 5 it is 2.78, so intervals tighten by about a third. Report mean and 95% CI. 4. **Pre-registration.** Write hypotheses, thresholds, and the analysis plan before running; changes after seeing data are labeled exploratory.

H.6 The Iteration Loop and Stopping Rules

The program cycles through: **hypothesize** (one component, one claim), **predict** (effect size and threshold), **test** (matched baseline, at least 5 seeds), **adjudicate** (pass rule only), **log** (including failures), and **update** (keep, revise, or drop). Components are added one at a time, and a component that fails two consecutive gates is dropped.

Most novel architectures do not beat strong baselines at scale, so the program is planned around partial outcomes as well as a breakthrough:

- **Kill rule.** If, after three full cycles, no component has produced a Pareto improvement over the strongest baseline at Level 1 or higher, stop and report.
- **Salvage outcomes.** A corrected mathematical specification, a reproducible harness, a negative-results log, and any engineering gain that stands alone (for example ternary weights with early-exit speculation) are valid results even if the full THME fails.
- **Negative-results log.** A table of every attempted variant, the pass rule, the outcome, and the reason, kept in the repository.

H.7 Register Additions

ID	Claim	Status	Note
V-53	Trial and error can reach a breakthrough without a formal protocol	C	Multiple comparisons inflate false positives (Equation H1)
V-54	Component gains are real if they appear in one run	C	Needs at least 5 seeds and matched budgets
V-55	AI-generated critique counts as validation	C	Useful for ideas; measurements decide
V-56	A stage-gated program with kill rules can establish or refute THME	S	Standard experimental practice

Appendix I. Synthesis of Nine Source Papers into New THME Techniques

This appendix reads nine documents supplied for the project and re-derives their useful ideas as THME techniques instead of copying them. Each technique states its source, the reframing, an equation where one exists, a test, and a risk. Nothing here has been trained or measured; every technique is a hypothesis with a falsification test in I.7. Reading depth differs by source and is stated in Table I1, so that the weight given to each claim is visible.

I.1 The Sources and How Much Weight They Carry

Table I1

Sources, Facts Used, Reading Depth, and Evidence Weight

Source	Facts used	Reading depth	Evidence weight
Hopfield (1999), Brain, neural networks, and computation, Rev. Mod. Phys. 71(2)	Computation as flow to a fixed point; integrate-and-fire neuron, C du/dt = −(u − u_rest)/R + i(t), firing period P = C(u_thresh − u_rest)/i_c; Hebbian synapse dS/dt = α i f − decay with a bounding nonlinearity; symmetric coupling gives a Lyapunov energy; feed-forward cost of one multiply and one add per connection plus one look-up per neuron; connection strengths of trained networks carry little meaning, fit quality does	Full	High for theory; no benchmarks

Source	Facts used	Reading depth	Evidence weight
Pipkin (2020), Mapping the mind of a fly, eLife Insight on Scheffer et al.	Half of a fly brain: over 25,000 neurons, 20 million synapses; over 20 TB of images reduced to a 26 MB network diagram; three quarters of neurons linked by three or fewer interneurons; neuropils electrically segregated; map incomplete (missed twigs, other half unmapped)	Full	Medium; secondary commentary, structure not function
Polché et al. (2026), Hybrid quantum-classical forecasting, arXiv:2605.24252	Repeated-measurement quantum reservoir with kernel readout; projected quantum kernel Gaussian process; hardware runs on a 156-qubit device; baselines and error figures	Full	Medium; small test windows, authors call it a feasibility study
Zhang et al. (2014), Relative Accuracy Evaluation, PLoS ONE 9(8)	Average relative error ARE = 1 − mean relative error; truth discovery without ground truth by entity resolution and EM; accuracy of a query result versus of the whole data set; propagation formulas through selection, union, difference; incremental update formulas	Full	Medium; synthetic TPC-H data with injected errors, estimates within 5–15%

Source	Facts used	Reading depth	Evidence weight
Kandala et al. (2025), TinyLLM, framework for training and deploying language models at the edge	30–124M GPT-2-style models; parameter formula $N = 0.05\,l^3 + 3.2\,l$; 124M model on about 10B tokens in roughly 9 hours on one H100; sensor/web data mix near 40:60 best; LoRA fine-tuning with 440 examples; accuracy rises with size and the 30M model scored 0% on gestures	Full	Medium; datasets of 350–3,730 samples; baselines are heavily quantized larger models
Fishman, Sun et al. (2026), Dory1.0-4B, arXiv:2611.03998	Looped normalized-GPT; 16/40/16-layer input/recurrent/output split; effective depth 32 + 40k; average over 12 benchmarks 35.2 (k=1), 39.7 (k=2), 40.6 (k=4); last-loop KV caching; self-speculative decoding with k=1 draft and k=4 verification	Main text through inference optimization; post-training tables skimmed	High; large-scale, open checkpoints; vendor evaluations
Hosen et al. (2026), Acta Innovations 63(2s), predictive analytics	Comparison of gradient-boosted trees, LSTM, Transformer, with SHAP explanations	Abstract, methods, results section	Low; results given as qualitative ratings (High, Very High, Highest), no numerical table seen
Thomas (2018), Mathematics for Machine Learning (47 pages)	Orthogonal projections, SVD, operator norms, Eckart–Young–Mirsky low-rank approximation, convexity conditions	Contents and selected theorems	High for standard mathematics

Source	Facts used	Reading depth	Evidence weight
Azure Quantum documentation (729 pages)	Resource estimator (inputs, Pareto-optimal qubits versus runtime), hybrid quantum computing sessions, quantum error correction models (error suppression with code distance below a threshold, qubit overhead scaling with distance squared)	Keyword search of overview, hybrid, estimator, and error-correction sections; not read in full	Product documentation, not evidence on machine learning

I.2 What the Sources Change in the Earlier Appendices

1. **Measured training throughput.** TinyLLM reports about 9 hours for a 124M model on about 10B tokens on one H100. With $6 \cdot N \cdot D$ FLOPs this is 7.4×10^{18} FLOPs in 32,400 s, an effective 2.3×10^{14} FLOP/s. The earlier 10 TFLOP/s assumption applies to a consumer GPU. On an H100-class device Stage 1 (50M parameters, 1B tokens, 3×10^{17} FLOPs) takes about 22 minutes per run, so 24 runs take about 9 hours. Stage 2 (1.24B parameters, 10B tokens, 7.4×10^{19} FLOPs) takes about 3.75 days, extrapolating the same utilization and ignoring quantization-aware overhead. The claim V-26 (days on one consumer GPU) stays contradicted; days on one datacenter GPU is plausible.
2. **Speculative-decoding model validated.** Equation D1 with Dory's settings (draft cost ratio $c = (32 + 40)/(32 + 160) = 0.375$, $\gamma = 4$) gives speedups of 1.37 at acceptance $a = 0.81$ and 1.70 at $a = 0.92$. Dory measured 1.37–1.71× throughput, so the model is consistent with acceptance rates of roughly 0.8–0.9. Dory additionally reuses KV caches and activations, which the model omits.
3. **The output head limits early exit at small scale.** For a 124M GPT-2-style model ($V = 50{,}257$, $C = 768$, 12 blocks) the output projection costs 38.6M MACs against 84.9M for all blocks, 31% of per-token compute. With $p_{exit} = 0.8$ at $\lambda_{exit} = 0.3$, the body-only depth fraction 0.44 becomes $(0.44 \times 84.9 + 38.6)/123.5 = 0.615$ once the head is counted. Early exit saves about 38% of compute at this scale, not 56% (V-17 refined; technique T9).
4. **Normalization is the working stabilizer in a looped model.** Dory L2-normalizes the state at both loop boundaries. The control condition for hypothesis H-S (Table 11) should therefore be hypersphere normalization as well as RMSNorm.
5. **Looping saturates.** Dory's base-model results change little beyond four loops (for example MMLU 71.19 at k=4 and 71.17 at k=8), so $K_{max} \approx 4$ is a sensible default.

I.3 Techniques Re-Derived from the Sources

T1. The Looped Abyss (elastic-depth shared block). *Sources: Dory1.0-4B; Hopfield (1999).* Replace the separately parameterized Zones 2–3 with one shared recurrent block applied k times, with the block's input re-injected on every pass:

$$s_0 = x,\ u_t = N(s_{t-1} + x),\ s_t = N(F_\theta(u_t)),\ t = 1, \ldots, k \quad (I1)$$

where N is per-token L2 normalization, so the state stays on a hypersphere. This is the right form for the attractor of Section 4.7. The re-injected x plays the role of the drive term b(x) in Equation 23, normalization supplies the confinement that the quartic term was meant to give, and weight sharing means the block's weights are read once per token if they stay resident in fast memory, so looping raises arithmetic intensity, which helps on bandwidth-bound hardware. Executed layers are $L_{in} + L_{rec} \cdot k + L_{out}$ (72 at k = 1 and 192 at k = 4 for Dory's 16/40/16 split, a factor 2.67). Last-loop KV caching holds the KV cache at the one-loop size for any k; Dory measured score changes of −2.1 to +0.4 points at k = 4. THME adds three things: the block F_θ is ternary; k is chosen per token (T2); and the normalized recurrence gives a convergence signal $\delta_t = \|s_t - s_{t-1}\|$ that serves as a Lyapunov proxy (hypothesis H-N1: δ_t decays geometrically). If the block's Jacobian has operator norm below one, the Banach fixed-point theorem guarantees convergence, and the norm can be estimated by power iteration (Rayleigh quotient, Thomas, 2018). *Risk:* Dory trained at 6T tokens; a ternary looped block at small scale is untested.

T2. Integrate-to-Threshold Halting. *Source: Hopfield's integrate-and-fire neuron.* A leaky neuron obeys $\tau\, du/dt = -u + i$ and fires when u reaches a threshold θ. For constant input $i > \theta$ the time to threshold is

$$T = \tau \ln(i / (i - \theta)),\ \text{no firing if } i \leqslant \theta \quad (I2)$$

so T/τ is 2.40 at $i = 1.1\theta$, 0.69 at $i = 2\theta$, 0.29 at $i = 4\theta$, and 0.105 at $i = 10\theta$: strong evidence fires fast, weak evidence slowly, insufficient evidence never. Reframing: at each loop j of T1, add the logit margin (top minus second) to a leaky evidence accumulator; the token is emitted when the accumulator crosses θ, and tokens that have not crossed by K_{max} escalate (deeper path or abstention). This replaces the learned sigmoid gate (Equation 20) and the dispersion signal (Appendix E) by a sequential test whose cost is automatically proportional to difficulty. Dory routes by a training-time difficulty curriculum, not by per-token halting at inference, so T2 is a genuine addition. *Risk:* a logit margin is not a calibrated probability; θ must be set on the selection split (Appendix F.5) and the result reported as a risk–coverage curve (T5).

T3. Hebbian–Delta Memory. *Source: Hopfield's Equation 8.* Hopfield writes a synapse update as $dS/dt = \alpha\, i\, f -$ decay, and stresses that decay is needed to forget and a nonlinearity is needed to stop unbounded growth. The gated delta rule of Equation E1 expands to

$$M_t = \alpha_t M_{t-1} + \beta_t (v_t - \alpha_t M_{t-1} k_t) k_t^T \quad (I3)$$

that is, decay (α_t), a Hebbian outer-product write ($\beta_t v_t k_t^T$), and an error-correcting term that writes only what the memory does not already predict. The memory in the Consensus-Read Mixer is therefore Hopfield's synapse rule plus a delta correction. This gives it a physical reading and a known capacity behavior (Hebbian memories store a number of random patterns proportional to the number of units; the delta correction raises the usable capacity). The test is the recall task of Table E4 with state size varied.

T4. Reliability-Weighted Consensus. *Sources: Zhang et al.; Polché et al.; Azure Quantum error-correction models.* Zhang et al. estimate true values without ground truth using the mean (which minimizes MSE = Var + bias² for unbiased estimates) and, for categorical values, an EM algorithm with an error-transition matrix. For THME the K readings of a ternary or binary quantity are treated as K sources of unknown reliability. EM estimates each reading's transition matrix and the posterior of the true value, replacing the hard "nearest half" split of CRM v0 with learned weights. Two limits frame what consensus can achieve. For K independent readings each wrong with probability $p < ½$, the majority error is bounded by

$$P(\text{error}) \leqslant \exp(-2K (½ - p)^2) \quad (I4)$$

and the exact values (ties counted half) are in Table I2. Above $p = ½$ no number of readings helps. This is the classical threshold behavior that the Azure error-correction models express for quantum codes: below threshold, more redundancy suppresses errors exponentially at a cost that grows with the redundancy; above threshold it does not. Correlated errors destroy the effect, the same as the shared-state floor of Table E2. The shot noise of the hybrid quantum paper follows the same law as Table E3: estimating a probability from S shots has standard deviation $\sqrt{(p(1 - p)/S)}$, so the quantum "readings" do not escape the $1/\sqrt{K}$ behavior. One caution on Zhang et al.'s metric: relative error $|\hat{h} - h|/|h|$ is undefined at $h = 0$ and explodes near zero, which is common for activations, so a scaled form $|\hat{h} - h|/(|h| + \varepsilon)$ is required.

Table I2

Majority-Vote Error for K Independent Readings with Error Probability p (Ties Counted Half)

p	K = 1	K = 3	K = 5	K = 8	K = 16
0.1	0.100	0.028	0.0086	0.0027	< 0.0001
0.2	0.200	0.104	0.058	0.033	0.0042
0.3	0.300	0.216	0.163	0.126	0.050
0.4	0.400	0.352	0.317	0.290	0.213
0.5	0.500	0.500	0.500	0.500	0.500

T5. Risk–Coverage Relative Accuracy. *Sources: Zhang et al.; Appendix F.6.* Zhang et al. distinguish the accuracy of a whole data set from the "relative accuracy" of a query result and report the quadruple (precision, recall, F-measure, ARE). For THME the analogous object is the accuracy of the tokens emitted on a route. Report (coverage c, selective accuracy A(c), F, ARE) for every route. "Accuracy approaching 1" then has a precise, achievable meaning: A(c) → 1 as c decreases, by escalating or abstaining on uncertain tokens, which is the reject-option trade-off of selective classification (Chow, 1970). The risk–coverage curve and its area replace a single accuracy number. Certification cost follows the rule of three: to show a selective error below 10^{-6} at coverage c needs about $3 \times 10^{6} / c$ test items. Zhang et al. also give independence-based propagation (selection $P_X P_Y$; union $P_A + P_B - P_A P_B$; difference $P_R(1 - P_S)$) and incremental updates $ARE(T') = ARE(T) + |E_i|(ARE(E_i') - ARE(E_i))/|D|$. Both transfer: the first as an offline accuracy budget for a chain of components (an upper bound, since errors correlate), the second as a drift monitor for a frozen model in service.

T6. Small-World Sparse Scaffold. *Sources: Pipkin (2020) on Scheffer et al.; Hopfield (1999) on development.* Half a fly brain has over 25,000 neurons and 20 million synapses, about 800 synapses per neuron, so at most 3.2% of all possible neuron pairs are connected (an upper bound, since several synapses can join the same pair), and three quarters of neurons lie within three interneurons of each other. Reframing: short path length does not need dense layers. In an ideal random expander, h layers of fan-in d reach all D units when $d^h \geq D$, so

$$d \geq D^{(1/h)} \quad (I5)$$

For D = 4,096 this is d = 64 at h = 2, d = 16 at h = 3, and d = 8 at h = 4, a density of 0.39% at three hops. A sparse ternary layer with 16 non-zeros per row needs 4,096 × 16 × 13 bits (12-bit index plus sign) = 106 KB against 4.19 MB for a dense ternary layer, 39 times smaller, and no index storage at all if the pattern is a fixed hard-wired structure. Hopfield contrasts a nearly fixed wiring (about 90% identical connections between *C. elegans* animals) with mammalian wiring that is partly random and refined by activity. The THME analogue is a hard-wired sparse scaffold plus plastic fast-weight memory (T3). The segregation of neuropils, which lets one neuron compute different things in different regions, suggests a second reframing: a shared block (T1) with a small loop-conditioned

gain g_t applied to its ternary weights, so the same weights act differently on pass t (Dory shares all weights and scales across loops; this is an alternative to test, H-N6). *Cautions:* a connectome is a map of structure, not of function; the map is incomplete (Pipkin notes missed twigs and the unmapped other half); and fly sparsity says nothing about the quality of sparse random layers in language models, which must be measured.

T7. Reservoir–Readout Mixer and Classical Pre-Screening. *Source: Polché et al. (2026).* Their quantum reservoir trains only the readout, by closed-form kernel ridge regression $\beta = (K + \lambda I)^{-1} y$, on features stacked over a time window. The classical counterpart is an echo state network: a fixed sparse ternary recurrent matrix W_res with spectral radius below one drives $h_t = \tanh(W_{res} h_{t-1} + W_{in} x_t)$, and only a ridge readout is trained, with no backpropagation through the mixer. Its memory is limited by reservoir size, so it suits smooth sensor streams (the TinyLLM use case) and cheap initialization of the delta memory, not exact long-range recall. The paper also uses two diagnostics from Huang et al. that apply to any feature map, quantum or not: the geometric difference between a classical kernel K_C and a candidate kernel K_Q,

$$g(K_C \,\|\, K_Q) = \sqrt{\left\| \sqrt{K_Q}\, K_C^{-1} \sqrt{K_Q} \right\|_\infty}, \quad \kappa(K) = y^T K^{-1} y \quad (I6)$$

A feature map is promising only if g grows like $\sqrt{N}$ while $\kappa_Q \ll N$ and $\kappa_C \approx N$. Both cost $O(N^3)$ on a few hundred to a few thousand samples, seconds on a laptop, so they are a cheap Stage 0.5 pre-screen for CRM states and Clifford features (H-N10). Reading the paper as a challenger raises four points. The diagnostics were computed for a fidelity-based kernel, but the models that were tested use an RBF kernel on measured probabilities and a projected kernel, so the pre-screen did not test the models run. The classical Gaussian-process baseline scored an MAE of 0.41, worse than the naive persistence forecast (0.28), against 0.16 for the simulated quantum model and 0.24 on hardware, so the headline improvements of 62% and 40% are measured against a baseline that does nothing useful. The KQRC-RM hardware MAE of 0.34 barely beats persistence (0.36) and is worse than the classical ESN-KRR (0.16). And the training windows were 15 hours with 5-hour tests over five realizations, so the intervals are wide. The authors themselves conclude that the results are a feasibility study, not evidence of advantage. They also ran 100- and 114-qubit circuits on a matrix-product-state simulator, which indicates limited entanglement and makes a classical tensor-network or low-rank model the natural comparison. The lesson for Table H3 is to add a rule: the naive persistence baseline must be beaten, and the classical baseline must be tuned.

T8. Hard-Coded Boosted-Tree Router with Self-Labels. *Sources: Hosen et al. (2026); Appendix F.5.* Routing is a tabular prediction problem. Features: exit-head maximum probability, entropy, logit margin, the loop change $\delta_1 = \|s_1 - s_0\|$, position, previous routing decisions, and a token-frequency bucket. The label is free: y = 1 if the argmax at k = 4 differs from the argmax at k = 1, or if the loss at k = 4 is lower by more than ε. A gradient-boosted tree ensemble (XGBoost or LightGBM) trained offline can be exported

as threshold comparisons with no multiplications, which is exactly the “hard-code the router” step of F.5. SHAP values give an audit of which features drive routing, and the same attribution method can estimate channel sensitivities for the allocator of Appendix F (exact Shapley values are exponential, so sampling is required). The evidence in the Hosen et al. paper for tree ensembles is a qualitative table, so it carries little weight; the real test is AUROC of the router against logistic regression on the same features (H-N5).

T9. Output-Head-Aware Early Exit. *Source: TinyLLM.* TinyLLM’s formula $N = 0.05\, l^3 + 3.2\, l$ with $C = 64\, l$ reproduces its Table 1 (30.0, 51.2, 82.0, 101.8, 124.8 million parameters at $l = 6, 8, 10, 11, 12$), and it shows why small models differ: the embedding and output head ($V \times C$ = 38.6M parameters at $l = 12$) are a third of the per-token multiply-accumulates. Remedies that preserve the early-exit saving: a ternary or 4-bit head; a **vocabulary shortlist**, where the exit path scores only V' candidate tokens proposed by a cheap conditional frequency table and falls back to the full softmax when the shortlist’s probability mass is low; and adaptive-softmax-style frequency clustering. With $V' = 4{,}096$ the head costs 3.1M MACs instead of 38.6M, and the depth fraction drops from 0.615 to about 0.33 (H-N8). At this scale the packed ternary body of a 124M model is only about 21 MB, while a 16-bit embedding table is about 77 MB, but an embedding lookup reads one row per token, so storage and bandwidth are different questions.

T10. A Resource Estimator for THME, and the Error Budget as an Input. *Source: Azure Quantum documentation.* The Azure resource estimator takes an error budget, an application model, a hardware model, and a code model, and returns Pareto-optimal combinations of physical qubits and runtime. The same pattern fits THME: a small program that takes the architecture (layers, bits, loops k, exit fraction), a hardware profile (bandwidth, energy per bit and per operation), and a **selective-error budget ε**, and returns the Pareto frontier over bytes per token, joules per token, and tokens per second, with every assumption printed. The error budget closes the loop with T4 and T5: given ε, choose the smallest number of readings K (Equation I4) and halting threshold θ (Equation I2) that meet it. Equation 29 and Table 8 supply the energy model, validated against measurement (H-R).

T11. Mathematical Foundations from the Primer. *Source: Thomas (2018).*

- **Low-rank plus ternary residual.** The Eckart–Young–Mirsky theorem states that truncated SVD is the best rank-r approximation in spectral and Frobenius norm. This replaces the undefined SVD split of Section 6.2 (V-28) with a well-posed one: $W \approx L_r + \gamma T$, with $L_r = U_r \Sigma_r V_r^T$ and T the ternary rounding of $(W - L_r)/\gamma$. For $m = n = 2{,}048$ and $r = 32$ the low-rank part is $32 \times 4{,}096 \times 2$ bytes = 262 KB against 1.05 MB for the ternary part, a 25% increase over ternary alone (H-N9).
- **Projection test.** An orthogonal projection satisfies $P^2 = P$ and $P = P^T$ and gives the nearest point in the subspace; this is the unit test for Equation 11.

- **Convexity.** The distortion σ² 2^(−2b) is convex in b and the budget is linear, so the Lagrangian solution of Equation F2 is the global optimum of the continuous relaxation.
- **Norms.** Spectral and Frobenius norms are invariant under orthogonal transformations, so rotating weights to spread outliers does not change their norm; the spectral norm of a looped block's Jacobian (estimated by the Rayleigh quotient) decides contraction in T1.

I.4 Integrated Design: THME-nano v2

The techniques above combine into a version small enough to build and measure, at the 30–124M scale of TinyLLM where a ternary body can sit in fast memory.

Table I3

Components of THME-nano v2 and Their Origins

Component	Specification	Origin
Harness and sizing	GPT-2-style skeleton, N = 0.05 l³ + 3.2 l, llm.c-based pipeline, data mix near 40:60 domain to web, about 10B tokens	TinyLLM
Input block	Sparse small-world ternary layers, fan-in 16–64 (Equation I5)	Connectome (T6)
Shared looped block	k ∈ {1, …, 4}, normalized recurrence (Equation I1), ternary weights, last-loop KV caching	Dory (T1)
Token mixing	Hebbian–delta fast-weight memory (Equation I3) with reliability-weighted multi-read	Hopfield, CRM, Zhang (T3, T4)
Halting and routing	Integrate-to-threshold halting (Equation I2); frozen boosted-tree router with self-labels	Hopfield, Acta (T2, T8)
Output	Vocabulary-shortlist head with full-softmax fallback	TinyLLM numbers (T9)
Training	Loop count sampled from {1, …, 4}, truncated backpropagation through loops of depth one, difficulty-aware loop curriculum, then ternary quantization-aware training	Dory, BitNet

Component	Specification	Origin
Allocation	Channel-level ILP with Shapley or Hessian sensitivities, frozen at the end	Appendix F, T8
Deployment	Self-speculative decoding (draft k = 1, verify k = 4)	Dory
Evaluation	Risk–coverage curve, scaled ARE, naive-persistence and tuned baselines, joules per token from the estimator	Zhang, Polché, Azure (T5, T7, T10)
Optional sensor branch	Echo-state reservoir with ridge readout	Polché (T7)

In the cache-resident regime, where the ternary body is tens of megabytes, memory energy per bit is far below DRAM, so arithmetic savings (Table 8) matter and the earlier critique of Section 5.3 relaxes. This holds at the nano scale; it does not hold for 70B models.

I.5 Optional Quantum Branch, Gated

The hybrid paper and the Azure documentation are the real sources of the “quantum readings” idea, and the honest reading is that classical readings with consensus (T4) reproduce the structure at lower cost. A quantum branch is allowed to proceed only if all three conditions hold: (i) the classical diagnostics of Equation I6 are favorable for the candidate feature map; (ii) a tuned classical baseline, including naive persistence and a low-rank or tensor-network model, fails on the same task; (iii) a classical simulator cannot reproduce the result at the tested size. The first experiment would be the projected-kernel feature map of Polché et al. on a simulator of 20 qubits or fewer, using the Azure toolchain or any standard simulator, before any hardware use.

I.6 Revised Study Order

Stage 0.5 is added between Stages 0 and 1: run the pre-screens (Equation I6, projection tests, simulations of T4) in minutes. Stage 1 adds components in this order, each against the matched baseline of Table H3: (1) nano baseline reproduced from the TinyLLM recipe; (2) ternary QAT; (3) looped block against an unlooped stack at equal FLOPs; (4) integrate-to-threshold halting; (5) delta memory with multi-read; (6) boosted-tree router; (7) shortlist head; (8) sparse scaffold. With the measured throughput of I.2, each Stage 1 run takes about 22 minutes on an H100-class device.

I.7 Hypotheses and Register Additions

Table I4

Hypotheses from the Source Synthesis and Falsification Criteria (Proposed)

ID	Hypothesis	Falsified if
H-N1	The normalized loop contracts: δ_t decays geometrically	δ_t is non-monotone for more than 20% of held-out tokens
H-N2	Integrate-to-threshold halting gives a better risk–coverage curve than a learned sigmoid gate at equal mean loops	Area under the risk–coverage curve is not lower, CI excluding zero
H-N3	k = 2 improves held-out perplexity over k = 1 with the same parameters, for a ternary nano model	Gain below 1% across 5 seeds
H-N4	Reliability-weighted consensus beats the plain mean and nearest-half under heterogeneous reading errors	Not best in at least 3 of 4 noise settings
H-N5	A boosted-tree router on self-labels reaches AUROC of at least 0.7 and beats logistic regression	AUROC below 0.7 or no gain over logistic regression
H-N6	Loop-conditioned gains improve on fully shared scales	No perplexity gain
H-N7	A sparse scaffold (fan-in 16–64) is within 2% perplexity of dense ternary at 10× fewer weight bytes	Gap above 2%
H-N8	A 4,096-token shortlist covers at least 95% of emitted tokens and brings the depth fraction to about 0.33 with under 0.5% quality loss	Coverage below 95% or loss above 0.5%
H-N9	Rank-32 plus ternary residual lowers transmutation output error against ternary alone at no more than 25% more bytes	Output KL not lower
H-N10	Pre-screen scores (Equation I6) rank feature maps in the same order as measured gains	Rank correlation below 0.5 across at least 6 variants

ID	Claim	Status	Note
V-57	A looped shared block with elastic depth improves accuracy	S at 4B scale (reported); U for ternary nano	Dory: 35.2 to 39.7 to 40.6 at k = 1, 2, 4
V-58	Last-loop KV caching keeps KV memory at one-loop size	S (reported)	Score change −2.1 to +0.4 at k = 4
V-59	Self-speculation with k = 1 draft and k = 4 verify speeds decoding	S (reported)	1.37–1.71×; Equation D1 consistent with acceptance 0.81–0.92
V-60	Early exit saves 56% of compute at nano scale	M	38% once the output head is counted
V-61	Integrate-and-fire time-to-threshold gives adaptive halting	U	H-N2
V-62	The delta memory is Hopfield's synapse rule with decay and error correction	S	Algebra, Equation I3
V-63	Majority error of K independent readings is at most $\exp(-2K(\frac{1}{2} - p)^2)$	S	Requires independence and $p < \frac{1}{2}$
V-64	Quantum readings reduce noise beyond classical readings	C	Shot noise obeys the same $1/\sqrt{K}$ law
V-65	Hybrid quantum models beat classical baselines	U	Classical GP baseline worse than persistence
V-66	Fly-brain sparsity supports sparse language-model layers	U	Structural analogy; H-N7
V-67	Effective H100 training throughput of 2.3×10^{14} FLOP/s	M	Derived from the reported 9 hours; extrapolated to other sizes
V-68	Low-rank plus ternary residual is a well-posed transmutation	S for rank-r optimality; U for downstream quality	Eckart–Young–Mirsky; H-N9
V-69	The ARE metric applies directly to activations	M	Scaled form needed near zero

ID	Claim	Status	Note
V-70	Quantitative evidence for the GBM/LSTM/Transformer ranking in the Acta paper	C in the sections read	Qualitative ratings only

Note on sources. Method names and the items marked “verify” in earlier appendices still need checking before submission. Additional references used here from memory: Chow (1970) on the reject option; Jaeger (2001) on echo state networks; Huang et al. (2021) on quantum kernel diagnostics, cited through Polché et al.; Dao et al. on butterfly factorizations; Grave et al. (2017) on adaptive softmax; Guo et al. (2023) on low-rank plus quantized weights. The Azure documentation and the primer were read selectively, as Table I1 states.

Appendix J. Reconciliation with the THME-Epiphanes Repository and the Project History

This appendix folds in the first measured results (repository release 0.1.9, 12 November 2026) and the earlier project documents from the shared Drive folder. Three results matter. The measurements refute “convert a model to ternary without retraining.” They do not test the architecture proposed in the main text. And the repository’s non-additivity finding is much smaller in the right metric than it first appears (J.4).

J.1 What Was Read, and How Deeply

Table J1

Sources and Reading Depth

Source	Read	Not read
GitHub repository (public, AGPL-3.0 or commercial licence, 57 commits on main)	README in full; the measured-findings note in full; the benchmark dossier tables (per layer, per role, precision sweep, placement control, depth probe, provenance)	Source code, tests, ROADMAP, BACKLOG, CHANGELOG, Studio. I cannot confirm that the code produces the reported numbers, and I did not re-run anything

Source	Read	Not read
Drive: v2.2 research paper (markdown), critique by Omar Abu Halawah, research-history note, Engineering Playbook	In full	None
Drive: v1.0, v2.0, v2.1 documents; simulation results CSV	First-page text or a snippet only	The remainder
Drive: hodge.pdf, navier-stokes.pdf, P vs NP.pdf	First-page snippet only	The papers. I have not checked the Navier–Stokes claim
Drive: “THME - ChatGPT” folder	File listing only	Contents beyond the files named above

J.2 Two Lines of Work Under One Name

Table J2

The Conversion Tool and the Proposed Architecture

	THME conversion tool (v2.x, repository)	THME architecture (this paper)
Question	Can an existing model be re-stored and re-run in mixed formats at lower cost?	Can a model designed around low-bit, looped, hybrid-mixer components beat a Transformer on quality, memory, and energy?
Method	Post-training surgery, quality gate, search over recipes	Training with quantization awareness, loops, and exit heads
Retraining	Optional recovery step, planned for v0.2.3	Required
Covered by the measurements	Yes, in the no-retraining regime	No

Every number in J.3 therefore bears on the left column only. The right column's components (ternary quantization-aware training, looped block, early-exit speculation, delta memory) remain untested.

Naming conflict to resolve. In this paper and in the critique of v2.0, α, β, γ are the binary, ternary, and phase families. The v2.2 text says the prototype defines Beta as binary and initializes every parameter to (0, 1, 0), and the repository note describes v3.0 uses (α as hash tables, γ as phase superposition). Three meanings are in circulation. This paper should name formats by what they are (B, T, P, or Q8_0, TQ1_0) and use α, β, γ only for fractions, as the v2.2 format-contract requirement already implies.

J.3 Reported Measurements

All figures come from the repository's reference CPU forward pass in 32-bit floating point (Intel i5-12400F, WikiText-2 test split, no retraining or calibration). Perplexity is lower-is-better, and Δ is the change from the source model. The first four rows use the full split (303,118 scored tokens); the rest use the first 16,384 tokens (baseline 17.97).

Table J3

SmolLM-135M Results as Reported by the Repository

Experiment	Perplexity	Δ	Reading
Source model	19.80		Baseline
Exported GGUF, uniform Q8_0	19.83	+0.03	Passes the 0.45 gate
Surgery recipe (mlp_down ternary, attention_q Q8_0, layers 1–28)	48.71	+28.91	Fails the gate
Every linear tensor ternary (TQ1_0)	1.59×10^7	$+1.59 \times 10^7$	Model destroyed
One layer ternary at a time		+0.9 to +6.8 (layers 1–28), +13.1 (layer 29), +1,429 (layer 0)	Layer 0 is extremely fragile
One tensor role ternary at a time		+39 (attention_q) to +359 (attention_v)	Roles differ widely
Recipe vs three random placements, equal tensor counts		Recipe +25.5; random +53.3, +82.3, +173.2	All four far outside the gate

Experiment	Perplexity	Δ	Reading
Tokens whose top prediction has settled		1.6% at 30% depth; 28.7% at 80% depth	No trained exit heads
Middle block (layers 7–22) applied twice, untrained	18.0 → 55.3	+37.3	Looping needs training

Pythia-14M (92% of its parameters are embeddings): source 114.29; Q8_0 export 115.12 (Δ +0.83, fails the 0.45 gate); all linear tensors Q4_0 Δ +74.0; every layer ternary one at a time Δ +207 to +1,209.

J.4 Re-Analysis

(a) Additivity depends on the metric. The repository reports that per-layer ternarization costs sum to +1,509 perplexity, while ternarizing all layers together costs +15.9 million, and concludes that an allocator that adds per-layer costs cannot rank aggressive candidates. Perplexity is the exponential of cross-entropy, so effects that add in cross-entropy multiply in perplexity. Taking logs of the per-layer values in the dossier:

$$\Delta CE = \ln((PPL_0 + \Delta) / PPL_0) \quad (J1)$$

Table J4

Additivity of Per-Layer Ternarization Cost in Two Metrics (SmolLM-135M, 30 Layers)

Metric	Sum of 30 single-layer effects	All layers together	Ratio
Perplexity	+1,509	+15,877,276	10,520×
Cross-entropy (nats per token)	8.36	13.69	1.64×

In cross-entropy the interaction is real (super-additive) but moderate, not four orders of magnitude. Layer 0 alone is 4.39 of the 8.36 nats; without it the sum is 3.97. The all-ternary model's cross-entropy is $\ln(1.59 \times 10^7) = 16.6$ nats, above $\ln(V) \approx 10.8$ nats for a vocabulary of about 49,000 tokens, so it is worse than guessing uniformly. At that point no cost model carries information. The accurate statement is that an additive model in log-perplexity underestimates the all-ternary candidate by about 1.6×, and that rank preservation for moderate candidates is untested (hypothesis H-J1). The critique's statement (J.6, point 3) that failure is "mathematically guaranteed" is not supported by this evidence, and the repository's practice of gating whole candidates is still correct.

(b) The gate is not scale-free. A fixed Δ of 0.45 is 2.3% of SmolLM's perplexity but 0.39% of Pythia-14M's. Uniform Q8_0 costs 0.15% on SmolLM and 0.73% on Pythia. Under a 2.3% relative gate the Pythia export passes, and under a 0.39% relative gate it fails, so the verdict depends on the gate. State the gate as both an absolute and a relative margin before any run (Section H.5).

(c) Size baseline. The 513 MiB source equals about 134.5 million parameters at 4 bytes (FP32), and 136 MiB matches Q8_0 at 34 bytes per 32 weights (136.3 MiB). Against an FP16 source of about 257 MiB the reduction is 47%, not 73%. The dossier's nominal-bits column (13.06 bits for Q8_0) does not match a stored size of about 8.5 bits per weight; it should be defined or dropped, in line with the v2.2 rule that theoretical bit counts must not be presented as measured size.

(d) Placement test power. With n random placements, the smallest one-sided p-value a rank test can return is 1/(n + 1): 0.25 for n = 3, and 0.05 only at n = 19. Three draws cannot support a significance claim, as the repository itself says. On Pythia-14M two of the three random draws gave identical results (+587.5467), so the draws were not distinct. Use at least 19 distinct random placements per comparison.

(e) Role sensitivity per weight. Dividing each role's Δ by its share of weights changed gives, in Δ perplexity per percent of weights: attention_v 143, attention_k 22.9, mlp_down 8.6, attention_out 6.6, attention_q 5.3, mlp_gate 4.1, mlp_up 3.8. The value projection is about 38 times more fragile per weight than the MLP up projection. These are single-role effects, so perplexity space is acceptable here. They describe post-training ternarization without calibration, and quantization-aware training may reorder them (hypothesis H-J3).

J.5 Speculative Decoding as the Answer to Routing Divergence

The critique argues that a static execution prior (80% of tokens exiting at 30% depth) fails when real workloads are harder. Under self-speculative decoding (Appendix D.3, and Dory's k = 1 draft and k = 4 verify) the point changes. Because rejection sampling preserves the verifier's output distribution in exact arithmetic, a harder workload costs speed, not quality. With acceptance rate a and draft length γ, the verifier reads the deep weights once per verification step, so the deep-layer reads per emitted token are

$$q_{deep}(a, \gamma) = (1 - a) / (1 - a^{(\gamma+1)}) \quad \text{(J2)}$$

which replaces the fixed prior $q_l \approx 0.2$ in the execution-weighted allocator. Combining Equation J2 with the speedup model of Equation D1 at Dory's settings (draft cost ratio c = 0.375, γ = 4):

Table J5

Speed and Deep-Layer Read Weight Against Acceptance Rate (Model Values)

Acceptance a	Speedup vs verifier alone	Deep-layer reads per emitted token
0.30	0.57×	0.70
0.50	0.78×	0.52
0.64	0.99× (break-even)	0.40
0.80	1.34×	0.30
0.90	1.64×	0.24

The speed floor as a → 0 is 1/(γc + 1) = 0.40×, and speculation pays only above a ≈ 0.64. So the “catastrophic cascade” of the critique becomes a bounded slowdown with unchanged quality, which is a weaker failure but still a real one on hard, low-acceptance workloads. The execution prior must be measured per workload (code, mathematics, long context), not assumed. The draft cost ratio c must be measured too.

J.6 Challenger–Defender Record: The Four Critiques of v2.0

Table J6

Critique by Omar Abu Halawah, Evidence Now Available, and Response

Critique	Evidence now	Concede	Defend	What settles it
1. Mixed formats inside a tensor break kernel alignment and erase bandwidth savings	None: the repository measures no speed	Channel-interleaved formats need custom kernels, and savings are unproven	Allocate per tensor or in contiguous channel blocks sorted by format (Appendix F.4); the repository’s recipes are per tensor already	Stage 3 microbenchmark: tokens per second at batch 1 against uniform 4-bit
2. A static 80/20 routing prior diverges on hard workloads	Untrained exit is weak (1.6% settled at 30% depth)	A fixed prior of 0.2 is not defensible	Speculation keeps quality invariant; speed floor 0.40× (J.5)	Acceptance rate on code, mathematics, long context

Critique	Evidence now	Concede	Defend	What settles it
3. Additive screening hides non-additive interactions and is “guaranteed” to fail	Super-additive by 1.64× in cross-entropy; 10,520× in perplexity (J.4a)	Additive screening cannot decide final candidates	The guarantee is not supported; whole-candidate gating already exists	Predicted against measured cost on at least 20 random candidates
4. Any retained attention layer leaves the KV cache unbounded	Not measured	Correct; Appendix D.2 already calls zero cache the least likely claim	A hybrid reduces the cache to the fraction of full-attention layers (1 in 8 gives 12.5%) plus bounded windows, which is reduced, not zero	Context-length sweep of KV bytes and recall

The critique also asserts that no training or hardware result exists. That was correct for v2.0; as of J.3 there are measured quality results for conversion without retraining, and still no speed, energy, cache, or trained-model result.

J.7 Cross-Reference of Hypotheses

Table J7

Repository Hypotheses H1–H9 Against This Paper

Repo ID	Hypothesis	Counterpart here	Status after J.3
H1	Import and execution	Gate G0, Stage 0	Partly supported: forward pass and Q8_0 round trip measured; per-tensor export not yet written

Repo ID	Hypothesis	Counterpart here	Status after J.3
H2	Heterogeneous beats uniform at matched storage	H-A1, H-T	Not tested; uniform Q8_0 is nearly free and every ternary assignment without retraining is expensive
H3	Structured placement beats random	H-A1	Exploratory support (3 of 3; test cannot reach p below 0.25)
H4	Global refinement helps	H-A4	Not tested; the 1.64× interaction shows why it is needed
H5	Cache efficiency	H-M, H-C1	Not tested
H6	Energy efficiency	H-R, H-D4	Not tested
H7	Teacher recovery	H-I, Stage 2	Not tested, and now the critical path
H8	Task specialization	None (see TinyLLM, Appendix I)	Not tested
H9	Design-space dependence	None	Not testable on one model and one task

J.8 Changes to Earlier Appendices, New Hypotheses, and Next Steps

Table J8

Corrections and Refinements to Earlier Appendices

Where	Earlier statement	Change
Appendix F.1 (linear program)	Distortion is linear in the fractions and additive across layers	Use cross-entropy (or logit KL), not perplexity, as the additive quantity; calibrate the additive model against measured joint cost (H-J1); keep whole-candidate gating

Where	Earlier statement	Change
Appendix F.7	Multiple-choice allocation treated as an integer program	The problem is NP-hard in general but pseudo-polynomial: dynamic programming over L layers, K formats, and S budget units costs $O(L \cdot K \cdot S)$, for example $30 \cdot 5 \cdot 10{,}000 = 1.5 \times 10^6$ operations, so NP-hardness is not a practical obstacle at this size
Appendix D.9	First experiment: MLP-only ternary conversion	Measured without recovery, ternarizing mlp_down alone costs Δ +170, mlp_gate +80, mlp_up +75 on SmolLM-135M; this experiment needs quantization-aware training or distillation to be meaningful
Appendix D.3 and I.2	Exit-layer agreement should be measured	Untrained baseline now exists (1.6% settled at 30%, 28.7% at 80%); the test is whether fitted exit readouts improve on it
Appendix I, T1	Looped block risk: untested at small scale	Untrained looping costs Δ +37.3 on SmolLM-135M, consistent with Dory needing about 1.5 trillion tokens of loop training
Table H3 and Section H.5	Random controls unspecified	At least 19 distinct random placements per comparison; gate stated as absolute and relative
Execution-weighted allocation	Fixed deep-layer prior	Replace by Equation J2 with measured acceptance

Table J9

New Hypotheses and Falsification Criteria (Proposed)

ID	Hypothesis	Falsified if
H-J1	An additive model in cross-entropy ranks whole candidates correctly	Spearman correlation between predicted and measured cost below 0.8 on at least 20 random candidates, or calibration slope outside 0.5–2
H-J2	A role-by-layer precision floor (layer 0, last layer, attention V kept high) shrinks the search space without losing the best candidate	A candidate below the floor beats the best candidate above it, at equal size
H-J3	Post-training sensitivity predicts post-QAT sensitivity	Rank correlation below 0.6 across layers and roles
H-J4	Fitted linear exit readouts raise settled-token share well above the untrained 1.6% at 30% depth	Settled share below 20% at 30% depth after fitting
H-J5	A passing heterogeneous allocation smaller than uniform Q8_0 exists with 4-bit formats and calibration	No candidate passes the gate at a size below uniform Q8_0

Ordered next steps: (1) test H-J1 on the existing per-layer data plus random candidates, because it costs only forward passes; (2) the repository's own next experiment, 4-bit search under the gate (H-J5), run on both models; (3) fitted exit readouts (H-J4); (4) quantization-aware recovery before any further ternary claim; (5) the speed and energy measurements of Stage 3, since none exists.

J.9 Register Additions

ID	Claim	Status	Note
V-71	Per-layer perplexity increases add across layers	C in perplexity; M in cross-entropy	1,509 against 15.9 million; 8.36 against 13.69 nats (1.64×)
V-72	All-linear ternary without retraining gives a usable model	C	Perplexity 1.59×10^7, worse than uniform guessing
V-73	Q8_0 export is near-lossless	S on SmolLM-135M; M on Pythia-14M	Δ +0.03 and +0.83; verdict depends on gate (J.4b)

ID	Claim	Status	Note
V-74	Sensitivity-guided placement beats random placement	U	3 of 3; smallest attainable p is 0.25
V-75	73% size reduction	M	Relative to FP32; 47% relative to FP16
V-76	Early exit is available without training	C	1.6% settled at 30% depth
V-77	Looping an untrained block is harmless	C	Δ +37.3
V-78	Distribution shift in token difficulty cannot change output quality under speculation	S	Speed floor 0.40×; break-even acceptance 0.64 (c = 0.375, γ = 4)
V-79	Deep-layer read weight under speculation is (1 − a)/(1 − a^(γ+1))	S	Derivation, J.5
V-80	Attention V is the most fragile role per weight	S for this model and regime	143 Δ perplexity per percent of weights; may change after QAT
V-81	A fixed absolute gate of 0.45 is scale-free	C	2.3% of SmolLM perplexity, 0.39% of Pythia
V-82	The critique's "guaranteed catastrophic failure" from additive screening	C	Interaction is 1.64× in cross-entropy

J.10 Open Items

- **Licence.** The Engineering Playbook states Apache-2.0 for the project under its earlier name, LLM-Tweaker; the repository now states AGPL-3.0 or commercial. Confirm which applies before sharing.
- **Nominal-bits column.** Define it or remove it (J.4c).
- **Code not reviewed.** Nothing here verifies the repository's code. A reviewer should confirm that the reported commands regenerate the dossier tables.
- **Clay-problem files.** The Hodge and P versus NP statements have no operational role in THME beyond the Calabi–Yau analogy (Section 4.5) and the knapsack hardness discussed in J.8. The Navier–Stokes file claims finite-time blow-up with bounded

energy; I have not checked it, and a blow-up result cannot supply a stability guarantee in any case.

- **Unread Drive documents.** The v1.0, v2.0, and v2.1 documents and the simulation CSV were seen only as snippets; they are analytic simulations, not trained-model evidence, according to the research-history note.

Appendix K. The Tool-Building Loop: A Proposer That Tunes Layers Under a Frozen Evaluator

The repository exists to build tools that are not publicly available, so this appendix specifies the self-tuning loop as a tool design. A small language model proposes changes, a deterministic harness tests them, and a result that passes is frozen into a configuration file. The open design question is the schedule: tune all layers at once, or lock one layer and tune it until it passes. K.3 answers it for a controlled toy, and K.3 also states what that answer does not transfer.

K.1 Licence and Tooling Notes

The project is moving to AGPL-3.0. I am not a lawyer, so treat these points as items to confirm. As sole author you can license your own code under AGPL-3.0 and also offer a commercial licence. Copies already released under Apache-2.0 stay Apache-2.0 for those who received them. Third-party code you bundle needs a licence compatible with AGPL-3.0, and an outside contributor needs a contributor agreement if you want to keep the commercial option. A software licence does not cover this paper's text and figures, which need a separate choice.

Table K1

Added Plugin Components and Their Use for This Research

Component	Fit	Use here
scientific-coding rules	High	Govern every script in the loop: explicit seeds, no silent defaults, fail loudly, raw data immutable, validated code not refactored
Wolfram tools	High	Symbolic checks of formulas in the paper, as in K.3
brainstorming-research and writing-research-plan	Medium	Turn H-J1 to H-J5 and H-K1 to H-K4 into a pre-registered plan before any run

Component	Fit	Use here
peer-reviewer (adversarial role)	Medium	Review the evaluator design and the paper before results are claimed
analyst agent	Medium	Runs the analysis scripts with assumption checks
literature, source-ingest, wiki and grant skills	Low	Built for theology and humanities sources and German and European funders; not needed for the maths, apart from ingesting your nine papers if you want a searchable wiki

K.2 Loop Design

The loop has four parts, and the separation between them is what keeps it honest.

1. **Proposer.** A small language model (or any search method) outputs a candidate. For allocation, a candidate is a table assigning a format to each layer and role. For a new formula, it is an expression in a restricted language built from the paper's primitives (semiring operations, solver steps, gates), never free-form code run with full access.
2. **Evaluator.** A deterministic program that scores a candidate with a fixed gate, fixed seeds, and fixed data splits. The proposer cannot read its source or change it.
3. **Ledger.** An append-only log of every candidate, its score on each split, and the evaluation count. The count feeds the multiple-comparison correction of Section H.5.
4. **Freeze.** A passing candidate is written to a configuration file, a regression test is generated from it, and from then on it is code, not a proposal. This is the "hardcoded by itself" step.

Three data splits are required: tune (the proposer sees scores), select (picks among passing candidates), and final (touched once per released result). Without them the loop learns the benchmark, not the model. The baseline for every claim is random search and a standard optimizer (an evolutionary or Bayesian search) at the same evaluation budget; a language-model proposer is worth its cost only if it beats those (H-K1).

The convergence target from Appendix F stands: accuracy approaches its ceiling by a sequence of smaller gains, and it is reported as a fitted curve with an interval, never as reaching 1.

K.3 Schedule: All Layers at Once or One Layer Locked

Consider L layers whose tuning variables interact through a symmetric positive definite matrix A. Take unit diagonal and equal pairwise coupling r:

A = (1 - r) I + r 11^T (K1)

Wolfram confirms the eigenvalues: 1 - r (repeated L - 1 times) and 1 + (L - 1) r. Updating all layers at once with a unit step is the Jacobi iteration, which converges only if every eigenvalue of A lies in (0, 2). That gives the condition

r < 1 / (L - 1) (K2)

For L = 30 the limit is r = 0.0345. Locking one layer at a time and solving it exactly is the Gauss-Seidel iteration, which converges for every symmetric positive definite A, so it has no such limit.

Table K2

Evaluations to Reach 10^-6 Relative Error, L = 30, Toy Quadratic (One Seed)

Coupling r	Simultaneous, unit step	Simultaneous, safe step 1/lambda_max	Same, black-box (31 evaluations per step)	Lock one layer, cyclic
0.01	10	10	310	150
0.03	81	19	589	240
0.05	diverges	29	899	330
0.10	diverges	53	1,643	600
0.30	diverges	185	5,735	3,060

The unit-step rows match Equation K2: converged at r = 0.03, diverged at r = 0.05. The safe step needs the largest eigenvalue, which a black-box search does not know. The black-box column charges 31 evaluations per step because a gradient estimated by finite differences needs one evaluation per layer plus one; that is an assumption about the setting, not a measurement. Where gradients are free (training with backpropagation), simultaneous updates with a controlled step are the cheaper schedule and are what quantization-aware training already does.

What transfers and what does not. The toy is quadratic with equal coupling and a known matrix. The repository's measured interaction is a 1.64-fold super-additivity in cross-entropy (Section J.4), which is not a pairwise coupling coefficient, so Equation K2 gives a shape of the risk, not a threshold for any real model. Discrete format choices have no gradient at all. Three consequences follow.

- For discrete allocation (no gradient), lock one layer or one small group, evaluate every format for it with the others frozen, accept the best, and sweep. A sweep costs L times K evaluations (30 layers and 5 formats give 150). Accept a move only if it

improves the tune score by more than the noise, estimated from a bootstrap over tokens.

- Order the sweep by measured fragility. Layer 0, the last layer, and the attention value projection come first, because the repository found them most fragile, and a late change there can invalidate what was tuned after them.
- For continuous parameters with gradients, update everything together but with a trust region: cap the step, and shrink it whenever the combined step is worse than the best single-layer step. Tuning in blocks of weakly coupled layers is the middle path.

A full sweep with no accepted move is the stopping rule for the lock-one schedule. A layer "passes" when no format change improves it beyond the noise while the global gate still holds on the select split.

K.4 What the Proposer Adds

The schedule does not need a language model. The model's value would be proposing structural changes that a numeric optimizer cannot express, such as a new gate or mixing rule, and using the ledger's history to avoid repeating failures. That is a claim to test, not assume (H-K1). Two risks are specific to this design: the proposer may find a candidate that exploits the evaluator (a gap between tune and final scores), and a small model may propose less diverse candidates than random search.

Table K3

Hypotheses for the Tool-Building Loop (Proposed)

ID	Hypothesis	Falsified if
H-K1	A language-model proposer reaches a passing allocation with fewer evaluations than random search and than a standard optimizer	At equal evaluation budget and at least 5 seeds, its median evaluations-to-pass is not lower, or the difference interval includes zero
H-K2	Lock-one, fragility-ordered sweeps reach the gate in fewer evaluations than proposing whole allocations at once	The whole-allocation arm needs fewer evaluations at the same gate and seeds
H-K3	The frozen evaluator is not exploitable	Final-split score is worse than select-split score by more than the bootstrap noise, over at least 5 seeds
H-K4	A frozen configuration reproduces its scores	A rerun on another seed set or machine differs by more than the stated tolerance

K.5 Register Additions

ID	Claim	Status	Note
V-83	Equal-coupling matrix has eigenvalues 1 - r (L - 1 times) and 1 + (L - 1) r	S	Symbolic check, L = 6; general form is standard
V-84	Simultaneous unit-step updates diverge when r > 1/(L - 1)	S	Toy: converged at 0.03, diverged at 0.05 for L = 30
V-85	Lock-one (Gauss-Seidel) converges for every symmetric positive definite coupling	S	Standard result; toy agrees
V-86	Lock-one is cheaper than simultaneous updates when gradients cost L + 1 evaluations	M	Toy only, one seed; depends on the 31-evaluation assumption
V-87	A language-model proposer beats random search	U	No test yet (H-K1)
V-88	The toy threshold applies to real layers	U	Real coupling not measured pairwise

K.6 Open Decisions for the Owner

- Which model is the proposer (a small local model is cheaper and fits the licence story), and what is the restricted formula language it writes in.
- The first target: allocation under the gate (smallest), or the new mixing operator (largest).
- Whether the final split is a held-out corpus other than WikiText-2, because tuning only on WikiText-2 risks fitting that corpus.

Appendix L. Open-Core Foundations and the Extension Interface

The plan is to release the foundations and the mathematical engine as open source under AGPL-3.0, then add proprietary formulations and algorithms later as a commercial tier. This appendix grounds that plan in the repository as it stands (commit c83a236, 13 November 2026, a shallow read of the module layout and the BYOA source file; I did not run the code or the tests). A second check later the same day found two newer commits (f634813: AWQ-style quantization and SmoothQuant-style scale migration), which are

reflected in Table J7 and the next steps below; every other observation in this appendix is as of c83a236, so a coding agent should run git log first and proposes what the foundation still needs.

L.1 What the Repository Contains Now, and Corrections to Appendix J

The source has about 31,700 lines of Rust in 17 modules. The largest are the Studio GUI (about 8,800 lines), the command line (4,400), the exporter (3,500), inference (2,900), profiling (2,400), and the intermediate representation (1,900). The allocator is small (about 190 lines). The micro-trainer is about 300 lines.

The licensing groundwork is already in place: the package declares AGPL-3.0-only, a LICENSING.md describes the dual-licence model, a deny.toml allow-list restricts dependencies to permissive licences so that a commercial licence stays possible, CONTRIBUTING.md carries contributor terms, and release archives include third-party notices. Several items from my earlier notes (dependency audit, contributor terms) were therefore done before I raised them.

The roadmap marks the following as done, which makes parts of Appendix J out of date. Appendix J was written from the README and dossier, and I had not read the roadmap.

- Per-tensor GGUF export (v0.2.2.1). Table J7 said the per-tensor export was not yet written; it is.
- A measured search (v0.2.1): a cost table per tensor, a greedy allocator under a size budget, global refinement that reverts moves until the whole candidate passes, random-placement controls (default 5 draws), a Pareto-front command, and a held-out test slice that the search never reads.
- Calibration activations from the forward pass (thme calibrate). Two later commits on 13 November add AWQ-style activation-aware quantization (v0.2.2.4, implemented, backlog status blocked on a real-model measurement) and SmoothQuant-style scale migration (v0.2.2.7, done; effect after quantization not measured).

No search report is committed yet. The benchmark folders hold only the precision sweep, the depth probe, and the verification report for each model. So hypotheses H2 and H-J5 are still untested, but the tool that tests them now exists. The existing greedy-then-refine design also gives a direct way to run H-J1: compare the additive prediction from the cost table with the measured cost of the refined candidates.

L.2 Proposed Open and Commercial Split

The line below is a proposal for you to change. The organizing idea is that the open core supplies mechanisms, interfaces, and measurement, while the commercial tier supplies tuned policies and proprietary mathematics.

Table L1

Proposed Open-Core and Commercial-Tier Split

Layer	Open core (AGPL-3.0)	Commercial tier (later)
Model I/O and formats	Importers, GGUF and SafeTensors writers, standard codecs (Q8_0, Q4_0, K-quants, ternary)	Proprietary formats and kernels
Measurement	Forward pass, quality gate, splits, ledger, provenance, benchmark reports	Hosted evaluation at scale
Mathematical engine	Published methods: linear and integer programs, dynamic programming for allocation, transport, embedded Runge-Kutta steppers, semiring operations, consensus reads	Proprietary formulations and algorithms
Search and tuning	Schedulers (Section K.3), baselines, the proposer interface, the freeze step	Trained proposers, prompt and recipe libraries, tuned policies
Extension interface	Protocol specification and reference plugins	Closed plugins built on it
Service	None	Hosting, support, updates

Two cautions about the line. First, anything committed to the public repository is public: LICENSING.md states that copies up to commit 56ae520 remain available under Apache-2.0 to whoever received them, so formulas in that history can be used commercially by anyone, and git history cannot be unpublished. Second, the research programme PDF and the findings note are already in the public docs folder. Decide what is proprietary before it is committed, not after.

L.3 The Foundation Gap: Algorithms Are Declared, Not Programmed

The BYOA engine takes an algorithm definition (identifier, author, version, category, bits per parameter, target roles) whose body is a TransformationRule: a fixed enumeration with seven variants (codebook quantization, absmean ternary, orthogonal rotation, structured sparsity, SmoothQuant-style migration, affine projection, rotated quantization). Each rule carries a weight contract (reconstruct, rotation, or not weight-only), and mixers and the affine projection are refused with a stated reason.

This design is safe, because a definition is data and runs no foreign code. It also means a user can only combine parameters of the seven built-in rules. A genuinely new piece of mathematics, open or closed, requires editing the open-source enumeration. So today BYOA cannot carry a closed-source formulation, and it cannot express the paper's new components (semiring layers, ODE steps, the consensus-read mixer). Only two Rust traits exist as extension points, CausalLm for the model runtime and QualityEvaluator for scoring, and the allocator is a concrete type.

L.4 Proposed Extension Interface

The aim is for plugins to bring new mathematics without touching the AGPL source and without the core trusting them.

Table L2

Proposed Plugin Kinds

Kind	Input	Output	Core-side check
Codec	Tensor, role, parameters	Encoded blocks and a decode description	Core decodes and measures reconstruction and size itself (the reconstruct contract)
Transform	Tensor, role	Rotated tensor plus the compensation it requires	Core verifies the compensated forward pass matches
Cost model	Candidate allocation	Predicted cost	Scored against measured cost (H-J1)
Proposer	Ledger history	Candidate edits	Evaluated only by the core gate
Scheduler	Layer list, history	Order of layers to tune	None needed; effects show in the ledger
Layer operation	Activations, state	Activations, state	Runs inside the forward pass; see below

Proposed rules: plugins run as separate processes, speaking a versioned message protocol over standard input and output, with tensor data passed as paths to memory-mapped files rather than inline. A plugin declares its kind, version, determinism, and resource limits, and runs without network access. It never sees the test split and cannot

read or change the gate; the core records the plugin identifier, version, and a hash of its binary in the ledger for every candidate. Because the core measures every result itself, a closed plugin can be used without being trusted.

Layer operations are the hard case. A mixer or an ODE step executes per token in the forward pass, where a process boundary is too slow. Two routes exist. A commercial build can link proprietary layers directly with the library under the commercial licence, which you can grant because you hold the copyright. Third parties who want the same in-process access need the commercial licence or must open their layers under the AGPL. Which route you choose is a licensing decision for your lawyer, and it should be settled before the plugin protocol is frozen, because the recipe schema is already published as version 1.

L.5 Technique Inventory

The table lists the integration techniques in this paper against the repository state. It is an inventory of what the paper discusses, not a claim that the list is complete.

Table L3

Techniques, Paper Sections, and Repository Status

Technique family	Paper	Repository (per roadmap and code)	Evidence
Per-tensor format assignment (Q8_0, Q4_0, K-quants, ternary)	D, F	Per-tensor export and K-quant codecs done	Q8_0 measured; others tested
Cost-table search with global refinement and random controls	F, J, K	Done (thme search); no committed report	Untested

Technique family	Paper	Repository (per roadmap and code)	Evidence
Activation-aware scaling, second-order rounding, Hadamard rotation, outlier migration	D	Updated at f634813 (13 Nov): AWQ-style rules (thme.awq_q4_0, thme.awq_q4_k, with --calibration) and SmoothQuant-style migration folded into the norms (inference::smoothing) are implemented; second-order rounding and measured rotation remain open; rotation exists as a BYOA rule	Tested on synthetic weights and the synthetic model; output unchanged before quantization (smoothing); not measured on a real model (model runs paused by the owner)
Recovery by distillation or quantization-aware training, LoRA	D, I	Micro-trainer exists; backward pass for the reference model open (v0.2.3.1)	Untested
Looping and early-exit heads	I	Depth probe measured untrained; training open	Untrained fails
Self-speculative decoding	D.3, J.5	Speculative module exists; speedups are estimates	Estimate
Consensus-read mixer and delta memory	E	Not in the repository (scoped as v0.3 architecture work)	Toy simulation only
Exact or dynamic-programming allocation, transport	F, G	Greedy allocator only	None
ODE or state-space layers, unrolled solvers	G, I	None	None
Proposer loop (language-model tuning)	K	Co-surgery loop and QualityEvaluator exist; no proposer	None
Speed, memory, and energy measurement	G, H	Roofline estimates; measured work is v0.2.4	Estimate

L.6 Build Order for the Open Core

The order follows the evidence: quality first, then recovery, then speed, with new mathematics added only where a measurement can test it.

1. Run the existing search on both models and commit the reports. This tests H2, H-J5, and (with the cost table) H-J1, and it satisfies the v0.2.1 exit criterion.
2. Finish v0.2.2 (activation-aware scaling and scale migration are implemented but unmeasured on a real model; second-order rounding and rotation measured with and without remain open) so that the 4-bit search has a fair baseline.
3. Build the recovery path (v0.2.3). Appendix J shows that ternary without retraining fails, so recovery is the critical path for everything ternary.
4. Add a mathematical-engine module with validated primitives: an exact dynamic-programming allocator, a transport solver, embedded Runge-Kutta steppers, and semiring operations. Each needs tests against analytic solutions, limiting cases, and convergence checks, in line with the scientific-coding rules.
5. Draft the plugin protocol and its reference plugins, and settle the layer-operation licensing question, before the recipe schema leaves version 1.
6. Add speed, memory, and energy measurement (v0.2.4), since no efficiency claim in the paper is measured yet.
7. Scan the full git history for anything proprietary before the public release, and keep the proprietary work in a separate private repository.

L.7 Register Additions

ID	Claim	Status	Note
V-89	The repository already has dual-licence files, a permissive-only dependency policy, and contributor terms	S	Observed at commit c83a236; legal sufficiency not assessed
V-90	BYOA is declarative, with seven rule variants and no foreign code	S	Observed in the BYOA source file
V-91	No search report is committed	S	Benchmark folders list three report types per model
V-92	Copies before commit 56ae520 stay available under Apache-2.0	S	As stated in LICENSING.md; legal effect to confirm
V-93	Table J7 entry for H1 (per-tensor export not yet written)	C	Superseded by v0.2.2.1

L.8 Open Items

- The open and commercial line in Table L1 is a proposal.
- I read the module layout, the BYOA file, the roadmap, and the licensing files. I did not read the allocator, search, or inference code, and I ran nothing, so the roadmap status of each item is the repository's own claim.
- The licensing questions (plugin exception, layer-operation linking, contributor terms) need a lawyer.

Appendix M. Math Engine Specification for Implementation

This appendix is a hand-off. It gathers every suggestion about the math engine, tools, graphing, visual engines, and conversions into work packages that Claude Code (or a human contributor) can implement in the THME repository later. It is written to be read next to the repository's own guide (AGENTS.md, docs/WORKFLOW.md, BACKLOG.md, ROADMAP.md), which takes precedence wherever the two differ.

Scope decisions you made: the engine is a Rust module inside the repository; conversions cover number formats, model and tensor file formats, mathematical representations, and units and domains; the visual engine covers solver traces, layer and tensor maps, geometry and manifolds, and dataflow graphs; and the focus is the mathematics and tooling, not further testing of language models. Brand, name, and design choices are on hold, so nothing here restyles them.

M.1 Rules the Repository Already Sets

The repository guide is strict, and every work package must satisfy it.

- Format and lint with zero warnings (cargo fmt --check, cargo clippy -- -D warnings) and pass cargo test on three operating systems; Rust 2021, minimum version 1.89, pinned toolchain in rust-toolchain.toml.
- No unwrap, expect, or panic on external input. Engine functions return a structured Result.
- Dependencies stay pure Rust and permissive (deny.toml allow-list). The prototype below uses none.
- Work starts from a ticket in BACKLOG.md (fields: milestone, roadmap item, priority, area, type, status, problem, done-means) and lands through a pull request using the template. Branches are feature/, fix/, perf/, docs/; commits are conventional.
- Every figure must carry an evidence level: Measured, Tested, Estimate, or Simulated. Engine results are Tested until a report under docs/benchmarks/ says otherwise.
- No absolute developer paths, no model weights, no binaries in the repository.

- Each file carries SPDX-License-Identifier: AGPL-3.0-only.

The scientific-coding rules add: explicit parameters with no hidden defaults for scientific quantities, explicit seeds, errors that stay visible instead of fallback values, and validation against analytic solutions, conservation laws, limiting cases, and convergence rather than against the code's own output.

M.2 What Exists, and Two Findings

On the checked-out commit (c83a236) there is no src/engine/ module. Related pieces exist: a normalized Walsh-Hadamard transform inside the BYOA transform code, used by the rotated-quantization rule; a threshold-based allocator; a software 3D renderer in the Studio live view; and Python scripts that generate the dossier charts from committed reports.

Finding 1: the allocator is not a dynamic program, in the part I read. The subsystem reference and the milestone ledger describe a 'Pareto DP allocator' that 'solves a multi-constraint knapsack problem'. The first 70 lines of src/allocator/pareto.rs, which I read, choose a format per tensor by comparing a sensitivity score with fixed thresholds (0.75 and 0.45 for aggressive targets), with a fallback compression target of 4.0 when none is given. I did not read the rest of the file or the search code, so the claim is limited to what I saw. If it holds, either the documentation should change or the exact solver in WP2 should be added and wired in. This also corrects Table L3 in Appendix L, which listed only a greedy allocator.

Finding 2: the Hadamard transform should be shared. The engine's basis module (WP5) should reuse or replace the existing transform, not copy it, because two copies can drift apart and silently confound a comparison.

M.3 Prototype

A first slice was prototyped outside the repository as a standard-library-only crate with the intended layout (error, numfmt, alloc). It compiled and passed 8 tests on rustc 1.97. It has not been checked with the repository's pinned toolchain (1.99.1) or MSRV (1.89), and cargo fmt and clippy were not run. The files are delivered with this document as an optional head start; they are not in the repository. Evidence level: Tested.

Table M1

Prototype Test Results (rustc 1.97, standard library only)

Test	Oracle	Result
Absmax integer quantization, bits 2, 4, 8, 12, 20 seeds each	Every element within scale / 2 of the original	Pass

Test	Oracle	Result
Error falls with bits (4 vs 8)	MSE ratio above 50 (theory about 256)	Pass
Binary sign quantization scale	Mean absolute value gives the lowest MSE among 39 alternative scales	Pass
Ternary absmean	Codes in {-1, 0, 1}; all-zero input reconstructs exactly	Pass
Codebook nearest level	Matches brute force on 300 values; empty codebook and NaN input return errors	Pass
Dynamic-programming allocation	Equals exhaustive search on 300 random instances; reconstruction cost matches reported cost; infeasible budgets return an error	Pass
Greedy against exact	Constructed instance: exact 100 against greedy 110; greedy never beats exact on 200 random instances	Pass

M.4 Work Packages

Each package lists what to build, the oracle that proves it right, and a done-means sentence suitable for BACKLOG.md. Suggested order is the table order; a package may start once the ones it depends on are merged.

Table M2

Math Engine Work Packages

WP	Module	Build	Oracle tests	Done means
WP1	engine::error	Result type with invalid-input, infeasible, no-convergence, and non-finite variants; finite-value guard	Each variant reachable from a test	Every engine function returns Result; none panics on bad input

WP	Module	Build	Oracle tests	Done means
WP2	engine::alloc	Exact dynamic-programming multiple-choice allocation (prototyped); extensions: second constraint (size and latency) by two-dimensional table or Lagrangian bound; additive cost in cross-entropy, not perplexity (Appendix J.4)	Exhaustive search on small instances; DP never worse than greedy; reconstruction equals reported cost	thme search accepts a solver choice; DP result is never worse than greedy at equal budget on the committed cost tables
WP3	engine::numfmt	Prototyped integer, ternary, binary, codebook; add FP16, BF16, FP8 (E4M3, E5M2) with round to nearest even; block formats as in the exporter; phase quantization onto K angles; tropical (log) conversion	Absolute bound scale / 2 for integers; relative bound 2^-(mantissa bits + 1) for floats; phase error at most pi / K; round trip of exactly representable values	Each conversion reports a stated error bound that its tests confirm
WP4	engine::transport	Northwest-corner start; Sinkhorn with a log-domain variant for small regularization; optional exact network-simplex solver	Marginals match; two-by-two case with known optimum; cost approaches the optimum as regularization falls; northwest-corner cost never below the optimum	Plans satisfy marginals to a stated tolerance; non-convergence returns an error

WP	Module	Build	Oracle tests	Done means
WP5	engine::basis	Fast Walsh-Hadamard (shared with the BYOA code), FFT with a naive DFT as oracle, Givens and Householder rotations	Orthonormal (norm preserved); applying twice returns the input; Parseval; FFT equals DFT; impulse gives a flat spectrum	The BYOA rotation calls the shared function, with identical results before and after
WP6	engine::ode	Dormand-Prince 5(4) embedded Runge-Kutta with step-size control, classic fixed-step RK4, explicit configuration with no default tolerances; function-evaluation count reported	y′ = -y against exp(-t); harmonic-oscillator energy drift; observed order about 4 for RK4 and 5 for Dormand-Prince; step count rises as tolerance tightens (adaptive compute, Appendix G)	Solutions meet the requested tolerance on analytic problems; coefficients verified by order tests, not by eye
WP7	engine::semiring	Generic matrix product over real, tropical min-plus and max-plus, Boolean, and saturating ternary semirings	Semiring axioms by property test; min-plus power equals Floyd-Warshall shortest paths; real case equals ordinary matrix product	Layer types of Appendix I (T-series) expressible as one function with a semiring parameter

WP	Module	Build	Oracle tests	Done means
WP8	engine::project	Symmetric eigen-solver (Jacobi, adequate to 11 dimensions); PCA with explained-variance ratio always returned; participation ratio (sum of eigenvalues squared over sum of squares) as an effective-dimension estimate; classical multidimensional scaling; stereographic and perspective projection; rotation in a chosen coordinate plane	Points on a known 2-plane in 5-dimensional space give explained variance of 1; projection of orthogonal axes; rotation preserves distances	Every projection reports how much structure it kept, so a view never claims more than it shows
WP9	engine::viz	Dependency-free SVG writer: log-axis line plots (solver traces), heatmaps (layer by role: precision, sensitivity, allocation), scatter (projected manifolds), layered directed graph (dataflow), Pareto front	Element counts match data; byte-identical output for identical input; NaN or non-positive value on a log axis returns an error; visual review by rendering to an image	Dossier and Studio can embed the SVG; colors come from the existing design tokens; palette passes a color-blindness check

WP	Module	Build	Oracle tests	Done means
WP10	engine::convert	Registry of typed conversions (below) with path search and accumulated error bounds	Round trips (Hadamard twice, FFT then inverse, discretize then compare to the analytic solution); path error bound at least the measured error	A conversion without a declared bound or contract cannot be registered
WP11	engine::clifford	Geometric product for Cl(p, q) on bitmask basis blades, rotors, norm	e_i squared equals its signature; distinct basis vectors anticommute; associativity; rotor sandwich preserves the norm; signature as fixed in Section 4 of this paper	Cl(m + 1, 1) products match hand-computed cases
WP12	engine::lp	Small dense simplex solver, branch and bound on top for integer variables	Textbook linear programs with known optima; duality gap zero; cross-check against Wolfram; integer instances against exhaustive search	Solves the allocation relaxation of Appendix F and matches WP2 on integer instances
WP13	integration	thme engine subcommands; Studio view that embeds WP9 output; BYOA codecs backed by WP3	Existing tests still pass; measured reports unchanged	No change to any committed measured number without a new report

M.5 Conversions as a Typed Registry

A conversion is an edge in a graph, not a loose function. Each edge declares its source, target, exactness (exact, bounded, or empirical), a formula for its error bound, whether an inverse exists, and a cost. This extends the weight contract the repository already uses (reconstruct, rotation, not weight-only) to every kind of conversion, and it lets a tool refuse a path whose accumulated error exceeds a budget. Absolute errors add along a path in the worst case, so the registry sums bounds and flags an empirical edge as unbounded.

Table M3

Conversion Families the Registry Should Cover

Family	Examples	Exactness
Number formats	FP32 to FP16, BF16, FP8; integer widths; ternary; binary; codebook; phase	Bounded (formulas in WP3)
File and tensor formats	SafeTensors and GGUF, per-tensor block layouts, packing and unpacking	Exact for lossless layouts; bounded for quantized
Mathematical representations	Matrix to semiring layer; continuous ODE to a discrete step (explicit Euler, RK4, adaptive RK); linear program to transport form where structure allows; objective in perplexity to objective in cross-entropy	Bounded by discretization order, or exact
Units and domains	Hadamard, Fourier, Givens, Householder, rotor; time and frequency	Exact (orthonormal) up to rounding

M.6 Visual Engine

The engine produces static, deterministic SVG so the same picture can go into the dossier, the paper, and the Studio. The existing Studio live view and chart scripts keep their roles; the engine adds a headless layer they can share.

- **Solver traces.** Error against iteration on a log axis, step size over time, and function evaluations against requested tolerance. These make the adaptive-compute claim of Appendix G visible.

- **Layer and tensor maps.** Heatmaps with layers on one axis and roles or formats on the other, showing precision, measured sensitivity, or allocated format. These are the natural display of the cost tables the search already produces.
- **Geometry and manifolds.** Networks are represented as point clouds or matrix rows in a space of dimension equal to the matrix width or a chosen embedding, anywhere from 3 to 11 or more. Nobody can draw dimensions beyond three, so the engine projects to two or three and always prints the explained variance and the effective dimension. A view that keeps 40% of the variance is shown as such. Rotating the hyperplane through chosen coordinate planes gives a slider-style tour.
- **Dataflow graphs.** Boxes and arrows for how data moves through mixed layers: nodes layered by topological depth, edges labelled with the format at each boundary.
- **Pareto fronts.** Size against quality, with the baseline and random controls marked.

Colors should come from the existing design tokens and pass a color-blindness check. Because design choices are on hold, the engine should read the tokens and never hard-code a palette.

M.7 Integration Notes

1. Add the engine as src/engine/ with a pub re-export in lib.rs, and change no existing behavior in the first pull requests.
2. Wire WP2 behind a solver option on thme search; leave the default unchanged until the exact solver is compared against the current result on committed cost tables.
3. Wrap new codecs as BYOA rules only by editing the open-source enumeration (Appendix L.3); an out-of-process interface comes later if it is needed.
4. Regenerate the dossier from reports only (the repository's measured-claims rule). Engine charts of simulated or toy data must be labelled.
5. The engine uses 64-bit floats as its reference. The repository's tensors are 32-bit, so every conversion test must state which precision it checks.

M.8 Register Additions

ID	Claim	Status	Note
V-94	The existing allocator is a dynamic-programming knapsack solver	C for the part read	First 70 lines use sensitivity thresholds; rest unread; documentation says DP
V-95	Exact DP allocation equals exhaustive search	T	300 random instances, prototype

ID	Claim	Status	Note
V-96	Greedy upgrade allocation can be strictly worse than exact	T	110 against 100 on a constructed instance; never better on 200 random
V-97	Absmax integer quantization error is at most scale / 2	T	Bit widths 2, 4, 8, 12; 20 seeds each
V-98	Mean absolute value is the MSE-optimal scale for sign quantization	T	Compared with 39 alternative scales
V-99	A Walsh-Hadamard transform already exists in the repository	S	Observed in the BYOA transform code; must be shared, not copied
V-100	The prototype is verified for the repository toolchain	U	Compiled on rustc 1.97 only; pinned 1.99.1, MSRV 1.89, fmt and clippy not run

M.9 Open Decisions

- Whether the engine exposes a thme engine subcommand or stays a library used by existing commands.
- Which dimension range the geometry view targets by default, and whether an embedding is chosen per tensor or per layer.
- The Clifford signature to implement (use the one fixed in Section 4 of this paper; confirm it before WP11).
- Whether the engine's reference precision stays 64-bit with conversion at the boundary.
- Which packages to prioritize. My suggestion is WP1 to WP4 and WP9 first, because they are small, independent, and give the visuals quickly.

Appendix N. Positioning: THME as a Bridge Between Existing Techniques

Most of the techniques in this paper already exist, and that is a reason for the project to be open source. THME's role is to bridge them: one intermediate representation, one measured quality gate, and one comparable report format across many tools, plus new tools and engine capabilities where nothing suitable exists. This appendix records that positioning and the rules it implies for the engine of Appendix M.

N.1 What THME Does and Does Not Claim

The contribution is the integration layer and the new tools, not the invention of the underlying methods. Quantization schemes, rotation methods, solvers, and speculative decoding come from other authors and must be credited to them. Every method name and citation in this paper that was written from memory still needs checking against its original source before the paper is shared.

N.2 Technique Catalog

The bridge needs a catalog in which each technique is one entry. Each entry records its identifier, the original source (a verified citation), the licence of the reference implementation, the integration mode, the contract (reconstruct, rotation, or other, as in the BYOA weight contract), the error bound if one exists, the evidence level inside THME, and its status. The integration mode is the main design choice.

Table N1

Integration Modes

Mode	Meaning	Typical use
Native	Small, well-specified method reimplemented in Rust, with a differential test against the reference	Sinkhorn, Runge-Kutta steppers, Walsh-Hadamard, simple quantizers
Wrapped crate	Permissively licensed Rust library used as a dependency	Anything already on the cargo-deny allow-list that does not break binary-size limits
External process	Tool run as a separate program, exchanging files	Large or differently licensed tools; format-compatibility checks against llama.cpp
Format interop	THME reads or writes the other tool's file format	GGUF, SafeTensors
Oracle only	Used in tests as a reference answer, not shipped	SciPy, Wolfram, exhaustive search

N.3 Wrap or Reimplement

Table N2

Decision Rule for a New Technique

Question	If yes	If no
Is the method small (a few hundred lines) and fully specified by a paper?	Native, with an oracle test	Wrap or run externally
Is the licence compatible with AGPL-3.0 and already allowed in deny.toml?	Wrapped crate is possible	External process only, and ask a lawyer first
Would the dependency push the CLI past 10 MB or the Studio past 15 MB (limits in AGENTS.md)?	External process or optional feature	Wrapped crate is possible
Does THME need to enforce a contract on the result (reconstruct, error bound)?	Native or wrapped, so the core can check it	External process is acceptable if the core re-measures the output

A native reimplementation must credit the source, and it should be checked against the reference implementation on fixed inputs. The repository's own pending cross-check of its forward pass against the Hugging Face reference (ticket THME-2001) is an example of this pattern.

N.4 Risks Specific to Bridging

- **Same name, different algorithm.** Tools differ in calibration data, group size, scale convention, and rounding. A '4-bit' result from one tool is not comparable to another's unless THME applies the same gate, the same data splits, and measures size from the written artifact (the rule in Appendix J).
- **Version drift.** Pin external tool versions and record them in every report, as the repository already records seeds and hardware.
- **Licences.** Copying code from an incompatible licence is not allowed; calling a differently licensed tool as a separate program is a different matter, and a lawyer should confirm each case. Model weights carry their own licences, which THME should show in reports.
- **Credit.** Reports should name the original method so that a result is never presented as THME's own invention.

N.5 What May Be New

The items below are candidates for 'new in THME', not established novelty. I have not searched the literature for any of them, and each needs a prior-art check before the paper or repository describes it as new. The research-superpowers literature scout (it queries OpenAlex, Crossref, and arXiv, among others) could run that check.

Table N3

Candidate New Contributions and Their Status

Candidate	Where	Status
Consensus-read mixer with loop-back sampling	Appendix E	Toy simulation only; prior art unchecked
Typed conversion registry with accumulated error bounds	Appendix M.5	Design only; prior art unchecked
Projection views that report retained structure for every manifold picture	Appendix M.6	Design only; prior art unchecked
Proposer loop under a frozen evaluator with lock-one-layer scheduling	Appendix K	Toy schedule comparison; prior art unchecked
Mixed-format allocation across semiring and phase layers	Appendices F, G, I	Analysis only

N.6 Register Additions

ID	Claim	Status	Note
V-101	THME's contribution is an integration layer plus new tools	Design decision	Stated by the owner; guides claims in all appendices
V-102	The candidates in Table N3 are new	U	No prior-art search done
V-103	A native reimplementation matches its reference	U	Required differential test per technique

Appendix O. THME as a Translation Layer: Compatibility, Reverse Engineering, and Rearrangement

The idea is a compatibility layer for models, in the spirit of software that lets programs built for one platform run on another. The corresponding THME capability is a translation layer: load a model from any source, understand its structure, rearrange or convert it, and run or export it for a different runtime, with the ability to reverse engineer undocumented models and to change where weights live and the order of layers. This appendix maps the idea onto the engine, separates what can be done exactly from what cannot, and records one numerical check.

O.1 The Translation-Layer Idea and Where It Breaks

A compatibility layer reimplements the interface that a program expects, so that unmodified programs run on another platform, and it is built from public documentation and observed behavior, without copying the original vendor's code. Such layers often add separate pieces that translate specific subsystems, and they publish compatibility ratings for the programs they have tested.

The idea fits in several places and fails in others. A program calls an interface at run time, so a compatibility layer translates calls as they happen. A model is data plus a graph of operations, so most translation is static: convert once, then run on a runtime that implements every operation in the graph. The part that behaves like a compatibility layer is the runtime. A translation layer can only run architectures whose operations it implements, and the repository's reference forward pass currently covers the LLaMA family and GPT-NeoX (roadmap item v0.2.0.1).

Table O1

Compatibility-Layer Components and Their THME Counterparts

Compatibility-layer component	THME counterpart	Repository today
Program loader	Model loaders for each container format	SafeTensors and GGUF importers exist
Reimplemented interface	Architecture description and operator semantics (norms, attention variants, rotary embeddings, gated MLP, routing)	Five named families plus an unknown case; forward pass for LLaMA and GPT-NeoX
Graphics call translation	Operator and weight-layout translation to a target backend	Exporters exist; GPU backends are deferred to v0.3

Compatibility-layer component	THME counterpart	Repository today
Per-program environment	Per-model workspace with pinned tool versions	Recipes and configuration files
Compatibility ratings	Per model and backend compatibility report built from measurements	Reports exist for two models
Clean-room rule	Reverse engineer from public documentation and observed behavior, never by copying code	Needs a written policy

O.2 What Closed Can Mean

Reverse engineering a model means different things depending on what is closed, and only some cases are possible or advisable.

Table O2

Cases of Closed or Undocumented Models

Case	What you have	What can be done	Guardrail
Open weights, documented architecture	Everything	Direct conversion and rearrangement	Surface the weight licence in reports
Open weights, undocumented or custom architecture	Tensors without a usable configuration	Infer structure from tensor names and shapes; resolve the rest by scored search (O.4)	Verify against any known outputs
Proprietary container or format	Weights in an undocumented layout	Analyze the format for interoperability	Do not defeat encryption or access controls; legal limits vary by country
Restrictive weight licence	Weights whose licence limits derivatives or commercial use	Technically convertible, but the licence still governs	THME can report the licence; it cannot grant permission

Case	What you have	What can be done	Guardrail
Model reachable only through a provider interface	Outputs only; no weights	Black-box evaluation and comparison of outputs you are entitled to produce	Recovering weights is not possible, and copying a model by querying it usually breaches provider terms; out of scope

I am not a lawyer. Rules on reverse engineering for interoperability differ by country and by contract, so a lawyer should review any feature that touches closed formats.

O.3 Rearranging a Model: Exact, Bounded, and Empirical

The conversion registry of Appendix M.5 labels each conversion exact, bounded, or empirical. Rearrangements of a model fall into the same three classes. To check the exact class, I built a small pre-norm transformer in NumPy (3 layers, width 16, 4 heads, feed-forward width 40, RMS normalization with learned weights, gated feed-forward, causal attention; 64-bit floats, one seed) and compared the output of each rearranged copy with the original.

Table O3

Effect of Rearrangements on a Tiny Random Transformer (Output Scale About 8.4)

Operation	Class	Max difference from original
Move weights in storage (reshard, tile, transpose, interleave)	Exact by construction; test for a bit-identical round trip	Not run here
Permute feed-forward hidden units (with matching rows and columns)	Exact	3.6e-15
Permute attention heads (with matching output-projection rows)	Exact	3.6e-15
Fold the normalization weight into the next linear layers	Exact	3.6e-15

Operation	Class	Max difference from original
Rotate the residual stream by an orthogonal matrix, after folding	Exact	1.2e-14
The same rotation without folding the normalization weights first	Not valid	4.7
Swap two layers	Changes the function	8.1
Skip a layer	Changes the function	3.9
Repeat a layer	Changes the function	4.8

The exact rows are the rounding level of 64-bit arithmetic. The last three rows are for random weights and say nothing about trained models, where the effect of reordering is an empirical question for the quality gate. The repository's own measurement is the guide: repeating the middle block of a trained model without retraining raised perplexity by 37.3 (Appendix J).

Limits of this check. The toy model has no rotary embeddings, grouped-query attention, bias terms, tied embeddings, or output head, and uses one seed. Each omitted feature adds a precondition that the real implementation must state and test: rotary pairs must be permuted together, grouped-query heads must be permuted by group, and rotating the residual stream also requires rotating the embedding and output matrices. Symmetries of this kind are described in the literature on rotation-based quantization; the exact method names and citations need verifying.

Exactness is lost in quantized storage. A permutation or rotation is exact in real arithmetic, but a quantizer works on blocks of neighboring values, so moving a weight into a different block changes the block scale and the rounding. After a permutation, quantized weights are therefore bounded, not exact. This is reasoning, not a measurement, and it is the reason rotation can improve quantization when chosen well.

O.4 Reverse Engineering an Undocumented Architecture

Some fields can be read from the tensors and some cannot. Reading them gives a first candidate; scoring candidates gives the rest.

Table O4

What Tensor Shapes Reveal

Property	Identifiable from names and shapes?	How
Number of layers, hidden width, feed-forward width	Yes	Repeated index pattern; matrix dimensions
Gated or ungated feed-forward	Yes	Three matrices against two
Presence of bias terms; normalization type	Yes	Bias tensors; normalization with or without a shift
Grouped-query ratio	Yes	Key and value widths against query width
Number of attention heads	No	Any divisor of the width gives a valid head size
Rotary embedding variant and base, activation function, normalization epsilon, attention scaling, tokenizer	No	Not stored in tensors

The unidentifiable fields need metadata or a scored search: run each candidate configuration on a fixed text, measure the loss, and prefer the lowest. A search needs an explicit candidate set, a seed, and a budget, and should report a confidence label with the answer, because two settings can score the same on short text. The repository already carries norm epsilon and the rotary base through GGUF metadata, which is the route when metadata exists.

O.5 Compatibility Reports

A compatibility rating should come from measurements, not opinion. Each rung below needs a committed report before it can be claimed.

1. **Loads.** All tensors parsed; no unknown tensor.
2. **Runs.** Shapes consistent; the forward pass completes.
3. **Agrees.** Logits match a reference implementation on fixed prompts within a stated tolerance.
4. **Passes the gate.** The named transformation passes the quality gate on held-out text.
5. **Measured speed.** Throughput and memory measured on named hardware.

O.6 Work Packages

These extend the list in Appendix M.4 and follow the same conventions.

Table O5

Translation-Layer Work Packages

WP	Module	Build	Oracle tests	Done means
WP14	engine::graph	Architecture-neutral graph of operations with shape inference: norms, linear layers, attention variants, rotary embeddings, gated feed-forward, routing	Reproduces a NumPy forward pass of a tiny random model in two architecture styles (LLaMA-style and GPT-NeoX-style)	A model described in the graph runs the same as the existing reference forward pass
WP15	engine::symmetry	Exact transformations of Table O3 plus the preconditions of O.3 (rotary pairs, grouped-query groups, embeddings, output head, tied weights)	Logit equality at 1e-10 in 64-bit floats; negative controls fail as expected; a measured effect on quantization error	Every transformation reports exact, bounded, or empirical, with its test
WP16	engine::layout	Weight placement planner (shard, tile, interleave, transpose, block alignment); layer reorder, skip, and repeat flagged empirical	Bit-identical round trip for placement; reorder operations refuse to run without a gate	Placement round-trips exactly; reorder needs a gate report
WP17	engine::infer	Architecture inference from tensors with the identifiable and scored fields of O.4	Mock models with known truth first; then real models with their configuration withheld	Recovers the configuration of known models, with a confidence label on every scored field

WP	Module	Build	Oracle tests	Done means
WP18	report schema	Compatibility ladder of O.5 as a JSON schema	Schema validation; no rung without a linked report	The Studio and dossier show the rung reached per model and backend

Suggested order: WP14, then WP15 and WP16 in parallel, then WP17, then WP18. The symmetry package is also the proper home for the rotation already used inside the quantization rule, which Appendix M.2 says should be shared, not duplicated.

O.7 Guardrails

- Build from public documentation and from behavior observed on copies you are entitled to use; never copy code from a product whose licence forbids it.
- Do not circumvent encryption, licence checks, or other access controls.
- Show the weight licence in every report and never imply that a conversion is permitted by it.
- Model extraction through a provider interface is out of scope.
- Record the provenance of every conversion (tool versions, source file hashes) in the report, as the repository already does for measured runs.

O.8 Register Additions

ID	Claim	Status	Note
V-104	Permuting feed-forward units or heads, and folding normalization weights, leave the output unchanged	T	Tiny random model, 64-bit, one seed; differences about 4e-15
V-105	Rotating the residual stream is exact only after the normalization weights are folded	T	Folded: 1.2e-14; unfolded: 4.7
V-106	Layer swap, skip, and repeat change the output	T	Random weights; says nothing about trained models
V-107	The attention head count cannot be read from tensor shapes	S	Any divisor of the hidden width is consistent with the shapes
V-108	Permutations and rotations are inexact after block quantization	U	Reasoned, not measured

ID	Claim	Status	Note
V-109	The repository's forward pass covers LLaMA-family and GPT-NeoX models only	S	Roadmap item v0.2.0.1
V-110	Symmetries hold with rotary embeddings, grouped-query attention, biases, and tied embeddings	U	Not in the toy; each needs its own test

O.9 Open Decisions

- Which architectures to add to the graph first (the roadmap lists mixture-of-experts and vision-language models among the paradigms).
- Whether to support loading from PyTorch pickles and ONNX, which carry code or opaque graphs and need extra care.
- How far to go on closed container formats, and who reviews the legal position.

Appendix P. Permission Rule, Tunable Gates, Routers, and Injected Needles

This appendix does two things. It adds a permission and ownership rule to the reverse-engineering features of Appendix O, and it specifies how gates are changed and tuned, how routing is edited, and how very small decision models ('needles') are injected into a model to route computation.

P.1 Permission and Ownership Rule (Amends O.2 and O.7)

Any reverse-engineering or architecture-inference feature, and any analysis of a closed container format, should work only on a model that the person

1. owns, meaning they trained it or hold the rights; or
2. has permission to analyze and modify, from the rights holder or under a licence that allows it; or
3. may use under an open licence whose terms allow this use.

Educational and research use is a supported purpose under the same rule. Educational purpose does not replace permission: whether analysis of a model is lawful for teaching depends on its licence and on the country, so the rule applies to students and teachers as it does to companies. Models reachable only through a provider interface stay out of scope (O.2).

Implementation proposal:

- Features that infer architecture or analyze closed formats require the person to state the basis (owner, permission, or open licence) and the purpose (research, education, or commercial). The statement is stored in the report next to the model hash.
- The tool reads the licence field from the model's metadata when it exists and warns when the licence looks restrictive.
- Unattended runs refuse to start without the stored statement.
- A teaching mode defaults to the synthetic mock models the repository already generates and to openly licensed small models, whose licences should be checked before use.

The tool cannot verify the statement. It is a record of accountability and a statement of policy, not a legal guarantee, and the wording of the terms should come from a lawyer.

P.2 Four Things Called a Gate

The word gate already means several things in this project, and mixing them causes errors. The names below are proposed for code, documents, and the Studio.

Table P1

Proposed Vocabulary

Term	Meaning	Where it lives
Quality gate	Pass or fail test on a candidate model against a baseline	configs/quality_gates.yaml; the quality module
Gate unit	A gating function inside the model, such as the gate of a gated feed-forward layer or a memory write gate	Model weights
Router	A function that selects among paths: experts, mixers, precision paths, exit points	Model or injected
Needle	A very small model that makes a routing or decision call for a frozen model	Injected (P.5)

P.3 Changing and Tuning Quality Gates

The quality gate is the part you can change most safely, because it only describes what counts as a pass. The repository already keeps tolerances in a configuration file. Three things should be added so that changing a gate stays honest.

- **Gates as versioned data.** Each gate has a version and a content hash, and every report records both. Changing a threshold creates a new version. A result produced under one version is never re-labelled under another.
- **Absolute and relative margins together** (the point of Appendix J.4b), and confidence intervals from a bootstrap over tokens.
- **No tuning after seeing results.** Gate changes that follow a failed candidate must be logged as such, because moving a goal post after the result biases the search (Appendix H.5, K.2).

An inconsistency to resolve. The tolerances file lists a perplexity increase of 0.25, and the unit tests in the quality module use 0.25, while the measured reports in the benchmark dossier use 0.45. The file also lists accuracy gates (MMLU and GSM8K) for which the repository has measured nothing. I did not trace which value the command line uses by default, so this needs checking.

P.4 Model Gates and Routers

Editing a gate unit or a router means moving a decision boundary, and the knobs differ in how much they can change the model.

Table P2

Decision Points in a Model and the Knobs on Them

Decision point	What it decides	Repository today	Knobs
Gate of a gated feed-forward layer	How much of each hidden unit passes	In the weights	Scale (edits the function; needs the quality gate)
Expert router	Which experts a token uses	Mixture-of-experts paradigm with a top-k setting	Top-k, softmax temperature, bias, load-balance penalty
Early-exit gate	Where a token stops	ConfigureEarlyExit operation; untrained exit measured weak	Threshold, target exit ratio, trained readouts

Decision point	What it decides	Repository today	Knobs
Mixer choice	Which sequence mixer a layer uses	ReplaceMixer, fixed per layer	Per-token routing is absent
Precision path	Which format path computes a token	Absent; related to Appendix E	Path set and routing rule
Speculative acceptance	Whether a draft token is kept	Speculative module (speedups estimated)	Draft length, acceptance rule (Appendix J.5)

Tuning knobs that only shift thresholds, such as temperature or top-k, are cheap to try and can be measured by routing statistics (entropy, load balance, path usage) as well as by the quality gate.

P.5 Injected Needles

The repository's findings note records that the earlier 'tiny needles' concept was drafted without repository access and is not implemented. This section turns it into a specification that can be tested.

Definition. A needle is a small network that takes a hidden state at a decision point and outputs a distribution over K paths: continue as usual, skip, take the low-precision path, exit, or pick an expert. It is attached to a frozen model and trained with the model's weights held fixed.

Cost. A single linear needle from width d to K paths adds d times K weights. A standard transformer block has about 12 times d squared weights, so the ratio is K divided by 12d. As an illustration at width 576 and K = 4, the ratio is 0.058%. A two-layer needle with hidden width 32 at the same width adds about 18,560 weights, 0.47% of a block. These are order-of-magnitude figures, since real blocks (grouped-query attention, SwiGLU) differ. Each needle's compute per token is about its weight count in multiply-adds.

Break-even. Let rho be the needle's cost as a fraction of the block, c the relative cost of the cheap path (zero for a skip), and p the share of tokens sent down it. Expected cost is rho + p c + (1 - p), so the needle saves compute when

p (1 - c) > rho (P1)

With rho of 0.5% and a skip, a needle pays for itself once 0.5% of tokens skip. So the compute overhead is rarely the limit. The limit is quality: a wrong call costs accuracy. To first order, the expected loss in cross-entropy is the misroute rate times the cost per

misroute, summed over decision points. Appendix J.4 shows interactions make this sum an underestimate, so the whole routed model must be measured.

Injection that preserves the model. A residual form, $x + g(x)\, f(x)$, with the gate initialised so that g is 1 on the usual path, reproduces the original model exactly at the start. Training then moves away from the original only where the needle says so. This is true by construction and is a test to write (H-P3), not yet a result.

Training without a backward pass through the model. One route needs no gradient through the language model. Collect hidden states at a decision point on calibration text, label each by whether the cheap path's output stays within a tolerance of the full path's output (the teacher label), and train the needle as an ordinary classifier. This needs forward passes plus a tiny trainer, so it does not wait for the backward pass in the roadmap (v0.2.3.1). Gradient-based alternatives (straight-through estimators, relaxed sampling, policy gradient) can follow. Use the three splits of Appendix K.2, because a needle can overfit its calibration text.

Decision as a trade-off. The needle's job is the per-token version of what the allocator does per layer: minimise quality loss plus a price times compute. The price is the allocator's shadow price from Appendix F, so one number links the two. The measured weakness of untrained exit (1.6% of tokens settled at 30% depth, Appendix J) is the baseline a trained needle must beat.

P.6 Hypotheses

Table P3

Hypotheses for Gates and Needles (Proposed)

ID	Hypothesis	Falsified if
H-P1	A needle under 1% of a block's weights can predict which tokens may skip a layer	Held-out AUC at or below 0.7
H-P2	Needle routing meets the quality gate at lower compute than static layer skipping at equal average compute	Static skipping matches or beats it over at least 5 seeds
H-P3	Identity-initialised injection leaves the model unchanged	Logit difference above 1e-6 in 32-bit arithmetic before any training
H-P4	Moving a router knob (temperature, top-k) changes quality smoothly	Quality is non-monotone over the tested range

ID	Hypothesis	Falsified if
H-P5	Teacher-label needles trained without gradients through the model match gradient-trained needles	Gradient-trained needles beat them by more than the bootstrap noise

P.7 Work Packages

These continue the numbering of Appendices M and O.

Table P4

Permission, Gate, and Router Work Packages

WP	Module	Build	Oracle tests	Done means
WP19	permission	Basis and purpose flags; report fields; refusal when absent; licence-field warning	Refuses without a stored statement; report contains the statement and model hash	Reverse-engineering features cannot run without the record
WP20	quality gate	Gate as versioned data with a hash; absolute and relative margins; reconcile 0.25 against 0.45	Same gate gives the same verdict; a threshold change changes the hash	Every report states its gate version
WP21	engine::route	Router abstraction (decision points, paths, hard and soft modes); identity-initialised injection into the WP14 graph; routing statistics	Output equals the original at initialisation; top-k equal to all paths equals the dense result	Routing statistics shown in reports
WP22	engine::needle	Teacher-label calibration on cached hidden states; small classifier trainer; three splits; ledger entries	Synthetic separable data reaches expected accuracy; held-out gap reported	A needle trained on one model reports AUC, size ratio, and break-even

WP	Module	Build	Oracle tests	Done means
WP23	visual	Routing heatmap (token by layer), path usage, decision-confidence histogram	Element counts match data; deterministic output	Studio and dossier can embed the pictures

P.8 Register Additions

ID	Claim	Status	Note
V-111	The repository's gate tolerances are consistent	C	0.25 in the tolerances file and tests; 0.45 in measured reports; default not traced
V-112	Tiny needles exist in the repository	C	The repository's findings note says the concept is not implemented
V-113	A linear needle costs K / 12d of a block	S	Arithmetic; order of magnitude
V-114	A needle saves compute when p (1 - c) exceeds rho	S	Derivation, Equation P1
V-115	Identity-initialised injection preserves the model	U	True by construction; not yet tested
V-116	Teacher-label needles need no backward pass through the model	S	Reasoning; to confirm in H-P5
V-117	Educational purpose removes the permission requirement	C	Policy is the opposite (P.1); legal effect varies

P.9 Open Items

- Which decision points to start with: layer skipping, precision path, or expert routing.
- Whether the permission statement is a command-line flag, a prompt in the Studio, or both.

P.10 Calibrated, Typed Needles and Many-Option Routing

Three properties make a needle usable for routing, and each is testable.

1. **Typed output.** A needle emits one value from a closed set of paths, checked against a schema, so a malformed decision cannot occur.
2. **Calibrated confidence.** The repository's early-exit operation routes on a confidence threshold, so the confidence must mean what it says: among decisions made at 0.9, about 90% should be right. Calibration is measured by expected calibration error (the average gap between stated confidence and observed accuracy across confidence bins) and by the Brier score, and it can often be repaired after training by fitting one temperature parameter on a held-out split.
3. **Single-pass scoring.** One forward pass of a tiny network gives every score at once; no sequence is generated.

When a decision has many options, scoring K options flat costs K scores. Splitting them into g groups and scoring the group first and then the option inside it costs g + K / g, which is smallest at g equal to the square root of K, giving a cost near 2 times the square root of K. For K = 255 that is about 32 scores instead of 255. The price is that a wrong group choice cannot be undone, so the accuracy loss must be measured against flat scoring.

Table P5

Calibration and Hierarchical Routing (Proposed)

ID	Item	Oracle or falsifier
WP24	engine::calib: expected calibration error, Brier score, reliability bins, temperature scaling	Perfectly calibrated synthetic data gives error near 0 as sample size grows; over-confident synthetic data is repaired by temperature scaling; Brier score matches a hand calculation
WP25	Two-stage hierarchical scoring beside flat scoring	Exact agreement with flat scoring when groups are consistent; measured gap otherwise
H-P6	A trained needle can be calibrated well enough for threshold routing	Held-out expected calibration error stays above 0.10 after temperature fitting
H-P7	Two-stage scoring keeps accuracy within 1 point of flat scoring at about 2 times the square root of K cost	Accuracy drop above 1 point on held-out data

Build WP24 before the needle trainer (WP22) reports any result, because a needle without a calibration number cannot be used for threshold routing.

ID	Claim	Status	Note
V-118	Two-stage scoring has minimum cost near 2 times the square root of K	S	Derivation; K = 255 gives about 32
V-119	Threshold routing needs calibrated confidence	S	Reasoning; the early-exit operation uses a confidence threshold
V-120	Calibration measured on the tune split transfers to the final split	U	Part of H-P6; to test

Appendix Q. Teacher-Led Distillation and Autonomous Training

This appendix records the product direction the owner described: you hold a large frontier model, you want a smaller model that behaves like it as closely as its size allows, and you want the large model to teach the small one, to choose the training algorithm and data, and to run the trial-and-error loop with limited supervision. The small model can then be trained on further datasets or converted to a different architecture. This is the use of BYOA, trial and error, and looping in the earlier appendices. It replaces the idea of a standalone decision function, which is not pursued.

Q.1 Scope and Pipeline

Table Q1

Pipeline Stages and the Evidence Each Needs

Stage	What happens	Evidence before moving on
S0 Student design	Choose size and architecture, possibly a hybrid or converted one	Parameter and memory estimate; the size formula and estimators in Appendix I
S1 Data	Collect, license-check, and split data (train, tune, select, final)	Splits disjoint; licences recorded; contamination check
S2 Teacher outputs	Run the teacher over the data and store what the student will learn from	Coverage report (Q.3); checksum; teacher version recorded
S3 Distill	Train the student against the teacher	Training loss falling; no use of the final split

Stage	What happens	Evidence before moving on
S4 Convert	Quantize or change the architecture (Appendices D to G, M, O)	Quality gate on held-out data
S5 Evaluate	Compare student with teacher and with the baseline on slices	Fidelity report (Q.4) with intervals
S6 Iterate	The autonomous loop proposes the next recipe (Q.5)	Ledger entry, budget left, stopping rule

Q.2 What Already Exists in the Repository

From my read of the code and roadmap, which did not include running anything: the micro-trainer lists cross-entropy, knowledge distillation against a teacher, and an energy-regularized loss; the BYOA categories include feature distillation; the surgery operations include paradigm conversion and architecture rebuilding; and the roadmap schedules quantization-aware distillation, recovery adapters, teacher-assisted calibration data, and an overfitting guard that keeps recovery data disjoint from evaluation data (v0.2.3). The roadmap also lists a backward pass for the reference model as open, with the alternative of a bridge to an external trainer. A repository that bridges existing tools (Appendix N) should prefer the bridge for anything beyond tiny models, using the existing PyTorch interoperability code, and keep the native trainer for small experiments and tests.

Q.3 Distillation Methods

The names below come from memory and need checking against their original papers.

Table Q2

Distillation Methods and Their Trade-Offs

Method	What the student learns from	Strength	Weakness
Logit distillation	The teacher's full next-token distribution at each position	Richest signal per token	Needs matching tokenizers; large storage
Top-k logit distillation	The k most likely tokens and their probabilities	Cheap to store and replay	Loses the tail (Equation Q2)

Method	What the student learns from	Strength	Weakness
Sequence-level distillation	Text generated by the teacher	Works across tokenizers and architectures	Only the sampled path; less signal per token
On-policy distillation	The student's own samples, scored by the teacher	Trains on the student's own mistakes	Teacher must be queried during training
Feature distillation	Hidden states or attention outputs of chosen teacher layers	Guides a converted architecture layer by layer	Needs a layer mapping; width mismatch needs a projection
Synthetic data and curriculum	Teacher-written examples, ordered by difficulty	Targets weak slices	Can copy the teacher's errors; contamination risk

The standard soft-target loss mixes the teacher distribution with the true labels:

L = a T^2 KL(p_T(tau) || p_S(tau)) + (1 - a) CE(y, p_S) (Q1)

where p(tau) is a softmax at temperature tau, a weights the two terms, and the factor T^2 keeps gradient size comparable across temperatures. This form is from memory; the exact definition should be checked.

Top-k cache. Storing only the top k teacher probabilities for each position, renormalised, is a common saving. Let m_k be the probability mass of the stored tokens, q the renormalised top-k distribution, and p the full teacher distribution. Then

KL(q || p) = -ln(m_k) (Q2)

I confirmed this identity numerically: the largest difference over 200 random distributions on a 50,000-token vocabulary was 9e-16. It says the information lost by truncation depends only on the covered mass, so a coverage report is the right check: if the stored tokens hold 95% of the mass, the loss is 0.051 nats per position. How much mass k tokens cover depends on how peaked the teacher is, so it must be measured on the real teacher. On synthetic flat logits (standard deviation 2) 64 tokens cover only 19%.

Storage. One entry costs k times (index bytes + probability bytes). With k = 64, 2-byte indices (enough for a vocabulary under 65,536) and 2-byte probabilities, a position costs 256 bytes and a billion positions cost 256 GB. The teacher's compute to produce those outputs is usually the larger cost, so cache once and reuse.

Q.4 Measuring How Closely the Student Acts Like the Teacher

Fidelity needs several numbers, not one: the KL divergence from teacher to student on held-out text, the share of positions where both pick the same top token, the perplexity gap, and task scores on named slices (for example code, mathematics, long context, and the languages you care about). Report each with a bootstrap interval. A small model cannot match a large one everywhere, so track the capacity gap per slice, and fit the improvement curve of Appendix F, a_n = a_inf - c rho^n, treating a_inf below the teacher's score as the expected case.

Q.5 The Teacher as Autonomous Trainer

The loop of Appendix K applies directly, with the teacher as proposer. Its search space is the training recipe: data mixture weights, temperature, loss weights, learning rate and schedule, layer mapping for feature distillation, student architecture edits, and BYOA rules. Four rules keep it honest and bounded.

- **The teacher does not grade alone.** Scores come from the frozen evaluator on splits the teacher never touches, using tasks the teacher did not write. A student can look excellent on teacher-written tests simply because it copied the teacher's habits.
- **Budgets are hard limits.** Caps on tokens, compute time, and money. A recipe that would exceed the remaining budget is refused, and spending above a threshold needs human approval.
- **Generated code is not run on trust.** Anything the teacher proposes that is more than data (new loss, new rule) goes through the declarative BYOA path or a sandboxed process (Appendix L.4) and is reviewed before its first run.
- **Every step is logged** in the ledger with the recipe, its cost, and its scores on every split, so the multiple-comparison correction of Appendix H.5 can be applied.

The teacher-led proposer is worth its cost only if it beats random search and a standard hyperparameter optimizer at the same budget (H-K1), which is untested.

Q.6 Guardrails

- The teacher must be a model you own or have permission to use (Appendix P.1). Distilling a model you do not own by querying it is out of scope.
- Training data licences and sources are recorded in every report.
- Contamination check: no evaluation item appears in training data or in teacher-written data.
- The student inherits the teacher's licence conditions where they apply; record both.

Q.7 Hypotheses

Table Q3

Distillation Hypotheses (Proposed)

ID	Hypothesis	Falsified if
H-Q1	A top-k cache with k near 64 covers enough mass on the real teacher	Median covered mass below 0.95 on held-out text
H-Q2	On-policy distillation beats off-policy at equal teacher tokens	Off-policy matches or beats it over at least 5 seeds
H-Q3	Adding feature distillation improves a converted architecture over logit distillation alone	No gain beyond bootstrap noise on the gate
H-Q4	Student fidelity approaches a limit below the teacher's score geometrically	A fit with a_inf at or above the teacher, or a poor fit, over at least 5 seeds
H-Q5	The teacher-led recipe search reaches a target fidelity with less budget than random search	Same or higher budget needed at equal seeds (this is H-K1 applied here)

Q.8 Work Packages

These continue the numbering of Appendices M, O, and P.

Table Q4

Distillation Work Packages

WP	Module	Build	Oracle tests	Done means
WP26	teacher cache	Writer and reader for top-k teacher outputs with a coverage report, checksum, and teacher version	Round trip is exact; reported mass matches direct computation; KL against the full distribution equals Equation Q2	Every cache states its covered mass

WP	Module	Build	Oracle tests	Done means
WP27	distillation losses	Soft-target, top-k, sequence-level, and feature losses	KL is zero only for equal distributions and never negative; analytic gradient matches finite differences; Equation Q2 holds	Losses usable by the micro-trainer and the external-trainer bridge
WP28	fidelity report	KL, top-1 agreement, perplexity gap, slice scores, all with intervals	Known synthetic teacher and student give known values	Report included in the dossier generator
WP29	autonomy controller	Budgets, approval thresholds, sandboxing rules, ledger, stopping rules	Refuses over-budget recipes; refuses unreviewed code; log is append-only	Loop cannot run without all four
WP30	student planner	Size and memory estimate for a candidate student architecture	Parameter counts match the actual model for known architectures	Planner output feeds S0

Q.9 Register Additions

ID	Claim	Status	Note
V-121	KL(q		p) equals minus the log of the top-k mass for renormalised top-k
V-122	A top-64 cache with 2-byte index and probability costs 256 bytes per position	S	Arithmetic; 256 GB per billion positions
V-123	The soft-target loss has the form of Equation Q1	U	From memory; check the original definition
V-124	The repository already has a distillation loss type and a feature-distillation category	S	Observed in the trainer and BYOA code; not run
V-125	A teacher-led recipe search beats random search	U	No test

ID	Claim	Status	Note
V-126	A student reaches a stated fraction of teacher quality	U	No student has been trained

Q.10 Open Decisions

- The teacher: its tokenizer, size, and whether its outputs can be served in bulk. A shared tokenizer allows logit distillation; otherwise only sequence-level distillation works.
- The target student size and whether it keeps the teacher's architecture or converts to a hybrid.
- The training compute available, and whether training runs inside THME or through the external-trainer bridge.
- The budget caps and the approval threshold for the autonomous loop.

Appendix R. Benchmarking Subsystem: Running Benchmarks Inside THME and Graphing the Scores

The owner's direction is to focus on tools, to include every type of benchmark test, and to run the benchmarks inside the software and graph their scores. This appendix specifies that subsystem. It was shaped by a reference image of a leaderboard-style dashboard that the owner supplied for orientation only. The design features taken from it are listed in R.1; its content, branding, and numbers are not used, and the visual style follows THME's own design tokens (which are on hold).

R.1 Design Principles

- **Scores by category, not one number.** Each model shows a score per capability family. A single composite, if shown at all, is secondary and its weights are written down.
- **Every score carries its uncertainty.** An interval is shown beside the point estimate, and a result whose interval is wide is labelled so.
- **Every score carries its evidence status.** Measured in THME, imported from another tool, estimated, or simulated. This extends the four evidence levels the repository already uses.
- **Not measured is not zero.** The display distinguishes not measured, pending, failed to run, and a real score of zero. Averages never treat a missing score as zero or fill it in.
- **Every view states its date and method version.** A score without the harness version, benchmark version, and run date cannot be compared with another.

- **Cost sits beside quality.** For THME the cost is model size, latency, memory, and energy, shown next to the score.

R.2 What the Repository Has Now

From my read of the code, without running it: a gate that compares a baseline score with a candidate score per named task against an allowed degradation, with a lower-is-better flag; an enumeration of evaluation corpora for perplexity (WikiText-2, C4, Penn Treebank, synthetic, custom); a bridge that runs an external evaluator command and parses a perplexity from its output; a tolerances file that lists accuracy tasks (MMLU, GSM8K); and a Python script for throughput and profiling telemetry whose header also mentions token-generation simulation. The task-score structure holds one number per side. It has no item count, no per-item results, and no interval, so the gate cannot tell a strong result from a noisy one. The roadmap and dossier report measured perplexity only.

R.3 Benchmark Families

Benchmark names below are from memory and should be checked against their sources before use.

Table R1

Benchmark Families to Support

Family	Examples	Metric	Needs	Notes
Language modeling	WikiText-2, C4, Penn Treebank	Cross-entropy, perplexity	Corpus, tokenizer	Exists; report cross-entropy as well (Appendix J.4)
Knowledge and multiple choice	MMLU, ARC, HellaSwag-style sets	Accuracy	Log-likelihood or generation scoring	Prompt format changes scores
Math and reasoning with a checkable answer	GSM8K, MATH-style sets	Exact match after answer extraction	Decoding settings, answer parser	Sensitive to extraction rules
Code	HumanEval, MBPP-style sets	pass@k from unit tests	Sandbox that runs untrusted code	Needs an isolated process

Family	Examples	Metric	Needs	Notes
Instruction following	Checkable-constraint sets	Share of constraints met	Programmatic checkers	
Long context	Retrieval-in-context, long-document questions, context sweeps	Accuracy against length	Long inputs; KV measurements	Tests the cache claims of the paper
Multilingual	Translated versions of the above	Accuracy per language	Per-language data	Report per language, not one average
Truthfulness and safety	Truthfulness and refusal sets	Task-specific	Careful design and review	Easy to misuse as a single number
Agentic tasks	Multi-step tasks with tools	Success rate	An environment; high cost	Add last
Multimodal	Image-text question sets	Accuracy	Vision paradigm support	Depends on the vision-language paradigm
Fidelity to a teacher	Held-out text	KL, top-1 agreement (Appendix Q.4)	Teacher outputs	Specific to THME
Calibration	Any labelled set	Expected calibration error, Brier score	Confidence outputs (Appendix P.10)	
System performance	Fixed prompts and loads	Throughput, latency percentiles, peak memory, KV bytes, energy	Named hardware, repeats	Roadmap v0.2.4

Family	Examples	Metric	Needs	Notes
Engine correctness	Analytic and brute-force oracles	Error against stated bound	None	Appendix M; the engine is benchmarked too

R.4 Harness Design

Benchmarks run inside THME through one registry and one result format, whatever the source of the numbers.

- **Task registry as data.** A benchmark is a declarative file: dataset path and hash, split, prompt template, answer-extraction rule, and metric. Users bring their own benchmarks the way they bring their own algorithms. Metrics come from a fixed library (exact match, normalized match, numeric tolerance, multiple-choice by likelihood, unit-test pass); a custom metric that is more than data runs in a sandboxed process.
- **Three run modes**, always labelled in the result: native (THME's own forward pass, slow but self-contained, limited to supported architectures), bridged (an existing evaluation tool run as an external process, as the repository already does for perplexity), and imported (numbers supplied by the user, labelled as supplied and never as measured).
- **Per-item results are stored.** Item identifier, model output, and correct or not. This allows interval estimates, paired comparisons, and re-aggregation later without re-running.
- **Determinism.** Greedy decoding by default; any sampling states its temperature and seed; results record the harness version, dataset hash, prompt-template hash, decoding settings, model hash, transformation identifier, and hardware.
- **Resumable and cached.** A run interrupted at item 3,000 resumes at item 3,001, and an unchanged (model, task, settings) combination is not run twice.
- **Safe execution.** Code from models runs in a separate process with no network, file, or time privileges beyond fixed limits.
- **Harness self-tests.** A trivial model that always answers the first option must score near chance, and a lookup model that knows the answers must score 1.0. A harness that fails these is not trusted with a real model.

Table R2

Fields of a Benchmark Result Record

Field	Purpose
Benchmark id, version, dataset hash, split	Which test exactly
Model hash, transformation id	Which model and which conversion
Harness version, run mode, prompt-template hash	How it was run
Decoding settings, seed	Reproducibility
Item count; per-item results	Intervals and paired tests
Score, interval, interval method	The result with its uncertainty
Evidence status	Measured, imported, estimated, simulated
Hardware, run date, duration	Context
Dataset licence	Compliance (Appendix N.4)

R.5 Statistics

Three facts shape the design, and I checked each numerically.

Small test sets cannot resolve small differences. The 95% interval half-width on an accuracy of 0.70 is 7.0 points for 164 items, 2.8 points for 1,000, and 0.76 points for 14,000. To resolve a one-point difference between two models at 95% confidence, independent runs need about 16,100 items (accuracy 0.70). A paired comparison on the same items, where 5% of items change between the two models, needs about 1,900, which is 8.4 times fewer. So bench results must store per-item outcomes and use paired tests. The tolerances file asks for a one-point accuracy gate; this arithmetic shows that gate is only decidable on large sets.

Use a better interval than the simple one. I simulated coverage of nominally 95% intervals over 4,000 repeats. At 30 items and true accuracy 0.9, the simple normal-approximation interval covered the truth only 80.1% of the time, the Wilson interval 97.0%, and a percentile bootstrap 95.0%. At 200 items and 1,000 items all three were within about one point of nominal. So Wilson (for a single proportion) or a bootstrap (for any score) should be the default, and the simple interval avoided.

Pass@k has an unbiased estimator. For n samples with c correct, pass@k = 1 - C(n - c, k) / C(n, k). I checked this against full enumeration for n = 6 and c = 2: 0.333, 0.600, 0.800 for k = 1, 2, 3, equal to brute force. The formula is from memory and the source should be checked.

Table R3

Oracle Values for the Statistics Library

Computation	Expected value
Wilson interval, 7 correct of 10	0.397 to 0.892
pass@1, pass@2, pass@3 for n = 6, c = 2	0.333, 0.600, 0.800
Exact McNemar test, 12 items improved and 3 regressed	two-sided p = 0.035
Half-width at accuracy 0.70, 164 items	7.0 points
Percentile bootstrap coverage, 200 items, accuracy 0.70	about 0.95

A three-state gate. Let D be the measured drop from baseline to candidate with a confidence interval from low to high, and T the allowed drop. Pass only if the high end is at most T; fail if the low end exceeds T; otherwise inconclusive. A point-estimate gate passes a candidate whose measured drop is under T even when the interval says it could be several times T. The three-state rule makes small test sets report inconclusive instead of passing by luck. Apply a correction across tasks (Appendix H.5).

R.6 Aggregation and Comparison

Category scores are means of normalized benchmark scores with stated weights, and they show coverage (how many of the category's benchmarks were measured). A model with a missing benchmark is shown with a coverage note, not with an average over fewer items presented as comparable. Two models are comparable only if the benchmark version, harness version, prompt template, and decoding settings match. A rank is shown only where intervals separate; otherwise the display says tied.

R.7 Graphs

All charts come from stored reports through the visual engine of Appendix M.6, are deterministic, and carry the evidence status.

Table R4

Charts for Benchmark Results

Chart	Shows	Notes
Category dot plot	Score and interval per category and model	Preferred over radar polygons, whose area depends on axis order
Delta plot	Candidate minus baseline with interval and tolerance line	Colored by pass, fail, inconclusive
Model by benchmark heatmap	Scores across a grid	Missing cells hatched, never colored as zero
Quality against cost	Score against size, latency, memory, or energy; Pareto front marked	Baseline and random controls included
Score against context length	Accuracy and KV bytes across a sweep	Tests the memory claims
Score against iteration	Fidelity or accuracy across tuning or training steps, with the fitted limit	Appendices F and K
Per-item agreement	Student against teacher outcomes on each item	Shows where the student diverges
Reliability diagram	Stated confidence against observed accuracy	Appendix P.10
Status table	Each score with its evidence chip and run date	Chips: measured, imported, estimated, simulated, not measured, pending

R.8 System Benchmarks

Throughput (prompt processing and generation separately), latency percentiles, peak memory, KV-cache bytes, and energy are measured with a stated method: warm-up runs, at least five repeats, the median with an interval, named hardware, and a note on frequency scaling and thermal state. Energy is reported only where hardware counters exist. Figures from analytic models are labelled estimates, and any telemetry from simulated token generation is labelled simulated.

R.9 Work Packages

These continue the numbering of Appendices M, O, P, and Q.

Table R5

Benchmarking Work Packages

WP	Module	Build	Oracle tests	Done means
WP31	bench registry	Declarative task files, result record (Table R2), schema validation	Malformed files are refused; a record round-trips	Any task file runs through the same path
WP32	bench runner	Native, bridged, and imported modes; resume; cache; sandbox for code	Harness self-tests (chance and oracle models); resumed run equals a single run	Run mode appears in every result
WP33	bench metrics	Exact match, normalized match, numeric tolerance, likelihood choice, pass@k	Hand-computed cases; pass@k against enumeration	Table R3 values reproduced
WP34	bench stats	Wilson, bootstrap, paired bootstrap, exact McNemar, required-items calculator, multiple-comparison correction	Table R3 values; simulated coverage near nominal	Every score carries an interval
WP35	three-state gate	Pass, fail, inconclusive from the interval; versioned with the gate (Appendix P.3)	Synthetic cases with known verdicts	Small sets report inconclusive
WP36	contamination check	Overlap between benchmark items and training or teacher data	Planted overlaps are found	Report lists overlaps before a score is shown
WP37	bench charts	The charts of Table R4 on the visual engine	Element counts match data; missing cells hatched	Dashboard shows them for any stored report

WP	Module	Build	Oracle tests	Done means
WP38	system bench	Throughput, latency, memory, KV bytes, energy with the method of R.8	Repeat variation reported; known synthetic load gives expected figures	Roadmap v0.2.4 exit criterion
WP39	engine bench	Error against bound for conversions, solvers, allocators	Same oracles as Appendix M	Run on every release

R.10 Hypotheses

Table R6

Benchmarking Hypotheses (Proposed)

ID	Hypothesis	Falsified if
H-R1	The three-state gate produces fewer false passes than the point-estimate gate	On simulated candidate pairs with known true drops, it does not
H-R2	Stored per-item results with paired tests let the existing candidates be resolved with far fewer items	Paired and unpaired requirements differ by less than 2 times on real pairs
H-R3	Harness self-tests catch scoring bugs before real runs	A planted scoring bug passes the self-tests

R.11 Register Additions

ID	Claim	Status	Note
V-127	The Wilson interval for 7 of 10 is 0.397 to 0.892	S	Computed
V-128	The simple interval undercovers at small n: 80.1% at 30 items, accuracy 0.9; Wilson 97.0%; bootstrap 95.0%	T	4,000 repeats, one seed
V-129	Half-widths at accuracy 0.70 are 7.0, 2.8, and 0.76 points for 164, 1,000, and 14,000 items	S	Arithmetic

ID	Claim	Status	Note
V-130	Resolving one point needs about 16,100 items unpaired and 1,900 paired at 5% discordance	S	Under the stated assumptions
V-131	The pass@k estimator equals enumeration	T	n = 6, c = 2 only; formula from memory
V-132	Exact McNemar for 12 and 3 gives p = 0.035	S	Computed
V-133	The repository's task score carries no item count or interval	S	Observed in the gate structure
V-134	Benchmark names and metrics in Table R1	U	From memory; verify
V-135	The repository's benchmark script reports only measured quantities	U	Its header mentions simulated token generation; contents not audited

R.12 Open Decisions

- Which benchmark families come first. Language modeling is done; knowledge, math, and long context are the natural next three because they test what the paper claims.
- Which existing evaluation tool to bridge, and which benchmarks to run natively.
- The licence of each dataset, and whether bundled or downloaded.
- Whether a composite score is shown at all.
- The compute available for system benchmarks, and on which named hardware.